



UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC
DEPARTAMENTO DE ENGENHARIAS E COMPUTAÇÃO - DEC
ENGENHARIA QUÍMICA

Rod. Jorge Amado, Km 16 — Ilhéus/BA

INTRODUÇÃO AO CONTROLE DE PROCESSOS

Aplicações em Engenharia Química

Prof. Dr. E. R. Edwards

Ilhéus, 25 de setembro de 2026

Sumário

1	Controle de Temperatura em Trocadores de Calor com Computação Científica	1
1.1	Introdução	1
1.2	Descrição do Sistema de Controle	2
1.3	Identificação das Variáveis do Processo	4
1.4	Instrumentação da Malha de Controle de Temperatura	5
1.4.1	Representação da Malha de Controle	6
1.4.2	Interpretação Física da Ação do Controlador	8
1.4.3	Ação Direta e Ação Reversa	8
1.5	Controle Proporcional Aplicado ao Trocador de Calor	9
1.5.1	Da Saída do Controlador à Vazão de Vapor	10
1.5.2	Limites Físicos do Elemento Final de Controle	11
1.5.3	Relação entre a Abertura da Válvula e a Vazão de Vapor	12
1.6	Transferência de Energia para a Corrente de Processo	13
1.6.1	Balanco de Energia Simplificado	14
1.6.2	Interpretação do Sinal da Derivada	15
1.7	Exemplo Resolvido: Análise de uma Condição de Operação	16
1.8	Da Medição Individual ao Conjunto de Dados do Processo	23
1.8.1	Repetição dos Cálculos para as Demais Medições	25
1.8.2	Do Registro Industrial ao Arquivo de Dados	26
1.8.3	Importação e Organização dos Dados Experimentais com Pandas	28
1.8.4	Cálculo Computacional do Erro de Controle	30
1.8.5	Cálculo Computacional da Ação Proporcional	32
1.8.6	Cálculo da Abertura da Válvula	33
1.8.7	Cálculo da Vazão de Vapor	35
1.8.8	Do Controle ao Balanco de Energia	36
1.8.9	Cálculo da Potência Térmica Fornecida pelo Vapor	36
1.8.10	Cálculo da Potência Térmica Associada à Corrente de Processo	38
1.8.11	Cálculo das Perdas Térmicas	39
1.8.12	Construção do Balanco de Energia no DataFrame	40
1.8.13	Cálculo da Taxa de Variação da Temperatura	41
1.8.14	O DataFrame como Representação Computacional do Processo	43

1.8.15	Verificação dos Resultados Computacionais	44
1.9	Visualização da Dinâmica do Processo com Matplotlib	46
1.9.1	Importação da Biblioteca Matplotlib	46
1.9.2	Evolução da Temperatura do Processo	47
1.9.3	Comparação entre a Temperatura Medida e o Setpoint	48
1.9.4	Análise Gráfica do Erro de Controle	49
1.9.5	Análise Gráfica da Ação Proporcional	51
1.9.6	Análise Gráfica da Abertura da Válvula	52
1.9.7	Análise Gráfica da Vazão de Vapor	54
1.9.8	Da Vazão de Vapor à Potência Térmica	55
1.9.9	Comparação das Parcelas do Balanço de Energia	56
1.9.10	Análise da Potência Térmica Líquida	57
1.9.11	Análise da Taxa de Variação da Temperatura	58
1.9.12	Interpretando a Malha de Controle a Partir dos Gráficos	59
1.9.13	Da Tabela ao Gráfico: Duas Formas de Observar o Mesmo Processo .	60
1.9.14	O Papel da Computação Científica na Análise do Processo	61
1.10	Investigando o Comportamento do Processo a partir dos Dados	62
1.10.1	Identificação de uma Condição de Operação	62
1.10.2	Análise da Condição em $t = 7$ min	63
1.10.3	Uma Pergunta Importante sobre a Condição Próxima ao Setpoint . .	65
1.11	Resposta do Sistema diante de uma Perturbação	65
1.11.1	Novo Conjunto de Dados Experimentais	66
1.11.2	Importação dos Dados contendo a Perturbação	67
1.11.3	Interpretação Física da Perturbação	68
1.11.4	Resposta do Controlador Proporcional	68
1.11.5	Aplicação das Equações ao Novo DataFrame	69
1.11.6	Incorporando a Perturbação ao Balanço de Energia	70
1.11.7	Visualização da Perturbação	72
1.11.8	Efeito da Perturbação sobre a Temperatura	72
1.11.9	Efeito da Perturbação sobre o Erro	73
1.11.10	Resposta da Válvula à Perturbação	74
1.12	O Surgimento do Offset no Controle Proporcional	75
1.12.1	Uma Questão para o Próximo Passo	76
1.13	Introdução à Ação Integral	77
1.13.1	O Erro que Permanece ao Longo do Tempo	78
1.13.2	Acumulação Manual do Erro	78
1.13.3	O Controlador Passa a Considerar a História do Erro	80
1.13.4	Da Soma dos Erros à Integral	80
1.13.5	Unidades do Erro Acumulado	81
1.13.6	Transformando o Erro Acumulado em uma Ação de Controle	82

1.13.7	Combinando as Ações Proporcional e Integral	83
1.13.8	Cálculo Manual da Ação PI	84
1.13.9	Efeito da Ação PI sobre a Válvula	86
1.13.10	Por que a Ação Integral Pode Eliminar o Offset?	87
1.13.11	O que Acontece quando o Erro Muda de Sinal?	88
1.13.12	Preparação para a Implementação Computacional	89
1.14	Implementação Computacional da Ação Integral	89
1.14.1	Construção da Coluna do Erro Integral	90
1.14.2	Definição do Intervalo de Amostragem	90
1.14.3	O Primeiro Valor da Integral	91
1.14.4	Por que Não Fazer Todos os Cálculos Manualmente?	92
1.14.5	Construção da Integral com uma Estrutura de Repetição	92
1.14.6	Acompanhando a Construção da Integral	93
1.14.7	Uma Observação Importante sobre o Intervalo de Tempo	93
1.15	Construção Computacional das Ações P e I	94
1.15.1	Cálculo da Parcela Proporcional	95
1.15.2	Cálculo da Parcela Integral	95
1.15.3	Construção da Ação PI	96
1.15.4	Da Ação PI à Abertura da Válvula	96
1.15.5	Da Abertura da Válvula à Vazão de Vapor	97
1.15.6	Lendo o DataFrame como uma Malha de Controle PI	98
1.15.7	Uma Primeira Visualização da Memória do Controlador	99
1.15.8	Comparação entre as Contribuições Proporcional e Integral	99
1.15.9	Visualização da Ação Total do Controlador PI	100
1.15.10	Da Equação ao DataFrame e do DataFrame ao Processo	101
1.16	Da Análise de Dados à Simulação Dinâmica do Processo	102
1.16.1	A Temperatura Deixa de Ser uma Entrada do Programa	102
1.16.2	Balanco de Energia Utilizado na Simulação	103
1.16.3	Da Derivada à Temperatura no Instante Seguinte	104
1.16.4	Uma Observação Importante sobre as Unidades	104
1.17	Simulação do Trocador com Controlador Proporcional	105
1.17.1	Definição do Tempo de Simulação	106
1.17.2	Primeira Etapa da Simulação	106
1.17.3	Cálculo da Nova Temperatura	107
1.17.4	O Ciclo de Cálculo se Repete	109
1.17.5	Implementação Computacional Passo a Passo	109
1.17.6	Criação das Variáveis da Simulação	110
1.17.7	Introdução da Perturbação	111
1.17.8	Construção do Algoritmo do Controlador P	112
1.17.9	Lendo o Algoritmo como uma Malha Física	113

1.17.10	Organização dos Resultados em um DataFrame	114
1.17.11	Visualização da Resposta com Controlador P	115
1.17.12	Uma Pergunta antes de Simular o Controlador PI	116
1.18	Preparação para a Simulação com Controlador PI	117
1.19	Simulação do Trocador com Controlador PI	118
1.19.1	Um Cuidado Importante com as Unidades de Tempo	118
1.19.2	Definição do Ganho Integral	119
1.19.3	Criação das Variáveis do Controlador PI	120
1.19.4	Inicialização da Memória Integral	120
1.19.5	Primeiro Instante: a Ação Proporcional	121
1.19.6	Primeiro Instante: a Memória Integral	121
1.19.7	Como a Memória Será Atualizada	122
1.19.8	Construção do Algoritmo PI	123
1.19.9	Onde Está a Memória no Algoritmo?	125
1.19.10	Finalização dos Valores no Último Instante	125
1.19.11	Organização dos Resultados PI em um DataFrame	126
1.20	Comparação Dinâmica entre os Controladores P e PI	127
1.20.1	Comparação das Temperaturas	127
1.20.2	O que Deve Ser Procurado no Gráfico?	128
1.20.3	Comparação dos Erros	129
1.20.4	Observando a Memória Integral	130
1.20.5	Observando as Parcelas P e I	131
1.20.6	Comparação das Aberturas das Válvulas	132
1.20.7	A Descoberta do Aluno	133
1.21	Quantificando o Desempenho dos Controladores	134
1.21.1	O Erro Final é Sempre o Erro de Regime Permanente?	134
1.22	Compreendendo Fisicamente a Eliminação do Offset	135
1.22.1	Condição de Equilíbrio Térmico	135
1.22.2	Condição de Equilíbrio Antes da Perturbação	136
1.22.3	Qual Vazão de Vapor é Necessária?	137
1.22.4	Qual Deve Ser a Abertura da Válvula?	138
1.22.5	Uma Descoberta sobre a Abertura Nominal	138
1.22.6	O que Acontece quando a Vazão de Processo Aumenta?	139
1.22.7	Nova Vazão de Vapor Necessária	140
1.22.8	Nova Abertura de Equilíbrio da Válvula	140
1.23	Por que o Controlador P Produz Offset?	141
1.23.1	Estimativa Mais Precisa do Offset	142
1.24	Como o Controlador PI Resolve o Problema	143
1.24.1	Quanto de Erro Precisa Ser Acumulado?	145
1.24.2	Interpretação Final	146

1.25	Experiência Computacional de Rejeição de Perturbação	146
1.25.1	Separando Dois Problemas de Controle	147
1.25.2	Condição Inicial da Nova Experiência	148
1.25.3	Aplicação da Perturbação	148
1.25.4	O que Esperamos Fisicamente após a Perturbação?	149
1.26	Rejeição de Perturbação com Controlador P	150
1.26.1	Definição das Condições da Simulação	151
1.26.2	Construção da Perturbação	151
1.26.3	Variáveis que Serão Armazenadas	152
1.26.4	Simulação do Controlador P	152
1.26.5	Organização dos Resultados	154
1.26.6	Observando o Instante da Perturbação no DataFrame	155
1.26.7	Resposta do Controlador P à Perturbação	156
1.26.8	Comparando a Previsão Matemática com a Simulação	157
1.27	Rejeição de Perturbação com Controlador PI	157
1.27.1	Criação das Variáveis PI	158
1.27.2	Simulação do Controlador PI	159
1.27.3	Finalização e Organização dos Resultados	161
1.28	A Experiência P versus PI	163
1.28.1	Observando a Transferência da Ação P para a Ação I	164
1.28.2	A Simulação Confirma a Previsão Física?	165
1.29	Influência do Ganho Proporcional no Comportamento do Processo	167
1.29.1	Interpretação Física do Ganho Proporcional	168
1.29.2	Uma Primeira Previsão antes da Simulação	169
1.30	Experiência Computacional: Diferentes Valores de K_p	169
1.30.1	Por que Alterar Apenas uma Variável?	170
1.30.2	Transformando a Simulação em uma Função	170
1.30.3	Execução das Três Experiências	173
1.30.4	Comparação das Temperaturas	173
1.30.5	Interpretação dos Resultados	174
1.30.6	Quantificando o Offset para Diferentes Valores de K_p	175
1.30.7	Construção de uma Tabela Comparativa	176
1.31	O Surgimento do Problema de Sintonia	177
1.32	Influência do Ganho Integral	178
1.33	Experiência Computacional: Diferentes Valores de K_i	179
1.33.1	Função para Simulação do Controlador PI	179
1.33.2	Execução das Simulações	182
1.33.3	Comparação das Respostas para Diferentes Valores de K_i	183
1.34	Sintonia como um Problema de Compromisso	184

Capítulo 1

Controle de Temperatura em Trocadores de Calor com Computação Científica

1.1 Introdução

Trocadores de calor estão presentes em grande parte dos processos industriais e desempenham uma função fundamental sempre que é necessário aquecer ou resfriar uma corrente de processo. Esses equipamentos são encontrados em indústrias químicas, petroquímicas, alimentícias, farmacêuticas, de petróleo e gás, de papel e celulose, entre muitas outras aplicações.

Embora existam diferentes configurações construtivas, o princípio fundamental de funcionamento de um trocador de calor consiste na transferência de energia térmica entre duas correntes que se encontram a diferentes temperaturas, normalmente sem que ocorra mistura direta entre elas.

Em uma aplicação industrial, entretanto, não é suficiente que o equipamento seja capaz de transferir calor. Frequentemente, é necessário garantir que a corrente de processo deixe o trocador em uma determinada condição de temperatura, mesmo quando ocorrem alterações nas condições de operação.

Considere, por exemplo, uma corrente de processo que precisa ser aquecida antes de seguir para uma etapa posterior de produção. A temperatura desejada na saída do trocador é:

$$T_{SP} = 80^\circ\text{C}.$$

Para realizar o aquecimento, utiliza-se vapor como fluido de serviço. A energia liberada pelo vapor é transferida para a corrente de processo, provocando o aumento de sua temperatura.

Entretanto, durante a operação industrial, diferentes perturbações podem modificar a temperatura de saída do equipamento. A vazão da corrente de processo pode aumentar ou diminuir, sua temperatura de entrada pode variar, a pressão do vapor pode sofrer alterações e as próprias condições de transferência de calor podem mudar ao longo da operação.

Dessa forma, surge um problema típico de Controle de Processos:

Como manter a temperatura de saída da corrente de processo próxima ao valor desejado mesmo na presença de perturbações?

Para responder a essa questão, é necessário medir continuamente a temperatura de saída, comparar o valor medido com o setpoint e atuar sobre uma variável capaz de modificar a quantidade de energia fornecida ao processo.

1.2 Descrição do Sistema de Controle

Considere um trocador de calor utilizado para aquecer uma corrente líquida de processo. A corrente entra no equipamento com temperatura T_{in} e deixa o trocador com temperatura T .

O aquecimento é realizado utilizando vapor. A vazão de vapor que entra no trocador pode ser modificada por meio de uma válvula de controle instalada na linha de vapor. Após transferir energia térmica para a corrente de processo, o vapor condensa e deixa o equipamento como condensado.

A Figura 1.1 apresenta uma visão geral do sistema físico e da malha de controle de temperatura que serão utilizados como referência ao longo deste capítulo.

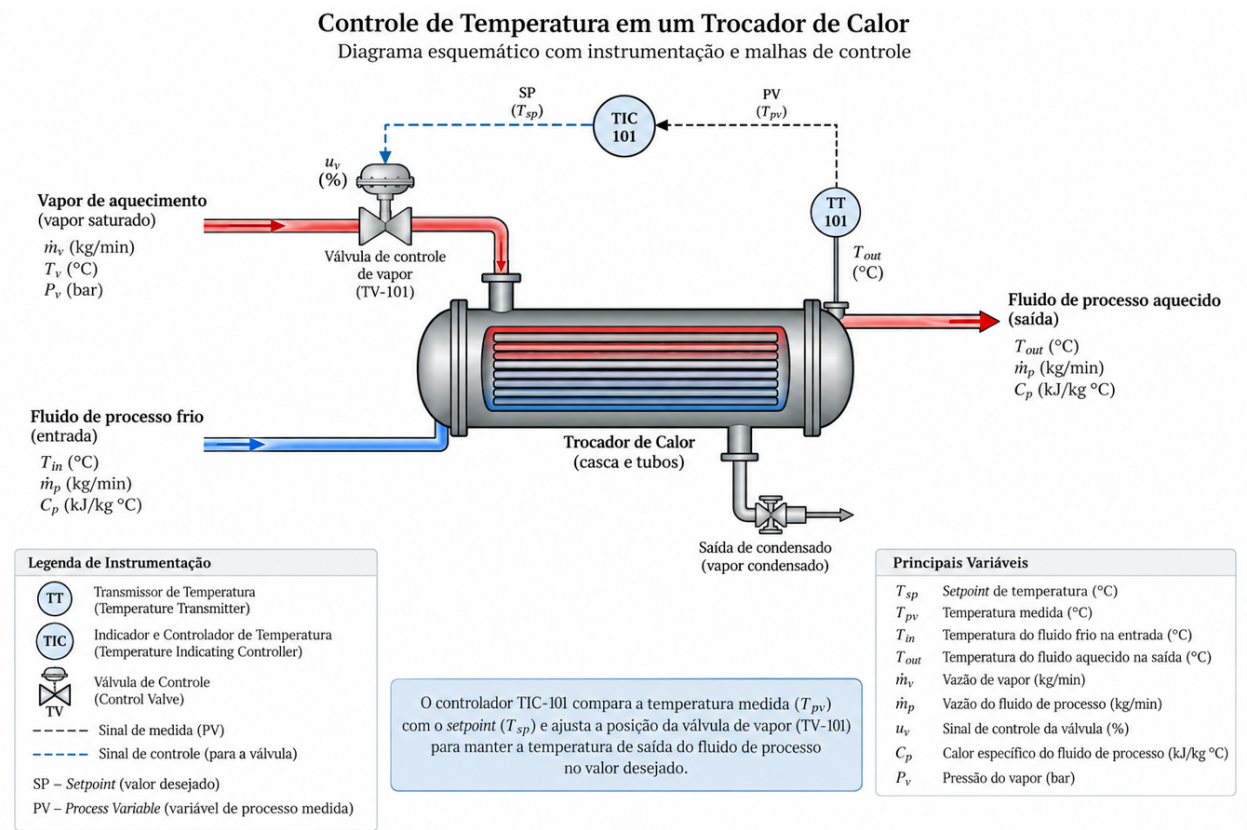


Figura 1.1: Representação esquemática do trocador de calor e da malha de controle da temperatura de saída da corrente de processo.

Na Figura 1.1, a corrente fria entra no trocador e recebe energia térmica do vapor. A temperatura da corrente na saída do equipamento é medida pelo transmissor de temperatura **TT-101** e enviada ao controlador de temperatura **TIC-101**.

Essa temperatura corresponde à variável de processo medida e será representada por:

$$T_{PV}.$$

O valor desejado para essa temperatura corresponde ao setpoint:

$$T_{SP} = 80^\circ\text{C}.$$

O controlador compara continuamente a temperatura medida com o valor desejado. O erro de controle é definido por:

$$e_k = T_{SP} - T_{PV,k}.$$

A partir desse erro, o controlador determina a ação necessária sobre a válvula de controle de vapor **TV-101**. A alteração da abertura da válvula modifica a vazão de vapor $\dot{m}_v$ admitida no trocador e, conseqüentemente, a quantidade de energia térmica fornecida à corrente de processo.

Quando a temperatura medida está abaixo do setpoint, o erro é positivo. Para a estratégia de controle considerada neste capítulo, o controlador deve aumentar sua ação sobre a válvula de vapor, permitindo maior fornecimento de energia térmica ao processo.

Quando a temperatura se aproxima do setpoint, o erro diminui e a ação corretiva necessária também tende a diminuir.

A Figura 1.1 permite, portanto, acompanhar fisicamente a sequência de atuação da malha:

$$\boxed{T_{PV} \longrightarrow \text{TIC-101} \longrightarrow \text{TV-101} \longrightarrow \dot{m}_v \longrightarrow \text{Trocador de Calor} \longrightarrow T}$$

Essa sequência não deve ser interpretada como uma cadeia aberta. A temperatura de saída é continuamente medida pelo **TT-101** e realimentada ao controlador. Assim, o comportamento completo da malha pode ser resumido por:

$$\boxed{T_{SP} \longrightarrow \text{TIC-101} \longrightarrow \text{TV-101} \longrightarrow \dot{m}_v \longrightarrow \text{Processo} \longrightarrow T_{PV}}$$

com T_{PV} retornando continuamente ao controlador por meio da realimentação.

Essa representação será utilizada ao longo do capítulo para relacionar os elementos físicos do trocador de calor às variáveis de processo, às equações do modelo, às ações de controle e, posteriormente, à implementação computacional da malha.

1.3 Identificação das Variáveis do Processo

Antes de realizar qualquer cálculo ou desenvolver um programa computacional, é importante identificar claramente as principais variáveis envolvidas no problema de Engenharia.

Neste sistema, a variável que se deseja manter próxima de um determinado valor é a temperatura de saída da corrente de processo. Portanto, a variável controlada é:

$$\boxed{\text{Variável controlada: } T_{PV}}$$

A variável que pode ser modificada para alterar a quantidade de energia fornecida ao processo é a vazão de vapor:

$$\boxed{\text{Variável manipulada: } \dot{m}_v}$$

A abertura da válvula de controle determina a passagem de vapor e, conseqüentemente, influencia a quantidade de energia térmica transferida para a corrente de processo.

Entre as principais perturbações que podem afetar o sistema encontram-se:

$$\boxed{\dot{m}_{in}, \quad T_{in}, \quad P_v}$$

em que $\dot{m}_{in}$ representa a vazão da corrente de processo, T_{in} sua temperatura de entrada e P_v a pressão do vapor.

Essa identificação é fundamental porque permite separar três conceitos diferentes:

- **variável controlada:** grandeza que se deseja manter próxima ao valor especificado;
- **variável manipulada:** grandeza modificada pelo sistema de controle para influenciar o processo;
- **perturbações:** variáveis que podem alterar o comportamento do processo sem serem diretamente determinadas pelo controlador.

No sistema considerado, a lógica física pode ser resumida por:

$$\boxed{\text{abertura da válvula} \longrightarrow \dot{m}_v \longrightarrow \text{energia transferida} \longrightarrow T_{PV}}$$

Portanto, quando o controlador modifica sua saída, ele não modifica diretamente a temperatura. Sua ação ocorre inicialmente sobre o elemento final de controle, que altera a vazão de vapor. A modificação da vazão provoca uma alteração na transferência de energia e, somente então, a temperatura do processo responde.

Essa sequência é particularmente importante para compreender fisicamente uma malha de controle industrial.

1.4 Instrumentação da Malha de Controle de Temperatura

Para que a temperatura de saída do trocador de calor possa ser controlada, é necessário que o sistema seja capaz de medir essa variável, comparar seu valor com a temperatura desejada e modificar a quantidade de energia térmica fornecida ao processo.

Em uma instalação industrial, essas funções são realizadas por instrumentos que atuam de maneira integrada, formando uma malha fechada de controle.

Considere novamente o trocador de calor utilizado para aquecer uma corrente de processo. A temperatura na saída do equipamento é medida continuamente por um elemento sensor associado a um transmissor de temperatura. O transmissor converte a medição em um sinal apropriado e o envia ao controlador de temperatura.

Na representação de instrumentação, podemos identificar os principais elementos da malha por:

$$\boxed{\text{TT}} \longrightarrow \boxed{\text{TIC}} \longrightarrow \boxed{\text{TV}}$$

em que:

- TT representa o transmissor de temperatura (*Temperature Transmitter*);
- TIC representa o controlador indicador de temperatura (*Temperature Indicating Controller*);
- TV representa a válvula de controle associada à malha de temperatura (*Temperature Valve*).

O transmissor fornece ao controlador a informação correspondente à temperatura medida na saída do trocador:

$$T_{PV}.$$

O controlador recebe também o valor desejado para a temperatura, definido pelo operador ou pelo sistema de supervisão:

$$T_{SP}.$$

A partir dessas duas informações, o controlador calcula o erro:

$$e(t) = T_{SP} - T_{PV}(t).$$

O sinal de saída do controlador é então enviado ao elemento final de controle. Neste exemplo, esse elemento é uma válvula instalada na linha de vapor.

Consequentemente, a atuação da malha pode ser representada pela sequência:

$$T_{PV} \longrightarrow e \longrightarrow u \longrightarrow \text{válvula} \longrightarrow \dot{m}_v \longrightarrow Q \longrightarrow T.$$

Essa representação evidencia um aspecto fundamental do Controle de Processos: o controlador não modifica diretamente a variável controlada. Sua saída u atua sobre um elemento físico do processo que, por sua vez, modifica outra grandeza capaz de influenciar a variável que se deseja controlar.

No trocador de calor, o controlador modifica a abertura da válvula. A abertura da válvula altera a vazão de vapor $\dot{m}_v$, que modifica a quantidade de energia térmica transferida para a corrente de processo. Como consequência, a temperatura de saída T_{PV} é alterada.

1.4.1 Representação da Malha de Controle

A representação física apresentada anteriormente permite identificar o trocador de calor, as correntes de processo, os instrumentos e a válvula de controle. Para compreender como as informações circulam entre esses elementos, é conveniente representar a mesma malha por meio de um **diagrama de blocos**.

Nesse tipo de representação, o interesse principal deixa de estar na geometria do equipamento e passa a estar nas relações entre a variável de referência, o erro, o controlador, o elemento final de controle, o processo e o sistema de medição.

A Figura 1.2 apresenta o diagrama de blocos correspondente à malha de controle de temperatura do trocador de calor.

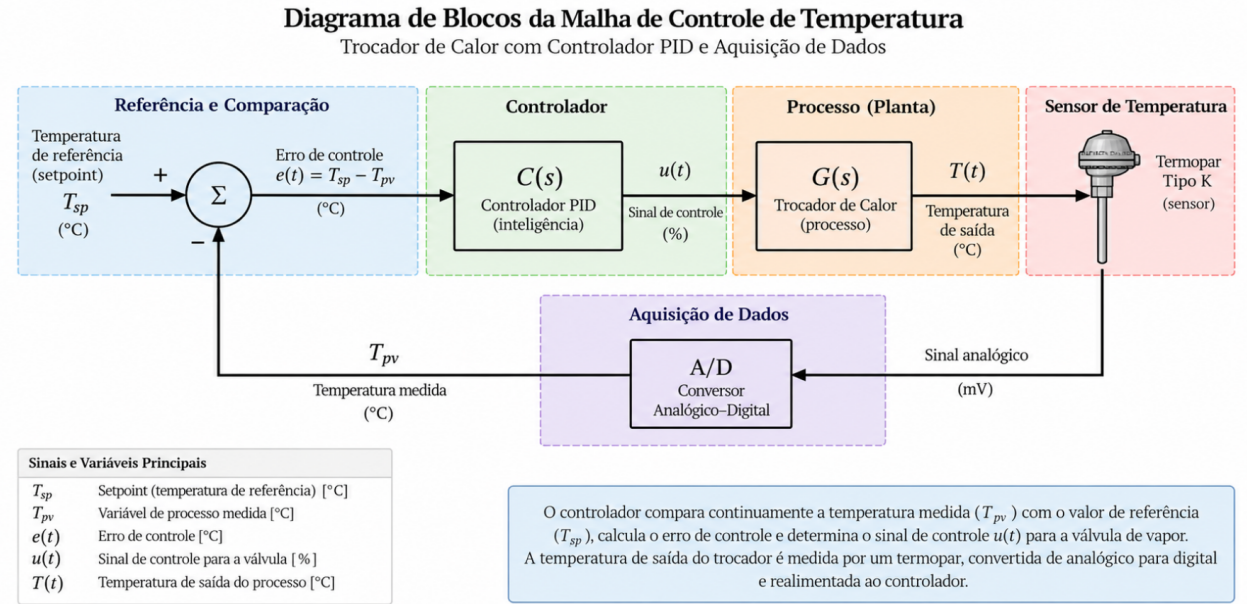


Figura 1.2: Diagrama de blocos da malha de controle de temperatura do trocador de calor, incluindo controlador, processo, medição e realimentação.

A leitura do diagrama pode ser iniciada pela temperatura de referência T_{SP} , que representa o valor desejado para a temperatura de saída do processo. Esse sinal entra no ponto

de soma com sinal positivo.

Pela malha de realimentação retorna a temperatura medida do processo, representada por T_{PV} , que entra no mesmo ponto de soma com sinal negativo. A diferença entre esses dois sinais produz o erro de controle:

$$e(t) = T_{SP} - T_{PV}(t).$$

O erro $e(t)$ é enviado ao controlador, elemento responsável por determinar a ação necessária para aproximar a variável de processo do valor desejado. Ao longo deste capítulo, esse bloco será utilizado para representar diferentes estratégias de controle, inicialmente proporcional e, posteriormente, proporcional-integral e proporcional-integral-derivativa.

A saída do controlador corresponde ao sinal de controle:

$$u(t).$$

No sistema físico estudado, esse sinal atua sobre a válvula de controle **TV-101**, instalada na linha de vapor. A modificação da posição da válvula altera a vazão de vapor $\dot{m}_v$ fornecida ao trocador e, conseqüentemente, a quantidade de energia térmica transferida à corrente de processo.

O trocador de calor constitui a **planta**, ou **processo**, da malha de controle. Sua resposta é observada por meio da temperatura de saída $T(t)$.

Essa temperatura é medida pelo sistema de instrumentação. No exemplo considerado, a medição pode ser realizada por um termopar, cujo sinal elétrico é posteriormente condicionado e adquirido pelo sistema de medição. Em uma implementação computacional, a conversão analógico-digital permite que a informação medida seja representada numericamente e utilizada pelo controlador como variável de processo:

$$T_{PV}.$$

O valor de T_{PV} retorna ao ponto de soma, onde é novamente comparado com T_{SP} , fechando a malha de realimentação.

Assim, o fluxo de informações e ações pode ser sintetizado por:

$$T_{SP} \rightarrow e(t) \rightarrow \text{Controlador} \rightarrow u(t) \rightarrow \text{TV-101} \rightarrow \dot{m}_v \rightarrow \text{Processo} \rightarrow T(t)$$

seguido pela medição e realimentação:

$$T(t) \rightarrow \text{Sensor} \rightarrow \text{Aquisição} \rightarrow T_{PV} \rightarrow \text{Ponto de Soma}.$$

A malha fechada resulta, portanto, da repetição contínua desse ciclo. O controlador não atua apenas uma vez: a temperatura é continuamente medida, comparada com o setpoint e utilizada para calcular uma nova ação de controle.

1.4.2 Interpretação Física da Ação do Controlador

Para compreender o funcionamento da malha, considere inicialmente uma situação em que a temperatura medida seja inferior ao valor desejado. Por exemplo:

$$T_{SP} = 80^{\circ}\text{C}$$

e

$$T_{PV} = 70^{\circ}\text{C}.$$

O erro de controle será:

$$e = 80 - 70 = 10^{\circ}\text{C}.$$

Como o processo necessita de maior aquecimento, o controlador deve produzir uma ação que resulte em maior fornecimento de vapor ao trocador.

Fisicamente, a sequência esperada é:

$$T_{PV} < T_{SP} \implies e > 0 \implies \text{maior fornecimento de vapor} \implies T_{PV} \uparrow.$$

À medida que a temperatura de saída aumenta e se aproxima do setpoint, o erro diminui. Por exemplo, se posteriormente for medida:

$$T_{PV} = 78^{\circ}\text{C},$$

teremos:

$$e = 80 - 78 = 2^{\circ}\text{C}.$$

Portanto, a necessidade de correção é menor.

Essa análise mostra que o controlador deve modificar continuamente sua saída em função do estado do processo. A temperatura não assume instantaneamente o valor do setpoint, pois o trocador apresenta capacidade térmica e necessita de determinado intervalo de tempo para responder às alterações na quantidade de energia fornecida.

1.4.3 Ação Direta e Ação Reversa

Neste ponto é necessário observar uma característica importante dos controladores industriais: o sentido da ação do controlador deve ser compatível com o comportamento físico do processo e com o funcionamento do elemento final de controle.

No sistema considerado, quando a temperatura de saída diminui abaixo do valor desejado, é necessário aumentar o fornecimento de energia térmica. Para isso, a válvula deve permitir maior passagem de vapor.

De maneira física, deseja-se obter o seguinte comportamento:

$$T_{PV} \downarrow \implies \dot{m}_v \uparrow \implies Q \uparrow \implies T_{PV} \uparrow.$$

Da mesma forma, se a temperatura aumentar excessivamente, o sistema deverá reduzir o fornecimento de vapor:

$$T_{PV} \uparrow \implies \dot{m}_v \downarrow \implies Q \downarrow.$$

O sentido necessário para o sinal do controlador depende também da forma como a válvula responde ao sinal recebido. Uma válvula pode, por exemplo, aumentar sua abertura quando o sinal de controle aumenta ou apresentar comportamento inverso, dependendo de sua configuração e de sua condição de segurança.

Por esse motivo, a definição da ação direta ou reversa de um controlador não deve ser feita apenas observando o sinal matemático do erro. É necessário analisar conjuntamente o processo, o controlador e o elemento final de controle.

O objetivo fundamental é garantir uma realimentação negativa: qualquer desvio da variável controlada deve provocar uma ação que conduza o processo novamente em direção ao setpoint.

1.5 Controle Proporcional Aplicado ao Trocador de Calor

Uma vez compreendida a estrutura física da malha, podemos aplicar ao trocador de calor o controlador proporcional estudado anteriormente.

Para cada instante de amostragem k , o erro é calculado por:

$$e_k = T_{SP} - T_{PV,k}.$$

A ação proporcional será representada por:

$$u_k = K_p e_k,$$

em que K_p representa o ganho proporcional do controlador.

Essa equação mostra que a intensidade da ação corretiva depende diretamente da magnitude do erro. Quanto maior a diferença entre a temperatura desejada e a temperatura medida, maior será a ação proporcional.

Considere, inicialmente:

$$K_p = 2\%/^{\circ}\text{C}.$$

Nesse caso, para:

$$T_{SP} = 80^{\circ}\text{C}$$

e

$$T_{PV} = 70^{\circ}\text{C},$$

o erro será:

$$e = 10^{\circ}\text{C}.$$

Consequentemente:

$$u = K_p e$$

e:

$$u = \left(2 \frac{\%}{^{\circ}\text{C}}\right) (10^{\circ}\text{C}) = 20 \%.$$

O resultado indica uma ação proporcional de 20%.

É importante observar as unidades envolvidas. Como o erro está expresso em graus Celsius e a saída u está sendo representada em porcentagem, o ganho proporcional deve possuir unidades compatíveis:

$$[K_p] = \frac{\%}{^{\circ}\text{C}}.$$

Portanto:

$$\left[\frac{\%}{^{\circ}\text{C}}\right] [^{\circ}\text{C}] = [\%].$$

Essa análise dimensional é particularmente importante quando se trabalha com diferentes representações da saída de controladores industriais.

Neste capítulo, inicialmente utilizaremos a saída do controlador em porcentagem para facilitar sua interpretação física. Posteriormente, essa saída será relacionada à abertura da válvula e à vazão de vapor fornecida ao trocador.

1.5.1 Da Saída do Controlador à Vazão de Vapor

A equação do controlador proporcional fornece uma ação de controle u . Entretanto, para compreender o comportamento do sistema, é necessário relacionar essa variável matemática a uma ação física sobre o processo.

No trocador de calor considerado, o elemento final de controle é uma válvula instalada na linha de vapor. Dessa forma, a saída do controlador deve produzir uma alteração na posição da válvula e, consequentemente, na vazão de vapor fornecida ao equipamento.

Para representar essa relação de maneira simples, considere inicialmente que a válvula opera em torno de uma condição nominal de abertura u_0 . A ação proporcional calculada pelo controlador representa uma correção em relação a essa condição de operação.

Assim, a abertura da válvula pode ser representada por:

$$u_{v,k} = u_0 + u_k$$

em que:

- $u_{v,k}$ representa a abertura da válvula no instante k , em %;
- u_0 representa a abertura da válvula na condição nominal de operação, em %;
- u_k representa a correção determinada pelo controlador, em %.

Como:

$$u_k = K_p e_k,$$

temos:

$$u_{v,k} = u_0 + K_p e_k.$$

Essa expressão permite interpretar fisicamente a ação proporcional.

Considere, por exemplo:

$$u_0 = 40\%$$

e o resultado obtido anteriormente:

$$u_k = 20\%.$$

A abertura determinada para a válvula será:

$$u_{v,k} = 40 + 20 = 60\%.$$

Portanto, o controlador não está indicando simplesmente que a válvula deve operar com abertura de 20%. A ação proporcional de 20% representa, neste modelo, uma correção em relação à condição nominal de operação.

Esse conceito é importante porque um processo industrial normalmente apresenta uma condição de operação na qual determinada quantidade de energia já precisa ser fornecida mesmo quando o erro é pequeno ou nulo.

1.5.2 Limites Físicos do Elemento Final de Controle

Embora a equação matemática possa fornecer diferentes valores para $u_{v,k}$, uma válvula real possui limites físicos de operação.

Considerando a abertura expressa em porcentagem:

$$0\% \leq u_{v,k} \leq 100\%.$$

Portanto, se o cálculo fornecer:

$$u_{v,k} > 100\%,$$

a válvula não poderá ultrapassar sua abertura máxima. Da mesma forma, se:

$$u_{v,k} < 0\%,$$

a válvula não poderá assumir uma abertura fisicamente negativa.

De maneira simplificada, podemos representar essa restrição por:

$$u_{v,k} = \begin{cases} 0\%, & u_{v,k} < 0\%, \\ u_{v,k}, & 0\% \leq u_{v,k} \leq 100\%, \\ 100\%, & u_{v,k} > 100\%. \end{cases}$$

Esse fenômeno é denominado **saturação do elemento final de controle**.

A existência desses limites mostra uma diferença importante entre um modelo matemático e um sistema físico. A equação do controlador pode produzir qualquer valor numérico, mas o equipamento responsável por executar a ação possui restrições operacionais.

Posteriormente, quando os cálculos forem realizados computacionalmente, esses limites poderão ser incorporados ao programa.

1.5.3 Relação entre a Abertura da Válvula e a Vazão de Vapor

A alteração da abertura da válvula modifica a quantidade de vapor que entra no trocador de calor.

Em uma instalação industrial real, a relação entre abertura da válvula e vazão pode depender de diferentes fatores, incluindo a característica da válvula, a diferença de pressão disponível, a densidade do fluido e o coeficiente de vazão.

Neste primeiro modelo, entretanto, será adotada uma relação simplificada entre a abertura da válvula e a vazão de vapor. Essa simplificação permitirá concentrar a análise na lógica da malha de controle.

Considere que, nas condições estudadas, a vazão de vapor possa ser aproximada por:

$$\dot{m}_{v,k} = K_v u_{v,k},$$

em que K_v representa um coeficiente simplificado que relaciona a abertura da válvula à vazão de vapor.

Adotando:

$$K_v = 0,01 \frac{\text{kg/min}}{\%},$$

uma abertura de:

$$u_{v,k} = 60\%$$

resultará em:

$$\dot{m}_{v,k} = \left(0,01 \frac{\text{kg/min}}{\%}\right) (60\%)$$

e, portanto:

$$\dot{m}_{v,k} = 0,60 \text{ kg/min.}$$

A sequência física pode agora ser ampliada:

$$T_{PV} \longrightarrow e_k \longrightarrow u_k \longrightarrow u_{v,k} \longrightarrow \dot{m}_{v,k}.$$

Observe que cada etapa possui um significado específico. A temperatura é medida pelo instrumento, o erro é calculado pelo controlador, a ação proporcional é determinada pela lei de controle, a posição da válvula é modificada e, finalmente, ocorre uma alteração na vazão de vapor.

1.6 Transferência de Energia para a Corrente de Processo

A vazão de vapor, por si só, ainda não explica diretamente a variação da temperatura da corrente de processo. Para completar a análise, é necessário relacionar a quantidade de vapor fornecida ao trocador com a energia térmica transferida.

Considere que o vapor entre no trocador e condense durante o processo de aquecimento. Em uma representação simplificada, a taxa de energia disponibilizada pela condensação pode ser estimada por:

$$\dot{Q}_{v,k} = \dot{m}_{v,k} \lambda_v,$$

em que λ_v representa a entalpia específica associada à condensação do vapor.

É necessário, entretanto, manter consistência entre as unidades utilizadas.

Como a vazão de vapor está sendo expressa em kg/min e a potência térmica será expressa em watts, a vazão deve ser convertida para kg/s:

$$\dot{m}_{v,k}^* = \frac{\dot{m}_{v,k}}{60}.$$

Consequentemente:

$$\dot{Q}_{v,k} = \frac{\dot{m}_{v,k}}{60} \lambda_v.$$

Considere, para fins didáticos:

$$\lambda_v = 2,0 \times 10^6 \text{ J/kg}.$$

Para:

$$\dot{m}_{v,k} = 0,60 \text{ kg/min},$$

temos:

$$\dot{m}_{v,k}^* = \frac{0,60}{60} = 0,010 \text{ kg/s}.$$

Assim:

$$\dot{Q}_{v,k} = (0,010) (2,0 \times 10^6),$$

resultando em:

$$\dot{Q}_{v,k} = 20\,000 \text{ W}.$$

Portanto:

$$\boxed{\dot{Q}_{v,k} = 20 \text{ kW}}.$$

Esse resultado representa a taxa de energia térmica disponibilizada pelo vapor de acordo com as hipóteses simplificadoras adotadas.

1.6.1 Balanço de Energia Simplificado

Para relacionar a energia fornecida pelo vapor à evolução da temperatura do processo, podemos utilizar um balanço de energia.

De maneira geral, a estrutura fundamental de um balanço pode ser representada por:

$$\text{Acúmulo} = \text{Entrada} - \text{Saída}.$$

Para uma representação concentrada do comportamento térmico do sistema, considere uma capacidade térmica equivalente C , expressa em J/°C.

A dinâmica térmica pode ser representada de maneira simplificada por:

$$C \frac{dT}{dt} = \dot{Q}_v - \dot{Q}_{\text{processo}} - \dot{Q}_{\text{perdas}}.$$

O termo $\dot{Q}_v$ representa a energia fornecida pelo vapor, enquanto $\dot{Q}_{\text{processo}}$ representa a energia utilizada para aquecer continuamente a corrente que atravessa o equipamento. O

termo $\dot{Q}_{perdas}$ representa as perdas térmicas para o ambiente.

A energia necessária para aquecer a corrente de processo pode ser aproximada por:

$$\dot{Q}_{processo} = \dot{m}_{in} c_p (T - T_{in}),$$

desde que a vazão mássica seja expressa em unidades consistentes.

As perdas térmicas podem ser representadas, em uma primeira aproximação, por:

$$\dot{Q}_{perdas} = U_L (T - T_{amb}),$$

em que U_L representa um coeficiente global simplificado de perda térmica para o ambiente.

Substituindo essas relações no balanço:

$$C \frac{dT}{dt} = \dot{Q}_v - \dot{m}_{in} c_p (T - T_{in}) - U_L (T - T_{amb}).$$

Essa equação estabelece a conexão entre a ação do controlador e a dinâmica térmica do processo.

1.6.2 Interpretação do Sinal da Derivada

A partir do balanço de energia, podemos definir a potência térmica líquida:

$$\dot{Q}_{liq} = \dot{Q}_v - \dot{Q}_{processo} - \dot{Q}_{perdas}.$$

Assim:

$$C \frac{dT}{dt} = \dot{Q}_{liq}$$

e:

$$\frac{dT}{dt} = \frac{\dot{Q}_{liq}}{C}.$$

O sinal dessa derivada fornece uma informação direta sobre a tendência da temperatura.

Se:

$$\dot{Q}_{liq} > 0,$$

então:

$$\frac{dT}{dt} > 0,$$

e a temperatura está aumentando.

Se:

$$\dot{Q}_{liq} < 0,$$

então:

$$\frac{dT}{dt} < 0,$$

e a temperatura está diminuindo.

Finalmente, se:

$$\dot{Q}_{liq} = 0,$$

temos:

$$\frac{dT}{dt} = 0.$$

Nesse instante, não existe tendência de aumento ou diminuição da temperatura segundo o modelo considerado.

A cadeia completa do problema de Engenharia pode agora ser representada por:

$$T_{PV} \rightarrow e \rightarrow u \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v \rightarrow \dot{Q}_{liq} \rightarrow \frac{dT}{dt} \rightarrow T$$

Essa sequência será particularmente importante quando forem analisadas várias medições consecutivas da temperatura do trocador de calor.

1.7 Exemplo Resolvido: Análise de uma Condição de Operação

Antes de analisar um conjunto contendo várias medições do processo, será considerada uma única condição de operação do trocador de calor. O objetivo é percorrer manualmente todas as etapas que relacionam a temperatura medida pelo instrumento à ação do controlador e, posteriormente, à resposta térmica do processo.

Essa etapa é importante porque os mesmos cálculos que serão posteriormente realizados com auxílio da Computação Científica devem, inicialmente, ser compreendidos do ponto de vista físico e matemático.

Considere um trocador de calor utilizado para aquecer uma corrente líquida de processo. A temperatura desejada na saída do equipamento é:

$$T_{SP} = 80^\circ\text{C}.$$

Em determinado instante de operação, o transmissor de temperatura registra:

$$T_{PV} = 70^\circ\text{C}.$$

O controlador proporcional possui ganho:

$$K_p = 2 \frac{\%}{^{\circ}\text{C}},$$

e a válvula de vapor opera em torno de uma abertura nominal de:

$$u_0 = 40\%.$$

Para o modelo simplificado da válvula, considere:

$$K_v = 0,01 \frac{\text{kg}/\text{min}}{\%}.$$

A entalpia específica associada à condensação do vapor será aproximada por:

$$\lambda_v = 2,0 \times 10^6 \text{ J/kg}.$$

Para a corrente de processo e para o comportamento térmico do equipamento, considere ainda os seguintes dados:

$$\dot{m}_{in} = 3,0 \text{ kg/min},$$

$$c_p = 4000 \text{ J/(kg } ^{\circ}\text{C)},$$

$$T_{in} = 30 ^{\circ}\text{C},$$

$$T_{amb} = 25 ^{\circ}\text{C},$$

$$U_L = 20 \text{ W}/^{\circ}\text{C},$$

e:

$$C = 1,0 \times 10^5 \text{ J}/^{\circ}\text{C}.$$

Determine o erro de controle, a ação proporcional, a abertura da válvula, a vazão de vapor, a potência térmica fornecida pelo vapor, a energia utilizada para aquecer a corrente de processo, as perdas térmicas, a potência térmica líquida e a taxa instantânea de variação da temperatura.

Ao final, interprete fisicamente a condição de operação do trocador.

Solução

1. Cálculo do erro de controle

O primeiro passo consiste em comparar a temperatura desejada com a temperatura efetivamente medida pelo transmissor.

O erro é definido por:

$$e = T_{SP} - T_{PV}.$$

Substituindo os valores:

$$e = 80 - 70,$$

portanto:

$$\boxed{e = 10^{\circ}\text{C}}.$$

O erro positivo indica que a temperatura medida está abaixo do valor desejado. Portanto, o processo necessita de maior fornecimento de energia térmica.

2. Cálculo da ação proporcional

Para o controlador proporcional:

$$u = K_p e.$$

Substituindo:

$$u = \left(2 \frac{\%}{^{\circ}\text{C}}\right) (10^{\circ}\text{C}),$$

obtém-se:

$$\boxed{u = 20\%}.$$

Essa é a correção calculada pelo controlador em relação à condição nominal de operação da válvula.

3. Determinação da abertura da válvula

A abertura da válvula é calculada por:

$$u_v = u_0 + u.$$

Assim:

$$u_v = 40 + 20,$$

resultando em:

$$u_v = 60\%.$$

Como o valor se encontra entre 0% e 100%, não ocorre saturação da válvula nessa condição de operação.

4. Cálculo da vazão de vapor

A relação simplificada entre a abertura da válvula e a vazão de vapor é:

$$\dot{m}_v = K_v u_v.$$

Portanto:

$$\dot{m}_v = \left(0,01 \frac{\text{kg/min}}{\%} \right) (60\%),$$

resultando em:

$$\dot{m}_v = 0,60 \text{ kg/min}.$$

Antes de calcular a potência térmica, a vazão deve ser convertida para kg/s:

$$\dot{m}_v^* = \frac{0,60}{60}.$$

Logo:

$$\dot{m}_v^* = 0,010 \text{ kg/s}.$$

5. Potência térmica fornecida pelo vapor

A potência térmica disponibilizada pela condensação do vapor é calculada por:

$$\dot{Q}_v = \dot{m}_v^* \lambda_v.$$

Substituindo:

$$\dot{Q}_v = (0,010) (2,0 \times 10^6),$$

obtem-se:

$$\dot{Q}_v = 20\,000 \text{ W}.$$

Ou:

$$\dot{Q}_v = 20,0 \text{ kW}.$$

6. Energia utilizada para aquecer a corrente de processo

A corrente de processo entra no trocador com vazão:

$$\dot{m}_{in} = 3,0 \text{ kg/min.}$$

Como a potência será calculada em watts, inicialmente é necessário converter essa vazão para kg/s:

$$\dot{m}_{in}^* = \frac{3,0}{60} = 0,050 \text{ kg/s.}$$

A potência térmica associada ao aquecimento da corrente é:

$$\dot{Q}_{processo} = \dot{m}_{in}^* c_p (T_{PV} - T_{in}).$$

Substituindo os valores:

$$\dot{Q}_{processo} = (0,050)(4000)(70 - 30).$$

Portanto:

$$\dot{Q}_{processo} = 8000 \text{ W}.$$

Ou:

$$\dot{Q}_{processo} = 8,0 \text{ kW}.$$

7. Cálculo das perdas térmicas

As perdas térmicas para o ambiente são aproximadas por:

$$\dot{Q}_{perdas} = U_L (T_{PV} - T_{amb}).$$

Substituindo:

$$\dot{Q}_{perdas} = 20(70 - 25).$$

Assim:

$$\dot{Q}_{perdas} = 900 \text{ W}.$$

Ou:

$$\dot{Q}_{perdas} = 0,90 \text{ kW}.$$

8. Cálculo da potência térmica líquida

A potência térmica líquida disponível para provocar a variação da temperatura do sistema é:

$$\dot{Q}_{liq} = \dot{Q}_v - \dot{Q}_{processo} - \dot{Q}_{perdas}.$$

Substituindo:

$$\dot{Q}_{liq} = 20\,000 - 8000 - 900.$$

Portanto:

$$\dot{Q}_{liq} = 11\,100 \text{ W}.$$

Ou:

$$\dot{Q}_{liq} = 11,1 \text{ kW}.$$

Como:

$$\dot{Q}_{liq} > 0,$$

há acúmulo positivo de energia no sistema.

9. Taxa instantânea de variação da temperatura

A dinâmica térmica simplificada é descrita por:

$$C \frac{dT}{dt} = \dot{Q}_{liq}.$$

Portanto:

$$\frac{dT}{dt} = \frac{\dot{Q}_{liq}}{C}.$$

Substituindo:

$$\frac{dT}{dt} = \frac{11\,100}{1,0 \times 10^5}.$$

Logo:

$$\frac{dT}{dt} = 0,111 \text{ °C/s}$$

ou, convertendo para minutos:

$$\frac{dT}{dt} = 0,111(60),$$

resultando em:

$$\frac{dT}{dt} = 6,66^\circ\text{C}/\text{min}.$$

10. Interpretação física do resultado

O resultado obtido mostra que:

$$T_{PV} < T_{SP},$$

o que produz um erro positivo. O controlador proporcional responde aumentando sua ação sobre a válvula de vapor. A abertura da válvula aumenta, permitindo maior vazão de vapor e, conseqüentemente, maior fornecimento de energia térmica ao processo.

Para a condição analisada:

$$\dot{Q}_v > \dot{Q}_{processo} + \dot{Q}_{perdas},$$

de modo que:

$$\dot{Q}_{liq} > 0.$$

Conseqüentemente:

$$\frac{dT}{dt} > 0.$$

Portanto, no instante analisado, a temperatura do trocador apresenta tendência de aumento.

A sequência completa dos cálculos realizados pode ser resumida por:

$$70^\circ\text{C} \rightarrow 10^\circ\text{C} \rightarrow 20\% \rightarrow 60\% \rightarrow 0,60\text{ kg/min} \rightarrow 20,0\text{ kW} \rightarrow 11,1\text{ kW} \rightarrow 6,66^\circ\text{C/min}$$

ou, em termos das variáveis:

$$T_{PV} \rightarrow e \rightarrow u \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v \rightarrow \dot{Q}_{liq} \rightarrow \frac{dT}{dt}$$

Esse encadeamento mostra que uma alteração na temperatura medida pelo instrumento percorre toda a malha de controle até produzir uma alteração física na quantidade de energia fornecida ao processo.

1.8 Da Medição Individual ao Conjunto de Dados do Processo

No exemplo anterior, foi analisada uma única condição de operação do trocador de calor. A partir de uma temperatura medida pelo transmissor, foi possível calcular manualmente o erro de controle, a ação proporcional, a abertura da válvula, a vazão de vapor e as diferentes parcelas envolvidas no balanço de energia.

Em uma instalação industrial, entretanto, a temperatura não é medida apenas uma vez. O transmissor realiza medições sucessivas ao longo da operação e essas informações podem ser registradas por um sistema de aquisição de dados, por um controlador, por um CLP ou por um sistema supervisor.

Considere que, durante a operação do trocador de calor, a temperatura de saída tenha sido registrada a cada minuto. O setpoint permanece constante em:

$$T_{SP} = 80^\circ\text{C}.$$

As primeiras medições obtidas são apresentadas na Tabela 1.1.

Tabela 1.1: Medições da temperatura de saída do trocador de calor.

Tempo (min)	T_{PV} ($^\circ\text{C}$)
0	70,0
1	72,4
2	74,3
3	75,8
4	77,0
5	77,9
6	78,6
7	79,1
8	79,4
9	79,6

A primeira linha da tabela corresponde exatamente à condição analisada no exemplo anterior:

$$t = 0 \text{ min}, \quad T_{PV} = 70^\circ\text{C}.$$

Para essa medição, já foi obtido:

$$e = 10^\circ\text{C},$$

$$u = 20\%,$$

$$u_v = 60\%,$$

$$\dot{m}_v = 0,60 \text{ kg/min.}$$

Entretanto, um minuto depois, o transmissor registra uma nova temperatura:

$$T_{PV} = 72,4^\circ\text{C}.$$

Como a temperatura mudou, o erro também deverá ser recalculado:

$$e = 80 - 72,4.$$

Portanto:

$$e = 7,6^\circ\text{C}.$$

A ação proporcional passa a ser:

$$u = K_p e,$$

logo:

$$u = \left(2 \frac{\%}{^\circ\text{C}}\right) (7,6^\circ\text{C}),$$

resultando em:

$$u = 15,2\%.$$

A nova abertura da válvula será:

$$u_v = u_0 + u,$$

portanto:

$$u_v = 40 + 15,2,$$

e:

$$u_v = 55,2\%.$$

Consequentemente, a vazão de vapor será:

$$\dot{m}_v = K_v u_v,$$

resultando em:

$$\dot{m}_v = (0,01)(55,2),$$

ou:

$$\dot{m}_v = 0,552 \text{ kg/min}.$$

Observe que a temperatura de saída se aproximou do setpoint:

$$70,0 \rightarrow 72,4^\circ\text{C}.$$

Como consequência, o erro diminuiu:

$$10,0 \rightarrow 7,6^\circ\text{C},$$

a ação proporcional também diminuiu:

$$20,0 \rightarrow 15,2\%,$$

e a válvula passou de:

$$60,0 \rightarrow 55,2\%.$$

Portanto, à medida que a temperatura se aproxima do valor desejado, o controlador proporcional reduz progressivamente sua ação corretiva.

1.8.1 Repetição dos Cálculos para as Demais Medições

O mesmo procedimento realizado para as duas primeiras medições deve ser aplicado às demais temperaturas registradas.

Para cada linha da tabela será necessário calcular:

$$e_k = T_{SP} - T_{PV,k},$$

$$u_k = K_p e_k,$$

$$u_{v,k} = u_0 + u_k,$$

e:

$$\dot{m}_{v,k} = K_v u_{v,k}.$$

Se também desejarmos analisar energeticamente cada instante de operação, será necessário calcular:

$$\dot{Q}_{v,k}, \quad \dot{Q}_{processo,k}, \quad \dot{Q}_{perdas,k}, \quad \dot{Q}_{liq,k},$$

e finalmente:

$$\left(\frac{dT}{dt}\right)_k.$$

Para dez medições, esses cálculos ainda poderiam ser realizados manualmente. Entretanto, imagine um sistema industrial registrando a temperatura a cada segundo.

Em apenas uma hora de operação seriam obtidas:

$$60 \times 60 = 3600$$

medições.

Em oito horas:

$$8 \times 3600 = 28\,800$$

medições.

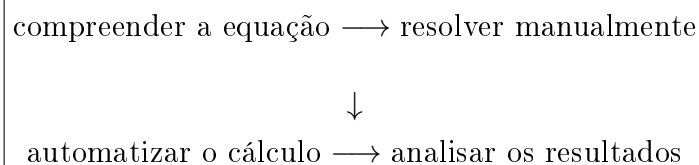
Em vinte e quatro horas:

$$24 \times 3600 = 86\,400$$

medições.

Realizar manualmente a mesma sequência de cálculos para milhares de registros seria extremamente demorado e sujeito a erros. O problema de Engenharia, entretanto, não mudou. As equações continuam sendo exatamente as mesmas. O que mudou foi a quantidade de dados que precisa ser processada.

Essa situação estabelece uma importante transição entre o cálculo manual e a Computação Científica:



O computador não substitui a compreensão do fenômeno físico. Ele permite aplicar, de maneira rápida e sistemática, as equações previamente estabelecidas pelo engenheiro a uma grande quantidade de dados.

1.8.2 Do Registro Industrial ao Arquivo de Dados

Para realizar a análise computacional, as medições podem ser armazenadas em um arquivo no formato CSV (*Comma-Separated Values*).

Neste estudo será utilizado o arquivo:

`dados_temperatura_trocador.csv`

contendo inicialmente duas variáveis:

- **tempo**: instante em que a medição foi registrada, em minutos;
- **Tpv**: temperatura de saída medida pelo transmissor, em °C.

O conteúdo inicial do arquivo será:

```
tempo,Tpv
0,70.0
1,72.4
2,74.3
3,75.8
4,77.0
5,77.9
6,78.6
7,79.1
8,79.4
9,79.6
```

Observe que, no arquivo CSV, foi utilizado o ponto como separador decimal. Essa convenção facilita a leitura direta dos valores numéricos pelas ferramentas computacionais que serão utilizadas neste capítulo.

A partir dessas duas colunas, todas as demais variáveis serão calculadas utilizando as equações desenvolvidas anteriormente.

Assim, o conjunto de dados será progressivamente transformado de:

$$\boxed{t, T_{PV}}$$

para:

$$\boxed{t, T_{PV}, e, u, u_v, \dot{m}_v, \dot{Q}_v, \dot{Q}_{processo}, \dot{Q}_{perdas}, \dot{Q}_{liq}, \frac{dT}{dt}}.$$

Cada nova coluna terá, portanto, uma interpretação física associada ao funcionamento da malha de controle e ao balanço de energia do trocador.

A questão passa a ser:

Como aplicar automaticamente as mesmas equações a todas as medições registradas?

Para responder a essa questão, serão utilizadas ferramentas de Computação Científica em Python.

1.8.3 Importação e Organização dos Dados Experimentais com Pandas

Até este ponto, a análise do sistema de controle foi realizada manualmente. A partir de uma temperatura medida pelo transmissor, foram calculados o erro de controle, a ação proporcional, a abertura da válvula, a vazão de vapor e as diferentes parcelas do balanço de energia.

Esse procedimento permitiu compreender fisicamente a sequência de acontecimentos que ocorre na malha:

$$T_{PV} \longrightarrow e \longrightarrow u \longrightarrow u_v \longrightarrow \dot{m}_v \longrightarrow \dot{Q}_v \longrightarrow \dot{Q}_{liq} \longrightarrow \frac{dT}{dt}.$$

Quando o número de medições aumenta, entretanto, a repetição manual desses cálculos torna-se pouco prática. A Computação Científica permite aplicar as mesmas equações a um grande conjunto de dados, preservando a interpretação física de cada variável. Nesta etapa será utilizada inicialmente a biblioteca Pandas, que fornece ferramentas para importação, organização e manipulação de dados estruturados em tabelas. Os cálculos serão realizados no ambiente Spyder, permitindo acompanhar não apenas os resultados apresentados no console, mas também as variáveis criadas durante a execução do programa por meio do *Explorador de Variáveis*.

Preparação do ambiente de trabalho

Inicialmente, crie uma pasta destinada à atividade. Nessa pasta deverão ser armazenados o programa Python e o arquivo contendo as medições do processo:

`dados_temperatura_trocador.csv`

O arquivo CSV contém:

```
tempo, Tpv
0, 70.0
1, 72.4
2, 74.3
3, 75.8
4, 77.0
5, 77.9
6, 78.6
7, 79.1
8, 79.4
9, 79.6
```

No Spyder, crie um novo arquivo Python e salve-o, por exemplo, com o nome:

`controle_trocador_calor.py`

Manter o programa e o arquivo CSV na mesma pasta facilita a leitura dos dados durante esta primeira atividade.

Importação da biblioteca Pandas

O primeiro passo consiste em importar a biblioteca Pandas:

```
# Importacao da biblioteca Pandas
import pandas as pd
```

A instrução `as pd` estabelece a abreviação `pd` para a biblioteca Pandas. Essa convenção é amplamente utilizada em programas Python destinados à manipulação e análise de dados. Após a importação da biblioteca, o arquivo contendo as medições pode ser lido utilizando a função `read_csv()`:

```
# Importacao da biblioteca Pandas
import pandas as pd

# Leitura do arquivo contendo as medicoes do trocador
dados = pd.read_csv("dados_temperatura_trocador.csv")
```

A variável `dados` passa a armazenar uma estrutura denominada *DataFrame*.

Neste momento, podemos interpretar um *DataFrame* simplesmente como uma tabela formada por linhas e colunas. Cada linha corresponde a uma observação do processo, enquanto cada coluna representa uma variável.

Para visualizar os dados no console, utilize:

```
# Exibicao dos dados importados
print(dados)
```

A saída será:

	tempo	Tpv
0	0	70.0
1	1	72.4
2	2	74.3
3	3	75.8
4	4	77.0
5	5	77.9
6	6	78.6
7	7	79.1
8	8	79.4
9	9	79.6

Os números apresentados à esquerda não representam o tempo. Eles correspondem ao índice criado automaticamente pelo Pandas para identificar cada linha do *DataFrame*. A variável física tempo encontra-se armazenada na coluna `tempo`, enquanto as temperaturas medidas pelo transmissor encontram-se na coluna `Tpv`.

Visualização dos dados no Explorador de Variáveis

Após executar o programa, observe o *Explorador de Variáveis* do Spyder. A variável `dados` deverá aparecer entre as variáveis disponíveis. Ao abrir essa variável, será possível visualizar o DataFrame em formato tabular. Neste primeiro momento, ele possui somente duas variáveis provenientes do sistema de aquisição:

`tempo` `Tpv`.

Nenhuma equação de controle foi aplicada até aqui. O computador apenas importou e organizou as medições realizadas no processo. Essa observação é importante. Para o Pandas, a coluna `Tpv` contém simplesmente uma sequência de valores numéricos. É o engenheiro que atribui significado físico a esses valores e estabelece que eles representam a temperatura de saída medida pelo transmissor.

Da mesma maneira, será o engenheiro quem determinará quais equações deverão ser aplicadas aos dados.

1.8.4 Cálculo Computacional do Erro de Controle

O primeiro cálculo a ser realizado é exatamente o mesmo desenvolvido manualmente. Para cada instante k :

$$e_k = T_{SP} - T_{PV,k}.$$

O setpoint do trocador permanece constante:

$$T_{SP} = 80^\circ\text{C}.$$

No programa, podemos representar essa grandeza pela variável `T_sp`:

```
# Temperatura desejada na saída do trocador (graus Celsius)
T_sp = 80.0
```

O erro de controle pode então ser calculado utilizando diretamente a coluna `Tpv`:

```
# Calculo do erro para todas as temperaturas medidas
dados["erro"] = T_sp - dados["Tpv"]
```

A instrução anterior merece atenção.

No cálculo manual, a equação:

$$e_k = T_{SP} - T_{PV,k}$$

precisaria ser aplicada separadamente a cada temperatura.

Com o Pandas, a expressão:

T_sp - dados["Tpv"]

é aplicada a todos os valores existentes na coluna Tpv. O resultado é armazenado em uma nova coluna denominada **erro**. Após executar essa instrução, abra novamente o DataFrame dados no Explorador de Variáveis.

A estrutura que inicialmente era:

tempo	Tpv
-------	-----

passa a ser:

tempo	Tpv	erro
-------	-----	------

Para visualizar o resultado no console:

```
# Exibicao do DataFrame apos o calculo do erro
print(dados)
```

O resultado será:

	tempo	Tpv	erro
0	0	70.0	10.0
1	1	72.4	7.6
2	2	74.3	5.7
3	3	75.8	4.2
4	4	77.0	3.0
5	5	77.9	2.1
6	6	78.6	1.4
7	7	79.1	0.9
8	8	79.4	0.6
9	9	79.6	0.4

Comparação com o cálculo manual

Antes de prosseguir, é importante verificar se os resultados computacionais são coerentes com os cálculos realizados anteriormente.

Para a primeira medição:

$$T_{PV} = 70,0^{\circ}\text{C}.$$

Manualmente foi obtido:

$$e = 80 - 70 = 10^{\circ}\text{C}.$$

No DataFrame, a primeira linha apresenta:

tempo	Tpv	erro
0	70.0	10.0

Para a segunda medição:

$$T_{PV} = 72,4^{\circ}\text{C},$$

e o cálculo manual forneceu:

$$e = 80 - 72,4 = 7,6^{\circ}\text{C}.$$

O mesmo resultado aparece na segunda linha do DataFrame.

Essa comparação é uma etapa importante da análise computacional. A utilização do computador não elimina a necessidade de verificar os resultados. Pelo contrário, o conhecimento adquirido durante os cálculos manuais permite avaliar se as operações realizadas pelo programa estão fisicamente e matematicamente corretas.

1.8.5 Cálculo Computacional da Ação Proporcional

Uma vez calculado o erro para todas as medições, o próximo passo consiste em determinar a ação proporcional do controlador.

A equação já conhecida é:

$$u_k = K_p e_k.$$

Para o sistema considerado:

$$K_p = 2 \frac{\%}{^{\circ}\text{C}}.$$

No programa, o ganho proporcional será definido por:

```
# Ganho proporcional do controlador (%/grau Celsius)
Kp = 2.0
```

A ação proporcional pode então ser calculada para todas as linhas do DataFrame:

```
# Calculo da acao proporcional do controlador
dados["u"] = Kp * dados["erro"]
```

Novamente, uma única instrução aplica a equação a todas as medições.

O DataFrame passa agora de:

tempo	Tpv	erro
-------	-----	------

para:

tempo	Tpv	erro	u
-------	-----	------	---

Ao executar:

```
# Exibicao dos resultados
print(dados)
```

será obtido:

tempo	Tpv	erro	u	
0	0	70.0	10.0	20.0
1	1	72.4	7.6	15.2
2	2	74.3	5.7	11.4
3	3	75.8	4.2	8.4
4	4	77.0	3.0	6.0
5	5	77.9	2.1	4.2
6	6	78.6	1.4	2.8
7	7	79.1	0.9	1.8
8	8	79.4	0.6	1.2
9	9	79.6	0.4	0.8

Observe que, à medida que a temperatura se aproxima do setpoint, o erro diminui e a ação proporcional também é reduzida.

Essa relação pode ser observada diretamente nos dados:

$$T_{PV} \uparrow \implies e \downarrow \implies u \downarrow.$$

O comportamento observado no DataFrame corresponde exatamente ao princípio físico estudado anteriormente para o controlador proporcional.

1.8.6 Cálculo da Abertura da Válvula

A ação proporcional u_k representa uma correção em relação à condição nominal de operação da válvula.

Para o sistema estudado:

$$u_0 = 40\%.$$

A abertura da válvula em cada instante é determinada por:

$$u_{v,k} = u_0 + u_k.$$

No programa:

```
# Abertura nominal da valvula (%)
```

```
u0 = 40.0
```

```
# Calculo da abertura da valvula para cada instante
```

```
dados["u_v"] = u0 + dados["u"]
```

Após a execução, uma nova coluna denominada `u_v` será acrescentada ao DataFrame.

Os resultados serão:

tempo	Tpv	erro	u	u_v
0	0	70.0	10.0	20.0
1	1	72.4	7.6	15.2
2	2	74.3	5.7	11.4
3	3	75.8	4.2	8.4
4	4	77.0	3.0	6.0
5	5	77.9	2.1	4.2
6	6	78.6	1.4	2.8
7	7	79.1	0.9	1.8
8	8	79.4	0.6	1.2
9	9	79.6	0.4	0.8

A primeira linha fornece novamente:

$$u_v = 60\%,$$

enquanto a segunda fornece:

$$u_v = 55,2\%.$$

Esses são exatamente os valores obtidos anteriormente por meio dos cálculos manuais. À medida que a temperatura se aproxima do setpoint, a correção proporcional diminui e a abertura da válvula aproxima-se progressivamente de sua condição nominal:

$$u_v \rightarrow u_0.$$

Esse comportamento pode ser compreendido diretamente a partir da equação:

$$u_v = u_0 + K_p e.$$

Quando:

$$e \rightarrow 0,$$

temos:

$$u \rightarrow 0,$$

e conseqüentemente:

$$u_v \rightarrow u_0.$$

Portanto, mesmo quando a ação proporcional tende a zero, a válvula não necessariamente fecha. Neste modelo, ela tende à abertura nominal necessária para manter o fornecimento de energia correspondente à condição de operação do processo.

1.8.7 Cálculo da Vazão de Vapor

A próxima etapa consiste em transformar a abertura calculada da válvula em uma variável física diretamente relacionada ao fornecimento de energia ao trocador.

No modelo simplificado adotado:

$$\dot{m}_{v,k} = K_v u_{v,k},$$

com:

$$K_v = 0,01 \frac{\text{kg/min}}{\%}.$$

No programa:

```
# Coeficiente simplificado da valvula (kg/min/%)
Kv = 0.01

# Calculo da vazao de vapor (kg/min)
dados["m_v"] = Kv * dados["u_v"]
```

O DataFrame possui agora as colunas:

tempo, Tpv, erro, u, u_v, m_v

Para visualizar somente essas variáveis:

```
# Exibicao das variaveis calculadas ate esta etapa
print(dados)
```

A saída será:

tempo	Tpv	erro	u	u_v	m_v
0	0	70.0	10.0	20.0	60.0
1	1	72.4	7.6	15.2	55.2
2	2	74.3	5.7	11.4	51.4
3	3	75.8	4.2	8.4	48.4
4	4	77.0	3.0	6.0	46.0
5	5	77.9	2.1	4.2	44.2

6	6	78.6	1.4	2.8	42.8	0.428
7	7	79.1	0.9	1.8	41.8	0.418
8	8	79.4	0.6	1.2	41.2	0.412
9	9	79.6	0.4	0.8	40.8	0.408

Observe como o DataFrame começa a representar computacionalmente a própria malha de controle.

Inicialmente estavam disponíveis apenas as medições:

$$t \rightarrow T_{PV}.$$

A partir das equações estabelecidas pelo engenheiro, foram acrescentadas novas informações:

$$t \rightarrow T_{PV} \rightarrow e \rightarrow u \rightarrow u_v \rightarrow \dot{m}_v.$$

O DataFrame não é, portanto, apenas uma tabela de números. Cada coluna representa uma etapa do raciocínio utilizado para compreender o funcionamento físico da malha de controle.

1.8.8 Do Controle ao Balanço de Energia

Até este ponto, a análise computacional permitiu partir da temperatura medida e chegar à vazão de vapor fornecida ao trocador. Entretanto, ainda é necessário responder a uma questão fundamental:

Qual é o efeito dessa vazão de vapor sobre o comportamento térmico do processo?

Para responder a essa pergunta, as próximas colunas do DataFrame serão construídas a partir do balanço de energia desenvolvido anteriormente.

A sequência será ampliada para:

$$T_{PV} \rightarrow e \rightarrow u \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v \rightarrow \dot{Q}_{processo} \rightarrow \dot{Q}_{perdas} \rightarrow \dot{Q}_{liq} \rightarrow \frac{dT}{dt}.$$

Assim, será possível acompanhar computacionalmente não apenas a resposta do controlador, mas também as consequências energéticas de sua atuação sobre o processo.

1.8.9 Cálculo da Potência Térmica Fornecida pelo Vapor

A primeira grandeza energética a ser acrescentada ao DataFrame é a potência térmica disponibilizada pela condensação do vapor.

Anteriormente foi estabelecida a relação:

$$\dot{Q}_{v,k} = \dot{m}_{v,k}^* \lambda_v,$$

em que $\dot{m}_{v,k}^*$ deve ser expressa em kg/s para que a potência térmica seja obtida em watts.

Entretanto, a coluna `m_v` do DataFrame contém a vazão de vapor em kg/min. Portanto, antes de calcular a potência térmica, será criada uma nova coluna contendo a vazão convertida para kg/s.

No programa:

```
# Conversao da vazao de vapor de kg/min para kg/s
dados["m_v_kg_s"] = dados["m_v"] / 60.0
```

Após executar essa instrução, abra novamente a variável `dados` no Explorador de Variáveis.

Uma nova coluna deverá aparecer:

m_v_kg_s

.

Para a primeira medição:

$$\dot{m}_v = 0,60 \text{ kg/min},$$

e, portanto:

$$\dot{m}_v^* = \frac{0,60}{60} = 0,010 \text{ kg/s}.$$

O mesmo cálculo foi realizado automaticamente para todas as linhas do DataFrame.

Considere agora:

$$\lambda_v = 2,0 \times 10^6 \text{ J/kg}.$$

No programa:

```
# Entalpia especifica associada a condensacao do vapor (J/kg)
lambda_v = 2.0e6
```

```
# Potencia termica fornecida pelo vapor (W)
dados["Q_v"] = dados["m_v_kg_s"] * lambda_v
```

A coluna `Q_v` representa a potência térmica disponibilizada pelo vapor em cada instante de operação. Observe novamente o DataFrame no Explorador de Variáveis. A sequência física representada computacionalmente já pode ser escrita como:

$T_{PV} \rightarrow e \rightarrow u \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v$

.

Para a primeira medição, o programa deverá fornecer:

$$\dot{Q}_v = 20\,000 \text{ W},$$

que corresponde exatamente ao resultado obtido anteriormente no cálculo manual. Essa comparação deve ser realizada antes de prosseguir. Quando um resultado computacional reproduz um cálculo manual previamente verificado, aumenta-se a confiança de que as equações e as unidades foram implementadas corretamente.

1.8.10 Cálculo da Potência Térmica Associada à Corrente de Processo

A próxima parcela do balanço corresponde à energia necessária para aquecer continuamente a corrente que atravessa o trocador.

A equação utilizada anteriormente foi:

$$\dot{Q}_{processo,k} = \dot{m}_{in}^* c_p (T_{PV,k} - T_{in}).$$

Considere:

$$\dot{m}_{in} = 3,0 \text{ kg/min},$$

$$c_p = 4000 \text{ J/(kg } ^\circ\text{C)}$$

e:

$$T_{in} = 30 ^\circ\text{C}.$$

Como a vazão da corrente de processo também está expressa em kg/min, inicialmente será realizada sua conversão para kg/s:

$$\dot{m}_{in}^* = \frac{\dot{m}_{in}}{60}.$$

No programa:

```
# Vazao massica da corrente de processo (kg/min)
m_in = 3.0

# Conversao da vazao da corrente para kg/s
m_in_kg_s = m_in / 60.0

# Calor especifico da corrente de processo (J/(kg.grau Celsius))
cp = 4000.0

# Temperatura de entrada da corrente de processo (graus Celsius)
T_in = 30.0
```

Observe que `m_in_kg_s`, `cp` e `T_in` são parâmetros do modelo e, portanto, aparecem como variáveis individuais no Explorador de Variáveis.

A temperatura de saída, por outro lado, varia ao longo do tempo e está armazenada na coluna `Tpv` do `DataFrame`.

A potência térmica associada ao aquecimento da corrente pode então ser calculada por:

```
# Potencia utilizada para aquecer a corrente de processo (W)
dados["Q_processo"] = (
    m_in_kg_s
    * cp
    * (dados["Tpv"] - T_in)
)
```

Uma nova coluna será acrescentada:

`Q_processo`.

Para a primeira medição:

$$T_{PV} = 70\text{ }^{\circ}\text{C},$$

o resultado deverá ser:

$$\dot{Q}_{processo} = 8000\text{ W}.$$

Novamente, esse valor coincide com o resultado obtido manualmente.

É importante perceber a diferença entre os dois tipos de grandezas que aparecem no programa. Algumas grandezas permanecem constantes durante a análise, como:

$$\dot{m}_{in}, \quad c_p, \quad T_{in}.$$

Outras variam a cada medição, como:

$$T_{PV,k}, \quad e_k, \quad u_k, \quad u_{v,k}, \quad \dot{m}_{v,k}.$$

No `DataFrame`, cada linha representa um instante de operação e as colunas permitem acompanhar como essas grandezas variáveis se relacionam entre si.

1.8.11 Cálculo das Perdas Térmicas

No modelo adotado, as perdas de energia para o ambiente são aproximadas por:

$$\dot{Q}_{perdas,k} = U_L(T_{PV,k} - T_{amb}).$$

Considere:

$$U_L = 20\text{ W}/^{\circ}\text{C}$$

e:

$$T_{amb} = 25^\circ\text{C}.$$

Esses parâmetros podem ser definidos no programa por:

```
# Coeficiente simplificado de perda termica (W/grau Celsius)
U_L = 20.0

# Temperatura ambiente (graus Celsius)
T_amb = 25.0
```

A nova coluna será calculada por:

```
# Perdas termicas para o ambiente (W)
dados["Q_perdas"] = U_L * (dados["TpV"] - T_amb)
```

Após executar essa linha, observe novamente o DataFrame.

Agora cada temperatura medida possui uma estimativa correspondente da perda térmica para o ambiente.

Para:

$$T_{PV} = 70^\circ\text{C},$$

temos:

$$\dot{Q}_{perdas} = 20(70 - 25) = 900 \text{ W}.$$

À medida que a temperatura do processo aumenta, a diferença entre a temperatura do sistema e a temperatura ambiente também aumenta. Segundo o modelo simplificado adotado, isso produz um aumento das perdas térmicas.

1.8.12 Construção do Balanço de Energia no DataFrame

Neste ponto, o DataFrame já contém as três parcelas necessárias para determinar a potência térmica líquida:

$$\dot{Q}_{v,k}, \quad \dot{Q}_{processo,k}, \quad \dot{Q}_{perdas,k}.$$

O balanço de energia desenvolvido anteriormente estabelece:

$$\dot{Q}_{liq,k} = \dot{Q}_{v,k} - \dot{Q}_{processo,k} - \dot{Q}_{perdas,k}.$$

No programa, essa equação pode ser escrita diretamente utilizando as colunas do DataFrame:

```
# Potencia termica liquida acumulada no sistema (W)
dados["Q_liq"] = (
    dados["Q_v"]
    - dados["Q_processo"]
    - dados["Q_perdas"]
)
```

Observe a correspondência direta entre a equação matemática:

$$\dot{Q}_{liq} = \dot{Q}_v - \dot{Q}_{processo} - \dot{Q}_{perdas}$$

e sua implementação computacional:

```
dados["Q_liq"] = (
    dados["Q_v"]
    - dados["Q_processo"]
    - dados["Q_perdas"]
)
```

Essa correspondência é uma característica importante da Computação Científica: quando o modelo matemático está corretamente estabelecido, sua implementação computacional pode conservar de maneira bastante clara a estrutura física do problema.

Para a primeira medição:

$$\dot{Q}_v = 20\,000\text{ W},$$

$$\dot{Q}_{processo} = 8000\text{ W},$$

e:

$$\dot{Q}_{perdas} = 900\text{ W}.$$

Portanto, o programa deverá fornecer:

$$\boxed{\dot{Q}_{liq} = 11\,100\text{ W}}.$$

Esse é novamente o mesmo resultado obtido manualmente.

1.8.13 Cálculo da Taxa de Variação da Temperatura

A última etapa do balanço consiste em relacionar a potência térmica líquida à tendência de variação da temperatura.

O modelo dinâmico simplificado estabelece:

$$C \frac{dT}{dt} = \dot{Q}_{liq}.$$

Portanto:

$$\frac{dT}{dt} = \frac{\dot{Q}_{liq}}{C}.$$

Considere a capacidade térmica equivalente:

$$C = 1,0 \times 10^5 \text{ J/}^\circ\text{C}.$$

No programa:

```
# Capacidade termica equivalente do sistema (J/grau Celsius)
```

```
C = 1.0e5
```

```
# Taxa de variacao da temperatura (graus Celsius/s)
```

```
dados["dT_dt"] = dados["Q_liq"] / C
```

A coluna `dT_dt` apresenta a taxa instantânea de variação da temperatura em $^\circ\text{C/s}$.

Como os dados experimentais foram registrados em minutos, também é conveniente criar uma coluna expressando essa grandeza em

$^\circ\text{C/min}$:

```
# Conversao da taxa de variacao para graus Celsius/min
```

```
dados["dT_dt_min"] = dados["dT_dt"] * 60.0
```

Para a primeira medição, deverá ser obtido:

$$\frac{dT}{dt} = 0,111 \text{ }^\circ\text{C/s},$$

ou:

$$\boxed{\frac{dT}{dt} = 6,66 \text{ }^\circ\text{C/min} .}$$

O sinal da coluna `dT_dt_min` permite interpretar diretamente a tendência térmica prevista pelo modelo.

Quando:

$$\text{dT_dt_min} > 0,$$

o modelo indica tendência de aumento da temperatura.

Quando:

$$\text{dT_dt_min} < 0,$$

o modelo indica tendência de redução da temperatura.

Quando:

$$dT_dt_min \approx 0,$$

o sistema encontra-se próximo de uma condição de equilíbrio térmico segundo as hipóteses adotadas.

1.8.14 O DataFrame como Representação Computacional do Processo

Após a execução de todas as etapas anteriores, o DataFrame deixou de conter apenas os dados originalmente registrados pelo sistema de aquisição.

Inicialmente tínhamos:

$$\begin{bmatrix} \text{tempo} & T_{pv} \end{bmatrix}.$$

A partir das equações de controle e do balanço de energia, foram progressivamente acrescentadas novas informações.

Agora temos:

$$\begin{bmatrix} t & T_{PV} & e & u & u_v & \dot{m}_v & \dot{Q}_v & \dot{Q}_{processo} & \dot{Q}_{perdas} & \dot{Q}_{liq} & \frac{dT}{dt} \end{bmatrix}.$$

No programa, essas grandezas aparecem aproximadamente como:

```
tempo
Tpv
erro
u
u_v
m_v
m_v_kg_s
Q_v
Q_processo
Q_perdas
Q_liq
dT_dt
dT_dt_min
```

Observe que somente duas dessas colunas estavam presentes no arquivo original:

$$\text{tempo} \quad \text{e} \quad T_{pv}.$$

Todas as demais foram produzidas a partir de modelos matemáticos estabelecidos pelo engenheiro. Esse processo pode ser interpretado como uma transformação:

$$\boxed{\text{dados medidos} \longrightarrow \text{dados processados} \longrightarrow \text{informação de Engenharia}}.$$

O computador realizou milhares de operações potenciais de maneira rápida, mas nenhuma das relações físicas surgiu automaticamente. As equações, as hipóteses, as unidades e a interpretação dos resultados foram estabelecidas a partir do conhecimento do processo.

Visualização da tabela completa

Para visualizar todas as colunas calculadas no console, utilize:

```
# Exibicao do DataFrame completo
print(dados)
```

Entretanto, neste estágio da atividade, o Explorador de Variáveis do Spyder torna-se particularmente útil.

Abra a variável **dados** e percorra horizontalmente suas colunas. Procure acompanhar cada linha da esquerda para a direita. Para uma determinada medição, a sequência deve ser interpretada como:

$$T_{PV,k} \rightarrow e_k \rightarrow u_k \rightarrow u_{v,k} \rightarrow \dot{m}_{v,k} \rightarrow \dot{Q}_{v,k} \rightarrow \dot{Q}_{liq,k} \rightarrow \left(\frac{dT}{dt} \right)_k.$$

Em seguida, percorra uma determinada coluna de cima para baixo.

Nesse caso, você estará observando como uma mesma variável se modifica ao longo do tempo.

Essas são duas formas diferentes e complementares de interpretar o DataFrame:

- **horizontalmente:** acompanha-se a cadeia de cálculos para uma determinada condição de operação;
- **verticalmente:** acompanha-se a evolução de uma determinada variável ao longo do tempo.

Essa distinção será importante na próxima etapa, pois os gráficos serão construídos justamente a partir da evolução temporal das colunas do DataFrame.

1.8.15 Verificação dos Resultados Computacionais

Antes de construir qualquer gráfico, é necessário verificar se os resultados obtidos são coerentes.

Uma estratégia simples consiste em comparar a primeira linha do DataFrame com o exemplo resolvido manualmente.

Para:

$$T_{PV} = 70^\circ\text{C},$$

os principais resultados esperados são:

$$e = 10^{\circ}\text{C},$$

$$u = 20\%,$$

$$u_v = 60\%,$$

$$\dot{m}_v = 0,60 \text{ kg/min},$$

$$\dot{Q}_v = 20,0 \text{ kW},$$

$$\dot{Q}_{processo} = 8,0 \text{ kW},$$

$$\dot{Q}_{perdas} = 0,90 \text{ kW},$$

$$\dot{Q}_{liq} = 11,1 \text{ kW},$$

e:

$$\frac{dT}{dt} = 6,66^{\circ}\text{C/min}.$$

Se esses valores forem reproduzidos pelo programa, temos uma primeira evidência de que as equações foram implementadas corretamente.

Entretanto, a verificação de um programa de Engenharia não deve se limitar à comparação numérica. Também é necessário verificar se os resultados fazem sentido fisicamente.

Por exemplo, nas primeiras medições:

$$T_{PV} < T_{SP}.$$

Portanto, espera-se que o controlador solicite maior fornecimento de vapor. À medida que:

$$T_{PV} \rightarrow T_{SP},$$

espera-se que:

$$e \rightarrow 0,$$

e conseqüentemente que a ação proporcional seja reduzida.

Essas tendências devem estar presentes nas colunas do DataFrame. Somente depois

dessa verificação matemática e física é recomendável avançar para a representação gráfica dos resultados.

1.9 Visualização da Dinâmica do Processo com Matplotlib

Até este ponto, as informações do processo foram analisadas numericamente por meio do DataFrame. O Explorador de Variáveis do Spyder permitiu acompanhar a construção progressiva das colunas e observar como cada grandeza foi obtida a partir das equações de controle e do balanço de energia.

Essa representação tabular é fundamental, pois permite visualizar os valores numéricos associados a cada instante de operação.

Entretanto, quando o objetivo é analisar tendências, comparar comportamentos e compreender como uma variável evolui ao longo do tempo, a representação gráfica torna-se particularmente útil.

A visualização científica permite transformar sequências de valores numéricos em informações que podem ser interpretadas de maneira mais rápida.

Neste estudo será utilizada a biblioteca Matplotlib. A estratégia adotada será construir os gráficos progressivamente, seguindo a mesma sequência utilizada durante a construção do DataFrame.

Assim, serão analisados:

$$T_{PV} \rightarrow e \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v \rightarrow \dot{Q}_{liq}.$$

Cada gráfico deverá ser interpretado antes que o próximo seja construído.

1.9.1 Importação da Biblioteca Matplotlib

Inicialmente, importe o módulo `pyplot` da biblioteca Matplotlib:

```
# Biblioteca para construcao de graficos
import matplotlib.pyplot as plt
```

A abreviação:

```
plt
```

será utilizada para acessar as principais funções de construção e configuração dos gráficos. O programa possui agora duas bibliotecas principais:

```
# Biblioteca para manipulacao de dados
import pandas as pd

# Biblioteca para construcao de graficos
import matplotlib.pyplot as plt
```

O Pandas foi utilizado para importar, organizar e processar os dados. O Matplotlib será utilizado para representar graficamente as informações já armazenadas no DataFrame.

Essa sequência pode ser resumida por:

arquivo CSV $\longrightarrow$ Pandas $\longrightarrow$ DataFrame $\longrightarrow$ Matplotlib

Portanto, o Matplotlib não substituirá a tabela construída anteriormente. Ele utilizará justamente as colunas dessa tabela para produzir as representações gráficas.

1.9.2 Evolução da Temperatura do Processo

A primeira variável a ser analisada graficamente será a temperatura de saída do trocador. O DataFrame possui as colunas:

tempo Tpv

que representam, respectivamente, o instante da medição e a temperatura medida pelo transmissor.

Para construir o gráfico:

```
# Grafico da temperatura medida ao longo do tempo
plt.figure()

plt.plot(
    dados["tempo"],
    dados["Tpv"],
    marker="o"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Temperatura de saida do trocador de calor")
plt.grid()

plt.show()
```

Observe cuidadosamente a relação entre o comando utilizado e as colunas do DataFrame. Na instrução:

```
plt.plot(dados["tempo"], dados["Tpv"])
```

a coluna `tempo` fornece os valores do eixo horizontal, enquanto a coluna `Tpv` fornece os valores do eixo vertical.

Portanto, o gráfico representa:

$$T_{PV} = f(t).$$

Antes de prosseguir, observe novamente essas duas colunas no Explorador de Variáveis e compare-as com os pontos apresentados no gráfico.

Cada ponto representado graficamente corresponde a uma linha do DataFrame.

Por exemplo:

$$t = 0 \text{ min} \quad \implies \quad T_{PV} = 70,0^\circ\text{C},$$

enquanto:

$$t = 9 \text{ min} \quad \implies \quad T_{PV} = 79,6^\circ\text{C}.$$

O gráfico não criou novas informações. Ele apresentou visualmente os mesmos dados que já estavam armazenados na tabela.

Entretanto, uma tendência que exige a leitura de várias linhas no DataFrame torna-se imediatamente perceptível no gráfico:

$$T_{PV} \uparrow.$$

A temperatura aumenta ao longo do período analisado.

1.9.3 Comparação entre a Temperatura Medida e o Setpoint

Embora o primeiro gráfico permita observar a evolução da temperatura, ainda falta uma informação fundamental para a análise de controle:

Qual é a temperatura que o sistema deveria atingir?

O valor desejado é:

$$T_{SP} = 80^\circ\text{C}.$$

Portanto, é conveniente representar no mesmo gráfico a temperatura medida e o setpoint.

Como o setpoint permanece constante durante todo o experimento, pode-se utilizar:

```
# Grafico da temperatura medida
plt.figure()

plt.plot(
    dados["tempo"],
    dados["TpV"],
    marker="o",
    label="TpV"
```

```

)

# Representacao do setpoint
plt.axhline(
    y=T_sp,
    linestyle="--",
    label="Tsp"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Temperatura medida e setpoint")
plt.grid()
plt.legend()

plt.show()

```

A função:

```
plt.axhline(y=T_sp)
```

produz uma linha horizontal na temperatura correspondente ao setpoint.

O gráfico passa agora a apresentar duas informações:

$$T_{PV}(t)$$

e:

$$T_{SP}.$$

A comparação visual permite perceber que a temperatura medida inicia consideravelmente abaixo do valor desejado e se aproxima progressivamente do setpoint.

Em termos do processo:

$$T_{PV} \rightarrow T_{SP}.$$

Essa tendência já estava presente na tabela, mas sua identificação torna-se muito mais imediata por meio da representação gráfica.

1.9.4 Análise Gráfica do Erro de Controle

A diferença entre a temperatura desejada e a temperatura medida foi representada anteriormente pela equação:

$$e_k = T_{SP} - T_{PV,k}.$$

O DataFrame já possui a coluna:

erro

contendo o resultado desse cálculo para cada medição.

Podemos agora representar:

$$e = f(t).$$

No programa:

```
# Grafico do erro de controle ao longo do tempo
plt.figure()

plt.plot(
    dados["tempo"],
    dados["erro"],
    marker="o"
)

plt.axhline(
    y=0.0,
    linestyle="--"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Erro (graus Celsius)")
plt.title("Erro de controle")
plt.grid()

plt.show()
```

A linha horizontal correspondente a:

$$e = 0$$

representa a condição em que:

$$T_{PV} = T_{SP}.$$

Observe inicialmente a coluna **erro** no Explorador de Variáveis:

```
10.0
7.6
5.7
4.2
```

3.0
2.1
1.4
0.9
0.6
0.4

O mesmo comportamento aparece agora graficamente.

O erro diminui ao longo do tempo:

$$10,0 \rightarrow 7,6 \rightarrow 5,7 \rightarrow \dots \rightarrow 0,4^{\circ}\text{C}.$$

Portanto:

$$\boxed{T_{PV} \rightarrow T_{SP} \implies e \rightarrow 0}.$$

Esse gráfico mostra uma informação fundamental sobre o desempenho da malha: o processo está se aproximando da condição desejada.

1.9.5 Análise Gráfica da Ação Proporcional

A ação proporcional está diretamente relacionada ao erro:

$$u_k = K_p e_k.$$

Como K_p permanece constante, espera-se que a redução do erro produza também uma redução da ação proporcional.

Essa hipótese pode ser verificada graficamente.

No programa:

```
# Grafico da acao proporcional do controlador
plt.figure()

plt.plot(
    dados["tempo"],
    dados["u"],
    marker="o"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Acao proporcional (%)")
plt.title("Acao proporcional do controlador")
plt.grid()

plt.show()
```

Observe as colunas **erro** e **u** no Explorador de Variáveis.

Na primeira medição:

$$e = 10^{\circ}\text{C}$$

e:

$$u = 20\%.$$

Na última medição:

$$e = 0,4^{\circ}\text{C}$$

e:

$$u = 0,8\%.$$

Portanto:

$$\boxed{e \downarrow \implies u \downarrow}.$$

O gráfico apresenta visualmente a própria definição da ação proporcional: quanto menor o erro, menor a intensidade da correção aplicada pelo controlador.

1.9.6 Análise Gráfica da Abertura da Válvula

A ação calculada pelo controlador modifica a posição da válvula segundo:

$$u_{v,k} = u_0 + u_k.$$

No DataFrame, essa informação encontra-se armazenada na coluna:

u_v.

Para visualizar sua evolução:

```
# Grafico da abertura da valvula
plt.figure()

plt.plot(
    dados["tempo"],
    dados["u_v"],
    marker="o"
)

plt.axhline(
```

```

        y=u0,
        linestyle="--",
        label="Abertura nominal"
    )

plt.xlabel("Tempo (min)")
plt.ylabel("Abertura da valvula (%)")
plt.title("Abertura da valvula de vapor")
plt.grid()
plt.legend()

plt.show()

```

O gráfico mostra que a válvula inicia com abertura de:

$$60\%$$

e reduz progressivamente sua abertura à medida que a temperatura se aproxima do setpoint.

Ao final do período analisado:

$$u_v = 40,8\%.$$

Observe que:

$$u_v \rightarrow u_0.$$

Esse resultado possui uma interpretação física importante.

Quando o erro se aproxima de zero:

$$e \rightarrow 0,$$

a ação proporcional também se aproxima de zero:

$$u \rightarrow 0.$$

Entretanto:

$$u_v = u_0 + u.$$

Consequentemente:

$$u_v \rightarrow u_0.$$

Portanto, a válvula não tende necessariamente ao fechamento.

Neste modelo, a abertura nominal representa a condição de operação necessária para continuar fornecendo energia ao processo mesmo quando a temperatura se encontra próxima do valor desejado.

1.9.7 Análise Gráfica da Vazão de Vapor

A alteração da abertura da válvula modifica diretamente a vazão de vapor:

$$\dot{m}_{v,k} = K_v u_{v,k}.$$

A coluna correspondente no DataFrame é:

`m_v`.

Sua evolução temporal pode ser representada por:

```
# Grafico da vazao de vapor
plt.figure()

plt.plot(
    dados["tempo"],
    dados["m_v"],
    marker="o"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Vazao de vapor (kg/min)")
plt.title("Vazao de vapor para o trocador de calor")
plt.grid()

plt.show()
```

No início do período analisado:

$$\dot{m}_v = 0,600 \text{ kg/min.}$$

Ao final:

$$\dot{m}_v = 0,408 \text{ kg/min.}$$

Portanto:

$$\boxed{T_{PV} \uparrow \Longrightarrow e \downarrow \Longrightarrow u \downarrow \Longrightarrow u_v \downarrow \Longrightarrow \dot{m}_v \downarrow}.$$

Agora o aluno pode visualizar graficamente algo que inicialmente foi estudado apenas por meio das equações.

Quando a temperatura está muito abaixo do setpoint, o controlador solicita maior fornecimento de vapor.

À medida que a temperatura se aproxima do valor desejado, a quantidade de vapor solicitada é progressivamente reduzida.

1.9.8 Da Vazão de Vapor à Potência Térmica

A vazão de vapor determina a potência térmica disponibilizada ao trocador:

$$\dot{Q}_{v,k} = \dot{m}_{v,k}^* \lambda_v.$$

O DataFrame contém essa grandeza na coluna:

`Q_v`.

Como os valores estão expressos em watts, é conveniente dividi-los por 1000 durante a representação gráfica para que sejam apresentados em quilowatts.

No programa:

```
# Grafico da potencia termica fornecida pelo vapor
plt.figure()

plt.plot(
    dados["tempo"],
    dados["Q_v"] / 1000.0,
    marker="o"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Potencia termica (kW)")
plt.title("Potencia termica fornecida pelo vapor")
plt.grid()

plt.show()
```

Observe que a divisão por:

`1000.0`

foi realizada apenas para apresentação do gráfico.

Os valores originais armazenados na coluna `Q_v` permanecem em watts.

Essa distinção é importante: alterar a unidade utilizada em uma representação gráfica não significa necessariamente alterar os dados armazenados no DataFrame.

A tendência observada é:

$$\dot{m}_v \downarrow \implies \dot{Q}_v \downarrow.$$

Portanto, o gráfico permite acompanhar a redução progressiva da quantidade de energia fornecida pelo vapor à medida que a temperatura se aproxima do setpoint.

1.9.9 Comparação das Parcelas do Balanço de Energia

Até este momento, os gráficos foram construídos individualmente para facilitar a interpretação de cada grandeza. Neste ponto, entretanto, existe uma razão física para representar algumas variáveis simultaneamente.

O balanço de energia envolve:

$$\dot{Q}_v, \quad \dot{Q}_{processo} \quad \text{e} \quad \dot{Q}_{perdas}.$$

Essas grandezas possuem a mesma dimensão física e podem ser comparadas diretamente.

No programa:

```
# Comparacao das parcelas do balanço de energia
```

```
plt.figure()
```

```
plt.plot(
    dados["tempo"],
    dados["Q_v"] / 1000.0,
    marker="o",
    label="Q_v"
)
```

```
plt.plot(
    dados["tempo"],
    dados["Q_processo"] / 1000.0,
    marker="o",
    label="Q_processo"
)
```

```
plt.plot(
    dados["tempo"],
    dados["Q_perdas"] / 1000.0,
    marker="o",
    label="Q_perdas"
)
```

```
plt.xlabel("Tempo (min)")
```

```
plt.ylabel("Potencia termica (kW)")
```

```
plt.title("Parcelas do balanço de energia")
plt.grid()
plt.legend()

plt.show()
```

Agora existe uma justificativa física para a presença de três curvas no mesmo gráfico: todas participam diretamente do mesmo balanço de energia. A potência fornecida pelo vapor diminui ao longo do tempo, enquanto a energia associada ao aquecimento da corrente de processo aumenta à medida que a temperatura de saída se eleva. As perdas térmicas também aumentam, pois a diferença entre a temperatura do processo e a temperatura ambiente torna-se maior.

Essas três tendências contribuem para determinar a potência térmica líquida:

$$\dot{Q}_{liq} = \dot{Q}_v - \dot{Q}_{processo} - \dot{Q}_{perdas}.$$

1.9.10 Análise da Potência Térmica Líquida

A coluna Q_liq representa o resultado do balanço de energia em cada instante.

Para visualizar sua evolução:

```
# Grafico da potencia termica liquida
plt.figure()

plt.plot(
    dados["tempo"],
    dados["Q_liq"] / 1000.0,
    marker="o"
)

plt.axhline(
    y=0.0,
    linestyle="--"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Potencia termica liquida (kW)")
plt.title("Potencia termica liquida do sistema")
plt.grid()

plt.show()
```

A linha:

$$\dot{Q}_{liq} = 0$$

é particularmente importante.

Quando:

$$\dot{Q}_{liq} > 0,$$

o modelo indica acúmulo de energia térmica e, conseqüentemente, tendência de aumento da temperatura.

Quando:

$$\dot{Q}_{liq} < 0,$$

existe tendência de redução da temperatura.

Durante o período analisado, os valores de $\dot{Q}_{liq}$ permanecem positivos, embora diminuam progressivamente.

Portanto, segundo o modelo simplificado adotado:

$$\dot{Q}_{liq} > 0 \implies \frac{dT}{dt} > 0.$$

Isso é coerente, qualitativamente, com as medições registradas, pois a temperatura aumenta durante todo o período analisado.

É importante observar, entretanto, que os valores de temperatura presentes no arquivo CSV representam medições do processo, enquanto $\dot{Q}_{liq}$ é uma grandeza calculada a partir do modelo simplificado.

Portanto, não se deve esperar que um modelo simplificado reproduza exatamente cada variação observada experimentalmente. O objetivo, neste momento, é verificar se o modelo descreve corretamente as principais tendências físicas do sistema.

1.9.11 Análise da Taxa de Variação da Temperatura

A potência térmica líquida está relacionada à taxa instantânea de variação da temperatura por:

$$\frac{dT}{dt} = \frac{\dot{Q}_{liq}}{C}.$$

A coluna `dT_dt_min` contém essa grandeza expressa em °C/min.

Sua representação gráfica pode ser obtida por:

```
# Grafico da taxa de variacao da temperatura
plt.figure()

plt.plot(
```

```

        dados["tempo"],
        dados["dT_dt_min"],
        marker="o"
    )

plt.axhline(
    y=0.0,
    linestyle="--"
)

plt.xlabel("Tempo (min)")
plt.ylabel("dT/dt (graus Celsius/min)")
plt.title("Taxa de variacao da temperatura")
plt.grid()

plt.show()

```

O gráfico mostra que:

$$\frac{dT}{dt} > 0$$

durante todo o período analisado.

Entretanto, sua magnitude diminui à medida que o sistema se aproxima da região de operação desejada.

Do ponto de vista físico, isso significa que o processo continua apresentando tendência de aquecimento, mas a intensidade dessa tendência é progressivamente reduzida.

1.9.12 Interpretando a Malha de Controle a Partir dos Gráficos

Os gráficos construídos individualmente podem agora ser analisados como partes de um mesmo fenômeno.

No início da operação considerada:

$$T_{PV} = 70^{\circ}\text{C},$$

enquanto:

$$T_{SP} = 80^{\circ}\text{C}.$$

Portanto, existe um erro positivo relativamente elevado.

Como consequência:

$$e \uparrow \implies u \uparrow \implies u_v \uparrow \implies \dot{m}_v \uparrow \implies \dot{Q}_v \uparrow.$$

A maior quantidade de energia fornecida pelo vapor produz uma potência térmica líquida positiva, contribuindo para o aumento da temperatura.

À medida que:

$$T_{PV} \rightarrow T_{SP},$$

ocorre:

$$e \downarrow.$$

Consequentemente:

$$u \downarrow,$$

$$u_v \downarrow,$$

$$\dot{m}_v \downarrow$$

e:

$$\dot{Q}_v \downarrow.$$

Portanto, aquilo que inicialmente foi apresentado como uma sequência matemática:

$$T_{PV} \rightarrow e \rightarrow u \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v \rightarrow \dot{Q}_{liq} \rightarrow \frac{dT}{dt}$$

pode agora ser observado diretamente nos dados e nos gráficos.

O estudante não está apenas calculando variáveis isoladas. Ele está acompanhando a propagação de uma ação através de uma malha de controle.

1.9.13 Da Tabela ao Gráfico: Duas Formas de Observar o Mesmo Processo

Neste estágio, é importante comparar novamente as duas formas de visualização utilizadas durante a atividade.

No Explorador de Variáveis, o DataFrame permite observar os valores numéricos de cada instante.

Uma leitura horizontal permite acompanhar:

uma medição $\longrightarrow$ todos os cálculos associados
--

Uma leitura vertical permite acompanhar:

uma variável $\longrightarrow$ sua evolução ao longo do tempo.

O gráfico é uma representação visual dessa segunda forma de leitura.

Assim, quando é construído o gráfico:

```
plt.plot(dados["tempo"], dados["u_v"])
```

o Matplotlib está utilizando justamente os valores que podem ser observados verticalmente na coluna `u_v` do DataFrame.

A relação entre tabela e gráfico pode ser representada por:

coluna do DataFrame $\longrightarrow$ variável ao longo do tempo $\longrightarrow$ gráfico.

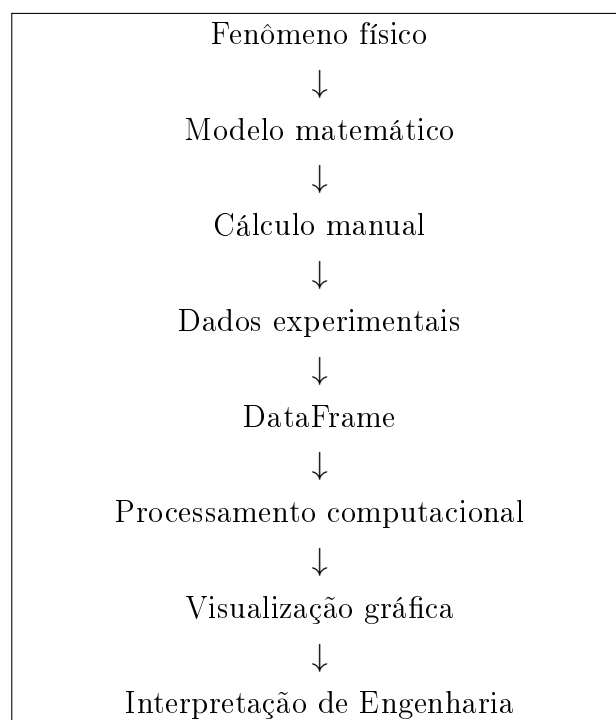
Essa relação é fundamental para a análise computacional de dados de processos industriais.

1.9.14 O Papel da Computação Científica na Análise do Processo

O percurso realizado neste capítulo permite compreender a função da Computação Científica em um problema de Controle de Processos.

Inicialmente, foi necessário compreender fisicamente o equipamento e identificar suas principais variáveis. Em seguida, foram estabelecidas as equações utilizadas para representar a malha de controle e o balanço de energia. Uma condição de operação foi resolvida manualmente. Posteriormente, o mesmo procedimento foi aplicado a várias medições por meio do Pandas.

Finalmente, os resultados foram representados graficamente com o Matplotlib. O processo completo pode ser sintetizado por:



A programação aparece, portanto, como uma ferramenta dentro de um processo mais amplo de raciocínio de Engenharia.

O objetivo não é apenas produzir códigos ou gráficos. O objetivo é utilizar essas ferramentas para compreender o comportamento do processo e obter informações que auxiliem a análise e a tomada de decisão.

1.10 Investigando o Comportamento do Processo a partir dos Dados

Até este momento, os dados foram utilizados principalmente para reproduzir computacionalmente os cálculos desenvolvidos anteriormente e para construir representações gráficas das principais variáveis do processo.

Entretanto, uma das funções mais importantes da Computação Científica é permitir que o engenheiro faça perguntas aos dados.

Em vez de simplesmente observar toda a tabela, podemos investigar condições específicas de operação.

Algumas questões relevantes são:

- Em qual instante o erro de controle se torna menor que 1°C ?
- Qual é a abertura da válvula nesse instante?
- Qual é a vazão de vapor correspondente?
- Quanto a potência térmica fornecida pelo vapor foi reduzida em relação à condição inicial?
- O controlador deixa de fornecer vapor quando a temperatura se aproxima do setpoint?

Essas perguntas transformam o DataFrame em uma ferramenta de investigação do processo.

1.10.1 Identificação de uma Condição de Operação

Inicialmente, considere a seguinte questão:

Em qual instante o erro se torna menor que 1°C ?

O aluno pode começar respondendo essa questão diretamente pelo Explorador de Variáveis.

Observe a coluna **erro**:

10.0
7.6
5.7
4.2
3.0
2.1
1.4
0.9
0.6
0.4

O primeiro valor inferior a 1 °C é:

$$e = 0,9\text{ °C}.$$

Esse valor ocorre em:

$$t = 7\text{ min}.$$

Podemos confirmar essa observação utilizando o próprio Pandas.

A expressão:

```
dados["erro"] < 1.0
```

verifica essa condição em todas as linhas do DataFrame.

Entretanto, em vez de visualizar apenas valores lógicos, podemos solicitar ao Pandas que apresente as linhas que satisfazem a condição:

```
# Medicoes nas quais o erro e menor que 1 grau Celsius
condicao_erro = dados.loc[dados["erro"] < 1.0]

print(condicao_erro)
```

Agora o programa não está simplesmente executando uma equação de controle. Ele está sendo utilizado para **selecionar condições de operação que possuem interesse para a análise de Engenharia**.

1.10.2 Análise da Condição em $t = 7\text{ min}$

Retorne ao Explorador de Variáveis e localize a linha correspondente a:

$$t = 7\text{ min}.$$

Para essa condição:

$$T_{PV} = 79,1\text{ °C},$$

$$e = 0,9^{\circ}\text{C},$$

$$u = 1,8\%,$$

$$u_v = 41,8\%,$$

e:

$$\dot{m}_v = 0,418 \text{ kg/min.}$$

Compare essa condição com o instante inicial:

$$t = 0 \text{ min.}$$

Na condição inicial:

$$T_{PV} = 70^{\circ}\text{C},$$

$$e = 10^{\circ}\text{C},$$

$$u_v = 60\%$$

e:

$$\dot{m}_v = 0,600 \text{ kg/min.}$$

Portanto, a aproximação da temperatura ao setpoint provocou uma redução da vazão de vapor de:

$$0,600 \longrightarrow 0,418 \text{ kg/min.}$$

Isso representa uma redução aproximada de:

$$\frac{0,600 - 0,418}{0,600} \times 100 \approx 30,3\%.$$

Como a potência térmica fornecida pelo vapor é diretamente proporcional à sua vazão, a potência também apresenta redução da mesma ordem.

Essa informação dificilmente seria percebida apenas observando isoladamente as equações. O tratamento computacional permite relacionar diferentes variáveis e quantificar as consequências da atuação do controlador.

1.10.3 Uma Pergunta Importante sobre a Condição Próxima ao Setpoint

Observe novamente a condição:

$$T_{PV} = 79,1\text{ }^{\circ}\text{C}.$$

A temperatura está muito próxima do setpoint:

$$T_{SP} = 80\text{ }^{\circ}\text{C}.$$

Entretanto, a válvula ainda apresenta abertura de:

$$u_v = 41,8\%.$$

Consequentemente, ainda existe vazão de vapor:

$$\dot{m}_v = 0,418\text{ kg/min}.$$

Portanto, aproximar-se do setpoint não significa interromper o fornecimento de energia.

O trocador precisa continuar recebendo vapor porque a corrente de processo continua entrando no equipamento e precisa ser aquecida continuamente.

Esse resultado reforça a interpretação física da abertura nominal u_0 .

A energia fornecida pelo sistema de aquecimento não é necessária apenas para elevar a temperatura. Ela também é necessária para **manter uma condição de operação**.

1.11 Resposta do Sistema diante de uma Perturbação

Até este momento, a análise foi realizada durante uma condição em que a temperatura se aproximava progressivamente do valor desejado.

Entretanto, processos industriais estão continuamente sujeitos a perturbações.

Considere agora que, durante a operação do trocador, a vazão da corrente de processo aumente repentinamente.

Inicialmente:

$$\dot{m}_{in} = 3,0\text{ kg/min}.$$

No instante:

$$t = 10\text{ min},$$

a vazão aumenta para:

$$\boxed{\dot{m}_{in} = 4,0\text{ kg/min}}.$$

O setpoint permanece inalterado:

$$T_{SP} = 80^{\circ}\text{C}.$$

Essa alteração representa uma **perturbação de carga** no processo.

Uma quantidade maior de fluido passa a atravessar o trocador a cada minuto. Consequentemente, será necessária uma quantidade maior de energia para manter essa corrente na temperatura desejada.

Antes de observar os novos dados, procure responder qualitativamente:

O que deverá acontecer com a temperatura de saída quando a vazão da corrente de processo aumentar?

E, em seguida:

Como o controlador proporcional deverá reagir?

1.11.1 Novo Conjunto de Dados Experimentais

Considere que o sistema de aquisição tenha registrado simultaneamente a temperatura de saída e a vazão da corrente de processo.

O arquivo será denominado:

`dados_trocador_perturbacao.csv`

e conterá:

```
tempo,Tpv,m_in
0,70.0,3.0
1,72.4,3.0
2,74.3,3.0
3,75.8,3.0
4,77.0,3.0
5,77.9,3.0
6,78.6,3.0
7,79.1,3.0
8,79.4,3.0
9,79.6,3.0
10,79.2,4.0
11,78.9,4.0
12,78.8,4.0
13,78.8,4.0
14,78.8,4.0
```

Observe uma diferença importante em relação ao arquivo utilizado anteriormente.

Agora existem três colunas medidas:

tempo	Tpv	m_in
-------	-----	------

A vazão da corrente de processo, que anteriormente era considerada constante, passa agora a fazer parte do conjunto de dados.

Isso ocorre porque ela se tornou uma variável importante para compreender a perturbação aplicada ao sistema.

1.11.2 Importação dos Dados contendo a Perturbação

O novo arquivo pode ser importado utilizando novamente o Pandas:

```
# Importacao do conjunto de dados contendo a perturbacao
dados_p = pd.read_csv("dados_trocador_perturbacao.csv")

# Visualizacao inicial
print(dados_p)
```

Abra a variável `dados_p` no Explorador de Variáveis.

Antes de realizar qualquer cálculo, observe apenas as três colunas medidas.

Percorra inicialmente a coluna `m_in`.

Até:

$$t = 9 \text{ min},$$

temos:

$$\dot{m}_{in} = 3,0 \text{ kg/min.}$$

A partir de:

$$t = 10 \text{ min},$$

temos:

$$\dot{m}_{in} = 4,0 \text{ kg/min.}$$

Portanto, o instante da perturbação pode ser identificado diretamente no DataFrame.

Agora observe a coluna `Tpv`.

Antes da perturbação:

$$T_{PV} = 79,6 \text{ °C.}$$

Após o aumento da vazão, a temperatura apresenta:

$$79,2, \quad 78,9, \quad 78,8^\circ\text{C}.$$

A perturbação provocou, portanto, uma redução da temperatura de saída.

1.11.3 Interpretação Física da Perturbação

A razão dessa redução pode ser compreendida pelo balanço de energia.

A potência associada ao aquecimento da corrente de processo é:

$$\dot{Q}_{processo} = \dot{m}_{in}^* c_p (T_{PV} - T_{in}).$$

Quando:

$$\dot{m}_{in} \uparrow,$$

uma quantidade maior de massa precisa ser aquecida por unidade de tempo.

Portanto, mantendo inicialmente as demais condições:

$$\boxed{\dot{m}_{in} \uparrow \implies \dot{Q}_{processo} \uparrow}.$$

A perturbação aumenta a demanda térmica sobre o trocador.

Se o fornecimento de energia pelo vapor não aumentar instantaneamente na mesma proporção, a temperatura de saída tende a diminuir.

Assim:

$$\boxed{\dot{m}_{in} \uparrow \longrightarrow \text{demanda térmica} \uparrow \longrightarrow T_{PV} \downarrow}.$$

1.11.4 Resposta do Controlador Proporcional

Quando a temperatura diminui, o erro:

$$e = T_{SP} - T_{PV}$$

aumenta.

Para:

$$T_{PV} = 78,8^\circ\text{C},$$

temos:

$$e = 80 - 78,8,$$

portanto:

$$e = 1,2^{\circ}\text{C}.$$

Como:

$$K_p = 2 \frac{\%}{^{\circ}\text{C}},$$

a ação proporcional será:

$$u = K_p e,$$

resultando em:

$$u = (2)(1,2) = 2,4\%.$$

A abertura da válvula passa a ser:

$$u_v = u_0 + u,$$

logo:

$$u_v = 40 + 2,4,$$

e:

$$u_v = 42,4\%.$$

A vazão de vapor correspondente será:

$$\dot{m}_v = K_v u_v.$$

Assim:

$$\dot{m}_v = 0,424 \text{ kg/min}.$$

Portanto, o controlador reagiu à perturbação aumentando o fornecimento de vapor.

A sequência física pode ser representada por:

$$\dot{m}_{in} \uparrow \rightarrow T_{PV} \downarrow \rightarrow e \uparrow \rightarrow u \uparrow \rightarrow u_v \uparrow \rightarrow \dot{m}_v \uparrow.$$

Essa é a ação da malha de realimentação diante de uma perturbação.

1.11.5 Aplicação das Equações ao Novo DataFrame

As mesmas equações utilizadas anteriormente podem ser aplicadas ao novo conjunto de dados.

Inicialmente:

```
# Parametros do controlador
T_sp = 80.0
Kp = 2.0
u0 = 40.0
Kv = 0.01

# Calculo do erro
dados_p["erro"] = T_sp - dados_p["Tpv"]

# Acao proporcional
dados_p["u"] = Kp * dados_p["erro"]

# Abertura da valvula
dados_p["u_v"] = u0 + dados_p["u"]

# Vazao de vapor em kg/min
dados_p["m_v"] = Kv * dados_p["u_v"]
```

Abra novamente `dados_p` no Explorador de Variáveis.

Agora procure especificamente as linhas:

$$t = 9, \quad t = 10, \quad t = 11, \quad t = 12 \text{ min.}$$

Acompanhe essas linhas horizontalmente.

O objetivo é observar a propagação da perturbação através das diferentes variáveis da malha.

1.11.6 Incorporando a Perturbação ao Balanço de Energia

Existe agora uma diferença importante em relação ao programa anterior.

A vazão da corrente de processo não é mais uma constante.

Anteriormente foi utilizada uma variável escalar:

```
m_in = 3.0
```

Agora a vazão está armazenada na própria coluna:

```
dados_p["m_in"]
```

Portanto, sua conversão para kg/s também será realizada linha a linha:

```
# Vazao da corrente de processo em kg/s
dados_p["m_in_kg_s"] = dados_p["m_in"] / 60.0
```

A vazão de vapor será igualmente convertida:

```
# Vazao de vapor em kg/s
dados_p["m_v_kg_s"] = dados_p["m_v"] / 60.0
```

Os parâmetros térmicos permanecem:

```
# Parametros termicos
lambda_v = 2.0e6
cp = 4000.0
T_in = 30.0
U_L = 20.0
T_amb = 25.0
C = 1.0e5
```

A potência fornecida pelo vapor será:

```
# Potencia fornecida pelo vapor
dados_p["Q_v"] = dados_p["m_v_kg_s"] * lambda_v
```

A potência necessária para aquecer a corrente será agora calculada utilizando a vazão existente em cada linha:

```
# Potencia associada ao aquecimento da corrente
dados_p["Q_processo"] = (
    dados_p["m_in_kg_s"]
    * cp
    * (dados_p["Tpv"] - T_in)
)
```

As perdas térmicas:

```
# Perdas termicas
dados_p["Q_perdas"] = (
    U_L
    * (dados_p["Tpv"] - T_amb)
)
```

A potência líquida:

```
# Potencia termica liquida
dados_p["Q_liq"] = (
    dados_p["Q_v"]
    - dados_p["Q_processo"]
    - dados_p["Q_perdas"]
)
```

E a taxa de variação da temperatura:

```
# Taxa de variacao da temperatura
dados_p["dT_dt"] = dados_p["Q_liq"] / C

# Conversao para graus Celsius por minuto
dados_p["dT_dt_min"] = dados_p["dT_dt"] * 60.0
```

O novo DataFrame contém agora não apenas a resposta do controlador, mas também a própria perturbação que originou essa resposta.

1.11.7 Visualização da Perturbação

A primeira variável que deve ser representada é justamente aquela que sofreu a alteração.

No programa:

```
# Grafico da vazao da corrente de processo
plt.figure()

plt.plot(
    dados_p["tempo"],
    dados_p["m_in"],
    marker="o"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Vazao da corrente (kg/min)")
plt.title("Perturbacao na vazao da corrente de processo")
plt.grid()

plt.show()
```

O gráfico apresenta claramente uma mudança de:

$$3,0 \longrightarrow 4,0 \text{ kg/min}$$

em:

$$t = 10 \text{ min.}$$

Agora existe uma referência temporal clara para analisar os demais gráficos.

1.11.8 Efeito da Perturbação sobre a Temperatura

Em seguida, represente a temperatura medida e o setpoint:

```

# Resposta da temperatura diante da perturbacao
plt.figure()

plt.plot(
    dados_p["tempo"],
    dados_p["Tp"],
    marker="o",
    label="Tp"
)

plt.axhline(
    y=T_sp,
    linestyle="--",
    label="Tsp"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Resposta da temperatura a perturbacao")
plt.grid()
plt.legend()

plt.show()

```

Observe o comportamento antes e depois de:

$$t = 10 \text{ min.}$$

Antes da perturbação, a temperatura estava próxima do setpoint.

Quando a vazão da alimentação aumenta, a temperatura diminui.

Posteriormente, a ação do controlador reduz a intensidade dessa queda e o sistema se aproxima de uma nova condição de operação.

1.11.9 Efeito da Perturbação sobre o Erro

O gráfico do erro pode ser construído por:

```

# Erro de controle diante da perturbacao
plt.figure()

plt.plot(
    dados_p["tempo"],
    dados_p["erro"],
    marker="o"
)

```

```
plt.axhline(
y=0.0,
linestyle="--"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Erro (graus Celsius)")
plt.title("Erro de controle apos a perturbacao")
plt.grid()

plt.show()
```

Após a perturbação, o erro volta a aumentar.
Esse comportamento confirma a sequência:

$$\dot{m}_{in} \uparrow \rightarrow T_{PV} \downarrow \rightarrow e \uparrow.$$

1.11.10 Resposta da Válvula à Perturbação

Para observar a reação do controlador:

```
# Resposta da valvula diante da perturbacao
plt.figure()

plt.plot(
dados_p["tempo"],
dados_p["u_v"],
marker="o"
)

plt.axhline(
y=u0,
linestyle="--",
label="Abertura nominal"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Abertura da valvula (%)")
plt.title("Resposta da valvula a perturbacao")
plt.grid()
plt.legend()

plt.show()
```

A redução da temperatura aumenta o erro e, consequentemente, a abertura da válvula.

Portanto, o controlador reage à perturbação aumentando o fornecimento de energia ao processo.

Até aqui, o comportamento parece exatamente aquele que seria desejado de uma malha de controle.

Entretanto, os dados revelam uma característica importante do controlador proporcional.

1.12 O Surgimento do Offset no Controle Proporcional

Observe as últimas medições do processo:

$$T_{PV} \approx 78,8^\circ\text{C}.$$

O setpoint permanece:

$$T_{SP} = 80^\circ\text{C}.$$

Portanto, mesmo após a resposta do controlador, permanece aproximadamente:

$$e = 80 - 78,8,$$

ou:

$$e \approx 1,2^\circ\text{C}.$$

O sistema atingiu uma região aproximadamente estável, mas a variável controlada não retornou exatamente ao setpoint.

A diferença permanente entre o valor desejado e o valor atingido pelo processo é denominada **offset** ou **erro em regime permanente**.

Neste caso:

$$\text{offset} \approx 1,2^\circ\text{C}.$$

A presença desse erro não significa necessariamente que o controlador proporcional deixou de atuar.

Na verdade, ocorre exatamente o contrário.

Para fornecer a quantidade adicional de energia exigida pela nova condição de operação, a válvula precisa permanecer mais aberta do que anteriormente.

Na nova condição:

$$u_v \approx 42,4\%.$$

Como:

$$u_v = u_0 + u,$$

é necessária uma ação proporcional de aproximadamente:

$$u \approx 2,4\%.$$

Entretanto, para um controlador proporcional:

$$u = K_p e.$$

Portanto, para existir:

$$u \neq 0,$$

é necessário que:

$$e \neq 0.$$

Esse resultado fornece uma interpretação extremamente importante do offset.

O controlador proporcional necessita manter um erro para produzir uma ação corretiva permanente.

A sequência pode ser representada por:

$$\boxed{\text{maior demanda térmica} \rightarrow \text{maior abertura necessária} \rightarrow u > 0 \rightarrow e > 0}.$$

Assim, após determinadas perturbações, o processo pode encontrar uma nova condição de equilíbrio apresentando erro diferente de zero.

1.12.1 Uma Questão para o Próximo Passo

O controlador proporcional conseguiu reagir à perturbação.

Ele aumentou a abertura da válvula.

Aumentou a vazão de vapor.

Aumentou o fornecimento de energia.

E evitou uma redução maior da temperatura.

Entretanto, permaneceu:

$$T_{PV} < T_{SP}.$$

Surge então uma nova questão de Controle de Processos:

Como manter a ação adicional necessária sobre a válvula e, ao mesmo tempo, eliminar o erro em regime permanente?

Para responder a essa questão será necessário acrescentar ao controlador uma nova forma de ação. Essa ação deverá considerar não apenas o valor instantâneo do erro, mas também sua permanência ao longo do tempo. Essa é a origem da **ação integral**.

1.13 Introdução à Ação Integral

A análise da resposta do controlador proporcional diante da perturbação revelou uma característica importante da malha de controle.

Após o aumento da vazão da corrente de processo, o controlador reagiu aumentando a abertura da válvula de vapor. Entretanto, a temperatura não retornou exatamente ao setpoint.

Na nova condição de operação, observou-se aproximadamente:

$$T_{SP} = 80^{\circ}\text{C}$$

e:

$$T_{PV} = 78,8^{\circ}\text{C}.$$

Portanto, permaneceu um erro de:

$$e = T_{SP} - T_{PV},$$

resultando em:

$$\boxed{e = 1,2^{\circ}\text{C}}.$$

Esse erro é necessário para que o controlador proporcional mantenha uma ação adicional sobre a válvula.

Como:

$$u = K_p e,$$

se o erro fosse completamente eliminado:

$$e = 0,$$

teríamos:

$$u = 0.$$

Entretanto, a nova condição de operação exige uma abertura adicional da válvula para compensar o aumento da demanda térmica provocado pela maior vazão da corrente de processo.

Surge então uma questão fundamental:

Como produzir uma ação adicional sobre a válvula sem depender da existência permanente de um erro de controle?

Para responder a essa pergunta, será necessário introduzir uma nova forma de ação de controle.

Antes de apresentar sua equação matemática, entretanto, vamos observar mais cuidadosamente o comportamento do erro ao longo do tempo.

1.13.1 O Erro que Permanece ao Longo do Tempo

Considere inicialmente uma situação simplificada na qual, após uma perturbação, permanece durante alguns minutos um erro constante de:

$$e = 1,2^\circ\text{C}.$$

Suponha que o sistema seja observado a cada:

$$\Delta t = 1 \text{ min.}$$

Podemos construir manualmente a seguinte tabela:

Tempo (min)	Erro ($^\circ\text{C}$)
0	1,2
1	1,2
2	1,2
3	1,2
4	1,2

O valor instantâneo do erro não está aumentando.

Em todos os instantes:

$$e = 1,2^\circ\text{C}.$$

Entretanto, existe outra informação importante.

O erro está **permanecendo** ao longo do tempo.

Essa permanência pode ser considerada pelo controlador.

Para compreender essa ideia, vamos acumular o erro a cada intervalo de tempo.

1.13.2 Acumulação Manual do Erro

No primeiro intervalo, temos:

$$e_1 = 1,2^\circ\text{C}$$

durante:

$$\Delta t = 1 \text{ min.}$$

A contribuição acumulada nesse intervalo será:

$$e_1 \Delta t = (1,2)(1) = 1,2^\circ\text{C min.}$$

No segundo intervalo, o mesmo erro permanece:

$$e_2 = 1,2^\circ\text{C.}$$

Portanto, uma nova contribuição é acrescentada:

$$e_2 \Delta t = 1,2^\circ\text{C min.}$$

O valor acumulado passa a ser:

$$1,2 + 1,2 = 2,4^\circ\text{C min.}$$

No terceiro intervalo:

$$2,4 + 1,2 = 3,6^\circ\text{C min.}$$

Prosseguindo da mesma maneira:

Tempo (min)	Erro ($^\circ\text{C}$)	Erro acumulado ($^\circ\text{C min}$)
0	1,2	0,0
1	1,2	1,2
2	1,2	2,4
3	1,2	3,6
4	1,2	4,8

Observe cuidadosamente o que aconteceu.

O erro instantâneo permaneceu constante:

$$1,2, \quad 1,2, \quad 1,2, \quad 1,2^\circ\text{C.}$$

Entretanto, o erro acumulado aumentou:

$$0 \rightarrow 1,2 \rightarrow 2,4 \rightarrow 3,6 \rightarrow 4,8^\circ\text{C min.}$$

Portanto:

$$\boxed{\text{erro persistente} \implies \text{erro acumulado crescente}}.$$

Essa é a ideia fundamental que dará origem à ação integral.

1.13.3 O Controlador Passa a Considerar a História do Erro

O controlador proporcional utiliza somente o erro existente no instante considerado.

Para:

$$e = 1,2^{\circ}\text{C},$$

e:

$$K_p = 2 \frac{\%}{^{\circ}\text{C}},$$

a ação proporcional é:

$$u_P = K_p e,$$

portanto:

$$u_P = (2)(1,2) = 2,4\%.$$

Se o erro permanecer constante, a ação proporcional também permanecerá constante:

$$u_P = 2,4\%.$$

O controlador proporcional, portanto, responde ao valor do erro **naquele instante**.

Uma ação integral apresenta uma característica diferente.

Ela considera que o erro:

$$1,2^{\circ}\text{C}$$

não ocorreu apenas uma vez.

Ele permaneceu durante vários intervalos de tempo.

Consequentemente, a ação integral utiliza a **história do erro**.

Essa diferença pode ser resumida por:

ação proporcional $\longrightarrow$ erro presente

enquanto:

ação integral $\longrightarrow$ erro acumulado ao longo do tempo.

É por essa razão que frequentemente se diz que a ação integral introduz uma forma de **memória** no controlador.

1.13.4 Da Soma dos Erros à Integral

Nos cálculos anteriores, o erro foi acumulado em intervalos discretos de tempo.

Para um intervalo de amostragem Δt , podemos escrever:

$$I_k = I_{k-1} + e_k \Delta t,$$

em que:

$$I_k$$

representa o erro acumulado até o instante k .

Para o primeiro intervalo:

$$I_1 = I_0 + e_1 \Delta t.$$

Considerando:

$$I_0 = 0,$$

temos:

$$I_1 = 0 + (1,2)(1) = 1,2^\circ\text{C min.}$$

No instante seguinte:

$$I_2 = I_1 + e_2 \Delta t,$$

portanto:

$$I_2 = 1,2 + (1,2)(1) = 2,4^\circ\text{C min.}$$

Essa soma sucessiva representa, em tempo discreto, a acumulação da área associada ao erro ao longo do tempo.

Quando o tempo é tratado de maneira contínua, essa acumulação é representada matematicamente por:

$$I(t) = \int_0^t e(\tau) d\tau.$$

A integral não deve ser interpretada aqui apenas como uma operação matemática.

Fisicamente, ela representa a acumulação do erro durante o tempo em que esse erro permaneceu presente no processo.

1.13.5 Unidades do Erro Acumulado

A análise das unidades ajuda a compreender o significado da grandeza integral.

O erro de temperatura possui unidade:

$$[e] = ^\circ\text{C}.$$

O intervalo de tempo utilizado nesta análise possui unidade:

$$[\Delta t] = \text{min}.$$

Portanto:

$$[I] = [e][\Delta t],$$

e conseqüentemente:

$$\boxed{[I] = ^\circ\text{C min}}.$$

Essa observação será importante quando for introduzido o ganho integral do controlador.

Se a saída do controlador for expressa em porcentagem, o termo integral também deverá resultar em porcentagem.

1.13.6 Transformando o Erro Acumulado em uma Ação de Controle

Acumular o erro, por si só, ainda não modifica a válvula.

É necessário transformar o valor acumulado em uma ação de controle.

Para isso, introduz-se o ganho integral:

$$K_i.$$

A contribuição integral para a saída do controlador é definida por:

$$u_I(t) = K_i \int_0^t e(\tau) d\tau.$$

Utilizando a variável $I(t)$:

$$u_I(t) = K_i I(t).$$

Como $I(t)$ possui unidade:

$$^\circ\text{C min},$$

e desejamos que u_I seja expresso em porcentagem, o ganho integral deverá possuir unidade:

$$\boxed{[K_i] = \frac{\%}{^\circ\text{C min}}}.$$

Considere, inicialmente, o valor:

$$K_i = 0,5 \frac{\%}{^{\circ}\text{C min}}.$$

Utilizando os valores acumulados anteriormente, podemos calcular a ação integral.

Para:

$$I_1 = 1,2^{\circ}\text{C min},$$

temos:

$$u_{I,1} = (0,5)(1,2) = 0,6\%.$$

No instante seguinte:

$$I_2 = 2,4^{\circ}\text{C min},$$

portanto:

$$u_{I,2} = (0,5)(2,4) = 1,2\%.$$

Prosseguindo:

Tempo (min)	Erro ($^{\circ}\text{C}$)	Erro acumulado ($^{\circ}\text{C min}$)	Ação integral (%)
0	1,2	0,0	0,0
1	1,2	1,2	0,6
2	1,2	2,4	1,2
3	1,2	3,6	1,8
4	1,2	4,8	2,4

Observe agora uma diferença fundamental.

Mesmo que o erro permaneça constante:

$$e = 1,2^{\circ}\text{C},$$

a ação integral continua aumentando:

$$0 \rightarrow 0,6 \rightarrow 1,2 \rightarrow 1,8 \rightarrow 2,4\%.$$

Isso ocorre porque o controlador reconhece que o erro continua presente ao longo do tempo.

1.13.7 Combinando as Ações Proporcional e Integral

Agora existem duas contribuições distintas para a ação do controlador.

A primeira é a ação proporcional:

$$u_P = K_p e.$$

A segunda é a ação integral:

$$u_I = K_i I.$$

A combinação dessas duas ações dá origem ao controlador **Proporcional–Integral**, ou controlador **PI**.

A ação total é:

$$u_{PI} = u_P + u_I.$$

Substituindo as expressões correspondentes:

$$u_{PI} = K_p e + K_i \int_0^t e(\tau) d\tau.$$

Em tempo discreto, podemos utilizar:

$$I_k = I_{k-1} + e_k \Delta t$$

e:

$$u_{PI,k} = K_p e_k + K_i I_k.$$

Essa forma é particularmente conveniente para implementação computacional, pois os dados provenientes de sistemas de aquisição são normalmente registrados em instantes discretos.

1.13.8 Cálculo Manual da Ação PI

Considere novamente:

$$K_p = 2 \frac{\%}{^{\circ}\text{C}}$$

e:

$$K_i = 0,5 \frac{\%}{^{\circ}\text{C min}}.$$

Para o erro constante:

$$e = 1,2^{\circ}\text{C},$$

a contribuição proporcional permanece:

$$u_P = (2)(1,2) = 2,4\%.$$

No primeiro minuto, a contribuição integral é:

$$u_I = 0,6\%.$$

Portanto:

$$u_{PI} = 2,4 + 0,6 = 3,0\%.$$

No segundo minuto:

$$u_I = 1,2\%,$$

e:

$$u_{PI} = 2,4 + 1,2 = 3,6\%.$$

No terceiro minuto:

$$u_I = 1,8\%,$$

resultando em:

$$u_{PI} = 4,2\%.$$

No quarto minuto:

$$u_I = 2,4\%,$$

e:

$$u_{PI} = 4,8\%.$$

Podemos organizar os resultados:

t (min)	e (°C)	I (°C min)	u_P (%)	u_I (%)	u_{PI} (%)
0	1,2	0,0	2,4	0,0	2,4
1	1,2	1,2	2,4	0,6	3,0
2	1,2	2,4	2,4	1,2	3,6
3	1,2	3,6	2,4	1,8	4,2
4	1,2	4,8	2,4	2,4	4,8

Agora é possível perceber claramente a diferença entre as duas ações.

A parcela proporcional permanece constante porque o erro permaneceu constante.

A parcela integral cresce porque o erro continua existindo ao longo do tempo.
Portanto:

$$\begin{array}{ll} u_P & \longrightarrow \text{ responde ao erro atual} \\ u_I & \longrightarrow \text{ responde à permanência do erro} \end{array}.$$

1.13.9 Efeito da Ação PI sobre a Válvula

A abertura da válvula continua sendo determinada em relação à abertura nominal:

$$u_v = u_0 + u_{PI}.$$

Considerando:

$$u_0 = 40\%,$$

podemos calcular:

Tempo (min)	u_{PI} (%)	u_v (%)
0	2,4	42,4
1	3,0	43,0
2	3,6	43,6
3	4,2	44,2
4	4,8	44,8

Esse resultado mostra o efeito físico da ação integral.

Enquanto o erro continuar positivo, a contribuição integral continuará aumentando a ação sobre a válvula.

Consequentemente:

$$u_v \uparrow,$$

o que provoca:

$$\dot{m}_v \uparrow,$$

e, portanto:

$$\dot{Q}_v \uparrow.$$

A sequência pode ser representada por:

$$e > 0 \rightarrow I \uparrow \rightarrow u_I \uparrow \rightarrow u_{PI} \uparrow \rightarrow u_v \uparrow \rightarrow \dot{m}_v \uparrow \rightarrow \dot{Q}_v \uparrow.$$

A maior potência térmica tende a elevar novamente a temperatura de saída.

À medida que:

$$T_{PV} \rightarrow T_{SP},$$

o erro diminui.

A ação proporcional também diminui.

Entretanto, a parcela integral já acumulada pode continuar fornecendo a ação adicional necessária para manter a nova condição de operação.

Essa é a característica que permite ao controlador PI eliminar o offset observado anteriormente com o controlador puramente proporcional.

1.13.10 Por que a Ação Integral Pode Eliminar o Offset?

A diferença entre os controladores P e PI pode agora ser compreendida fisicamente.

No controlador proporcional:

$$u_P = K_p e.$$

Para manter uma ação adicional:

$$u_P > 0,$$

é necessário manter:

$$e > 0.$$

Consequentemente, uma nova condição de carga pode exigir a permanência de um erro em regime permanente.

No controlador PI:

$$u_{PI} = K_p e + K_i I.$$

Quando o processo retorna ao setpoint:

$$e \rightarrow 0,$$

a parcela proporcional tende a:

$$u_P \rightarrow 0.$$

Entretanto, a parcela integral não precisa ser igual a zero.

Ela contém o efeito acumulado dos erros anteriores.

Assim, é possível ter:

$$e = 0$$

e, simultaneamente:

$$u_I \neq 0.$$

Consequentemente:

$$\boxed{e = 0 \quad \text{e} \quad u_{PI} \neq 0}.$$

Essa é a ideia central da eliminação do offset pela ação integral.

O controlador pode manter uma ação sobre o processo mesmo depois que o erro instantâneo foi eliminado.

1.13.11 O que Acontece quando o Erro Muda de Sinal?

Considere agora que a ação integral tenha elevado a temperatura ligeiramente acima do setpoint.

Por exemplo:

$$T_{PV} = 80,3^\circ\text{C}.$$

Como:

$$T_{SP} = 80^\circ\text{C},$$

o erro passa a ser:

$$e = 80 - 80,3,$$

portanto:

$$\boxed{e = -0,3^\circ\text{C}}.$$

Na forma discreta:

$$I_k = I_{k-1} + e_k \Delta t.$$

Como agora:

$$e_k < 0,$$

a nova contribuição para o termo acumulado será negativa.

Consequentemente:

$$I_k < I_{k-1}.$$

Isso significa que a ação integral começa a diminuir.

Portanto, a integral não cresce indefinidamente em condições normais de operação.

Seu comportamento depende do sinal e da duração do erro.

Podemos resumir:

$e > 0 \implies$	I aumenta
$e = 0 \implies$	I permanece constante
$e < 0 \implies$	I diminui

Essa propriedade permite que a ação integral se ajuste continuamente às necessidades energéticas do processo.

1.13.12 Preparação para a Implementação Computacional

Até este momento, a ação integral foi construída manualmente. Esse procedimento foi importante para compreender que a integral representa uma acumulação do erro ao longo do tempo. Agora podemos retornar ao conjunto de dados do trocador de calor. O objetivo será acrescentar progressivamente ao DataFrame novas colunas representando:

$$e_k \rightarrow I_k \rightarrow u_{P,k} \rightarrow u_{I,k} \rightarrow u_{PI,k} \rightarrow u_{v,k}.$$

A tabela computacional permitirá acompanhar, linha a linha, o surgimento da ação integral. No Explorador de Variáveis será possível observar simultaneamente o erro instantâneo e o erro acumulado. Assim, o aluno poderá verificar diretamente que essas duas grandezas possuem comportamentos diferentes. A coluna do erro responderá à condição atual do processo. A coluna do erro acumulado conservará informações sobre aquilo que ocorreu nos instantes anteriores. Somente depois de observar esse comportamento numericamente serão construídos os gráficos correspondentes.

1.14 Implementação Computacional da Ação Integral

A ação integral foi inicialmente construída por meio de cálculos manuais.

Para um sistema amostrado em intervalos de tempo Δt , foi utilizada a relação:

$$I_k = I_{k-1} + e_k \Delta t.$$

Essa equação possui uma característica diferente das expressões computacionais utilizadas anteriormente.

Por exemplo, o erro foi calculado por:

```
dados_p["erro"] = T_sp - dados_p["Tpv"]
```

Cada valor da coluna **erro** pode ser determinado diretamente a partir da temperatura existente na mesma linha.

Da mesma maneira:

```
dados_p["u"] = Kp * dados_p["erro"]
```

permite calcular a ação proporcional de cada linha independentemente das demais.

A ação integral apresenta uma característica diferente.

Para determinar I_k , é necessário conhecer:

$$I_{k-1}.$$

Portanto:

o valor atual depende do valor calculado anteriormente.

Essa dependência representa computacionalmente a memória introduzida pela ação integral.

1.14.1 Construção da Coluna do Erro Integral

Antes de realizar o cálculo, será criada uma nova coluna no DataFrame.

Inicialmente, todos os seus valores serão iguais a zero:

```
# Criação da coluna destinada ao erro acumulado
dados_p["erro_integral"] = 0.0
```

Execute somente essa instrução e abra novamente a variável `dados_p` no Explorador de Variáveis.

Uma nova coluna deverá aparecer:

`erro_integral`

Nesse momento, todos os valores dessa coluna são iguais a zero.

Isso significa que a estrutura necessária para armazenar a memória do controlador foi criada, mas o erro ainda não foi acumulado.

Essa separação entre **criar uma variável** e **calcular seus valores** é útil para compreender o funcionamento do algoritmo.

1.14.2 Definição do Intervalo de Amostragem

Os dados utilizados neste exemplo foram registrados em intervalos de:

$$\Delta t = 1 \text{ min.}$$

No programa:

```
# Intervalo de amostragem (min)
dt = 1.0
```

Como o erro está expresso em °C e o intervalo de tempo em minutos, a coluna `erro_integral` apresentará unidade:

$$^{\circ}\text{C min.}$$

Manter a unidade de tempo consistente será fundamental posteriormente, pois o ganho integral K_i será definido utilizando minutos.

1.14.3 O Primeiro Valor da Integral

Considere inicialmente que, no começo da análise, o controlador ainda não possui erro acumulado.

Portanto:

$$I_0 = 0.$$

Essa condição inicial já foi estabelecida quando a coluna `erro_integral` foi preenchida com zeros.

No instante seguinte:

$$I_1 = I_0 + e_1 \Delta t.$$

No Python, poderíamos calcular especificamente esse valor por:

```
dados_p.loc[1, "erro_integral"] = (
    dados_p.loc[0, "erro_integral"]
    + dados_p.loc[1, "erro"] * dt
)
```

Observe cuidadosamente essa instrução.

O termo:

```
dados_p.loc[0, "erro_integral"]
```

representa o valor acumulado no instante anterior.

O termo:

```
dados_p.loc[1, "erro"] * dt
```

representa a nova contribuição do erro durante o intervalo de amostragem.

Portanto, a instrução computacional reproduz diretamente:

$$\boxed{I_1 = I_0 + e_1 \Delta t}.$$

1.14.4 Por que Não Fazer Todos os Cálculos Manualmente?

O mesmo procedimento poderia ser repetido para a linha seguinte:

$$I_2 = I_1 + e_2 \Delta t,$$

depois:

$$I_3 = I_2 + e_3 \Delta t,$$

e assim sucessivamente.

Entretanto, um sistema industrial pode possuir centenas, milhares ou milhões de registros.

Escrever uma instrução diferente para cada linha seria inviável.

O padrão matemático, porém, é sempre o mesmo:

$$I_k = I_{k-1} + e_k \Delta t.$$

Quando uma mesma operação precisa ser repetida várias vezes, podemos utilizar uma estrutura de repetição.

1.14.5 Construção da Integral com uma Estrutura de Repetição

No Python, uma estrutura `for` pode percorrer sequencialmente as linhas do DataFrame.

Neste exemplo:

```
# Calculo do erro acumulado ao longo do tempo
for k in range(1, len(dados_p)):

    dados_p.loc[k, "erro_integral"] = (
        dados_p.loc[k - 1, "erro_integral"]
        + dados_p.loc[k, "erro"] * dt
    )
```

Não execute essa estrutura mecanicamente.

Antes, observe sua correspondência com a equação:

$$I_k = I_{k-1} + e_k \Delta t.$$

No programa:

```
dados_p.loc[k - 1, "erro_integral"]
```

representa:

$$I_{k-1}.$$

A expressão:

```
dados_p.loc[k, "erro"]
```

representa:

$$e_k.$$

E:

```
dt
```

representa:

$$\Delta t.$$

Portanto, o código não é uma operação independente do modelo matemático.

Ele é uma tradução direta da equação utilizada para representar a memória do controlador.

1.14.6 Acompanhando a Construção da Integral

Após executar o programa, abra novamente `dados_p` no Explorador de Variáveis.

Observe lado a lado as colunas:

tempo	Tpv	erro	erro_integral
-------	-----	------	---------------

Percorra essas colunas verticalmente.

A coluna `erro` mostra o erro existente em cada instante.

A coluna `erro_integral`, por outro lado, contém informações acumuladas dos instantes anteriores.

Essa diferença pode ser representada por:

$$e_k = \text{informação do instante atual}$$

enquanto:

$$I_k = \text{informação acumulada até o instante atual}.$$

Essa é uma das diferenças fundamentais entre a ação proporcional e a ação integral.

1.14.7 Uma Observação Importante sobre o Intervalo de Tempo

Até este momento foi utilizado:

$$\Delta t = 1 \text{ min},$$

porque as medições do exemplo foram registradas em intervalos regulares de um minuto.

Entretanto, em um conjunto de dados experimental, os instantes podem estar armazenados diretamente na coluna `tempo`.

Nesse caso, o intervalo entre duas medições consecutivas pode ser calculado por:

$$\Delta t_k = t_k - t_{k-1}.$$

Essa forma é mais geral e permite trabalhar inclusive com intervalos de amostragem diferentes.

Para o conjunto de dados atual:

```
# Intervalo de tempo entre medicoes consecutivas
dados_p["dt"] = dados_p["tempo"].diff()

print(dados_p[["tempo", "dt"]])
```

O primeiro valor da coluna `dt` não possui um intervalo anterior associado. Por essa razão, o Pandas o representa como um valor ausente.

Para os demais registros:

$$\Delta t = 1 \text{ min.}$$

Neste primeiro estudo do controlador PI será mantido o valor constante:

```
dt = 1.0
```

A coluna calculada anteriormente serve, entretanto, para mostrar que o intervalo de amostragem também faz parte das informações necessárias para uma implementação correta da ação integral.

1.15 Construção Computacional das Ações P e I

Agora que o erro acumulado está disponível no DataFrame, podemos separar as duas contribuições do controlador PI.

A parcela proporcional será denominada:

$$u_P,$$

enquanto a parcela integral será denominada:

$$u_I.$$

Essa separação será mantida inicialmente para que seja possível visualizar quanto cada ação contribui para a saída total do controlador.

1.15.1 Cálculo da Parcela Proporcional

A contribuição proporcional permanece:

$$u_{P,k} = K_p e_k.$$

No programa:

```
# Ganho proporcional (%/grau Celsius)
Kp = 2.0

# Parcela proporcional da acao de controle (%)
dados_p["u_P"] = Kp * dados_p["erro"]
```

Abra novamente o DataFrame no Explorador de Variáveis.

A nova coluna:

u_P

representa exclusivamente a contribuição proporcional.

1.15.2 Cálculo da Parcela Integral

Considere:

$$K_i = 0,5 \frac{\%}{^{\circ}\text{C min}}.$$

No programa:

```
# Ganho integral (%/(grau Celsius.min))
Ki = 0.5
```

A contribuição integral é:

$$u_{I,k} = K_i I_k.$$

Portanto:

```
# Parcela integral da acao de controle (%)
dados_p["u_I"] = Ki * dados_p["erro_integral"]
```

Uma nova coluna surgirá no DataFrame:

u_I.

Agora observe simultaneamente:

erro erro_integral u_P u_I

A relação entre essas colunas é:

$$e_k \longrightarrow u_{P,k}$$

e:

$$e_k \longrightarrow I_k \longrightarrow u_{I,k}.$$

A primeira relação depende diretamente do erro atual.

A segunda depende também da história dos erros anteriores.

1.15.3 Construção da Ação PI

A ação total do controlador PI é:

$$u_{PI,k} = u_{P,k} + u_{I,k}.$$

Como as duas parcelas já estão disponíveis no DataFrame, sua soma pode ser realizada diretamente:

```
# Acao total do controlador PI (%)
dados_p["u_PI"] = dados_p["u_P"] + dados_p["u_I"]
```

Agora o Explorador de Variáveis permite visualizar separadamente:

$$\boxed{u_P, \quad u_I, \quad u_{PI}}.$$

Essa separação é importante para compreender que a saída do controlador PI não corresponde a uma nova ação completamente independente.

Ela é formada pela combinação de duas contribuições:

$$\boxed{u_{PI} = u_P + u_I}.$$

1.15.4 Da Ação PI à Abertura da Válvula

A abertura da válvula será novamente calculada em relação à condição nominal:

$$u_v = u_0 + u_{PI}.$$

Considerando:

$$u_0 = 40\%,$$

temos:

```
# Abertura nominal da valvula (%)
```

```
u0 = 40.0
```

```
# Abertura calculada da valvula com controlador PI (%)
```

```
dados_p["u_v_PI"] = u0 + dados_p["u_PI"]
```

Como uma válvula real possui limites físicos de operação, sua abertura deve permanecer entre:

$$0\% \leq u_v \leq 100\%.$$

Podemos incorporar essa limitação utilizando:

```
# Aplicacao dos limites fisicos da valvula
```

```
dados_p["u_v_PI"] = dados_p["u_v_PI"].clip(
```

```
lower=0.0,
```

```
upper=100.0
```

```
)
```

A função `clip()` impede que valores matematicamente calculados sejam utilizados fora dos limites físicos adotados para o elemento final de controle.

Essa etapa mostra uma característica importante da modelagem de sistemas reais:

resultado matemático $\neq$ resultado fisicamente possível

em determinadas condições.

O modelo computacional deve, portanto, respeitar também as restrições físicas do equipamento.

1.15.5 Da Abertura da Válvula à Vazão de Vapor

Utilizando novamente:

$$K_v = 0,01 \frac{\text{kg/min}}{\%},$$

a vazão de vapor correspondente ao controlador PI será:

$$\dot{m}_{v,PI} = K_v u_{v,PI}.$$

No programa:

```
# Coeficiente simplificado da valvula (kg/min/%)
```

```
Kv = 0.01
```

```
# Vazao de vapor correspondente ao controlador PI (kg/min)
```

```
dados_p["m_v_PI"] = Kv * dados_p["u_v_PI"]
```

Nesse momento, o DataFrame passa a conter uma cadeia computacional completa para a atuação do controlador PI:

$$T_{PV} \rightarrow e \rightarrow \begin{cases} u_P \\ I \rightarrow u_I \end{cases} \rightarrow u_{PI} \rightarrow u_{v,PI} \rightarrow \dot{m}_{v,PI}.$$

Essa estrutura deve ser observada diretamente no Explorador de Variáveis.

1.15.6 Lendo o DataFrame como uma Malha de Controle PI

Abra o DataFrame `dados_p` no Explorador de Variáveis e localize as seguintes colunas:

```
tempo
Tpv
erro
erro_integral
u_P
u_I
u_PI
u_v_PI
m_v_PI
```

Agora escolha uma determinada linha e percorra-a horizontalmente.

A temperatura medida determina o erro.

O erro produz imediatamente uma contribuição proporcional.

Ao mesmo tempo, esse erro é incorporado à memória acumulada do controlador.

A memória produz a contribuição integral.

As duas parcelas são somadas.

A ação PI modifica a abertura da válvula.

A abertura modifica a vazão de vapor.

Portanto, uma única linha do DataFrame contém uma representação numérica da sequência de atuação do controlador.

Agora percorra verticalmente a coluna `erro`.

Em seguida, faça o mesmo com:

`erro_integral`.

As duas colunas não apresentam necessariamente a mesma tendência.

O erro instantâneo pode diminuir enquanto o valor acumulado ainda permanece significativo.

Esse comportamento representa numericamente a memória da ação integral.

1.15.7 Uma Primeira Visualização da Memória do Controlador

Antes de comparar controladores P e PI, é útil representar graficamente somente o erro acumulado.

No programa:

```
# Grafico do erro acumulado
plt.figure()

plt.plot(
    dados_p["tempo"],
    dados_p["erro_integral"],
    marker="o"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Erro acumulado (graus Celsius.min)")
plt.title("Memoria integral do controlador")
plt.grid()

plt.show()
```

Compare esse gráfico com o gráfico do erro instantâneo construído anteriormente.

O erro instantâneo informa:

qual é a diferença entre SP e PV agora?

O erro integral informa:

qual foi o efeito acumulado dessa diferença ao longo do tempo?

Essa distinção é fundamental para compreender o funcionamento do controlador PI.

1.15.8 Comparação entre as Contribuições Proporcional e Integral

Agora existe uma justificativa física para representar duas grandezas no mesmo gráfico.

As colunas `u_P` e `u_I` representam duas contribuições, expressas na mesma unidade, para a ação total do controlador.

No programa:

```
# Comparacao das parcelas proporcional e integral
plt.figure()

plt.plot(
    dados_p["tempo"],
```

```

dados_p["u_P"],
marker="o",
label="Acao proporcional"
)

plt.plot(
dados_p["tempo"],
dados_p["u_I"],
marker="o",
label="Acao integral"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Acao de controle (%)")
plt.title("Contribuicoes proporcional e integral")
plt.grid()
plt.legend()

plt.show()

```

Ao analisar esse gráfico, não procure apenas identificar qual curva possui maior valor.

Procure interpretar o motivo do comportamento de cada uma.

A ação proporcional acompanha diretamente o erro atual.

A ação integral carrega a influência dos erros acumulados anteriormente.

Portanto, as duas curvas representam maneiras diferentes de utilizar a informação produzida pela mesma malha de realimentação.

1.15.9 Visualização da Ação Total do Controlador PI

A ação total pode ser representada por:

```

# Acao total do controlador PI
plt.figure()

plt.plot(
dados_p["tempo"],
dados_p["u_PI"],
marker="o"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Acao de controle (%)")
plt.title("Acao total do controlador PI")
plt.grid()

```

```
plt.show()
```

O valor apresentado em cada instante corresponde à soma:

$$u_{PI} = u_P + u_I.$$

Portanto, o gráfico da ação PI somente pode ser plenamente interpretado depois que suas duas componentes foram analisadas separadamente.

Essa é uma razão importante para não iniciar a análise diretamente pela saída total do controlador.

A decomposição:

$$u_{PI} = \underbrace{u_P}_{\text{erro atual}} + \underbrace{u_I}_{\text{história do erro}}$$

permite compreender de onde surge cada parcela da ação de controle.

1.15.10 Da Equação ao DataFrame e do DataFrame ao Processo

O percurso realizado para a ação integral pode agora ser sintetizado.

Primeiro foi observado fisicamente um problema:

offset após uma perturbação.

Depois foi identificado que o erro permanecia ao longo do tempo.

Esse erro foi acumulado manualmente:

$$I_k = I_{k-1} + e_k \Delta t.$$

A acumulação foi transformada em ação de controle:

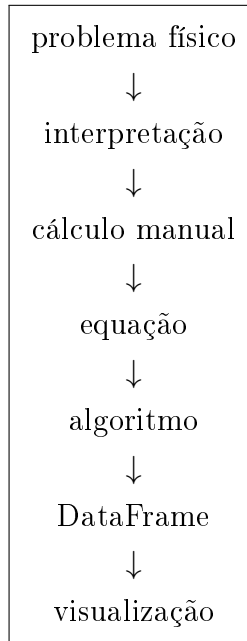
$$u_I = K_i I.$$

A ação integral foi combinada com a proporcional:

$$u_{PI} = u_P + u_I.$$

Finalmente, essas grandezas foram transformadas em colunas do DataFrame e puderam ser observadas no Explorador de Variáveis.

Assim:



A implementação computacional aparece, portanto, como consequência da compreensão do fenômeno e não como substituição dessa compreensão.

1.16 Da Análise de Dados à Simulação Dinâmica do Processo

Até este momento, a temperatura T_{PV} utilizada nos cálculos foi obtida a partir de medições armazenadas em arquivos de dados.

Nesse caso, o programa recebia a temperatura como uma informação já disponível e, a partir dela, calculava:

$$T_{PV} \rightarrow e \rightarrow u \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v.$$

Essa abordagem é adequada quando o objetivo é analisar dados provenientes de um processo real ou de um experimento.

Entretanto, surge agora uma nova questão:

É possível utilizar o modelo matemático para prever como a temperatura do processo evoluirá ao longo do tempo?

Para responder a essa pergunta, será necessário utilizar o balanço de energia para calcular a temperatura no instante seguinte.

Essa mudança representa a passagem da **análise de dados** para a **simulação dinâmica do processo**.

1.16.1 A Temperatura Deixa de Ser uma Entrada do Programa

Na análise anterior, uma linha típica do arquivo CSV fornecia diretamente:

$$T_{PV,k}.$$

A partir desse valor, as demais variáveis eram calculadas.

Na simulação dinâmica, entretanto, somente a condição inicial da temperatura será conhecida:

$$T_0.$$

A temperatura nos instantes seguintes deverá ser calculada pelo próprio modelo.

Portanto:

$$T_0 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \dots$$

será produzido computacionalmente.

A simulação passa, então, a representar uma sequência fechada de cálculos:

$$T_k \rightarrow e_k \rightarrow u_k \rightarrow u_{v,k} \rightarrow \dot{m}_{v,k} \rightarrow \dot{Q}_{v,k} \rightarrow \dot{Q}_{liq,k} \rightarrow \frac{dT}{dt} \rightarrow T_{k+1}.$$

Observe que a temperatura calculada em um instante será utilizada novamente como entrada do controlador no instante seguinte.

Essa realimentação é a própria essência da malha de controle.

1.16.2 Balanço de Energia Utilizado na Simulação

O modelo térmico desenvolvido anteriormente foi:

$$C \frac{dT}{dt} = \dot{Q}_v - \dot{Q}_{processo} - \dot{Q}_{perdas}.$$

Com:

$$\dot{Q}_{processo} = \dot{m}_{in} c_p (T - T_{in}),$$

e:

$$\dot{Q}_{perdas} = U_L (T - T_{amb}).$$

Portanto:

$$\frac{dT}{dt} = \frac{\dot{Q}_v - \dot{m}_{in} c_p (T - T_{in}) - U_L (T - T_{amb})}{C}.$$

Essa equação fornece a taxa instantânea de variação da temperatura.

Entretanto, para construir uma simulação computacional, ainda precisamos transformar essa derivada em uma nova temperatura.

1.16.3 Da Derivada à Temperatura no Instante Seguinte

Considere um pequeno intervalo de tempo:

$$\Delta t.$$

Uma aproximação simples para a evolução da temperatura é:

$$\frac{dT}{dt} \approx \frac{T_{k+1} - T_k}{\Delta t}.$$

Reorganizando:

$$T_{k+1} = T_k + \frac{dT}{dt} \Delta t.$$

Portanto:

$$T_{k+1} = T_k + \left(\frac{dT}{dt} \right)_k \Delta t.$$

Essa expressão será utilizada para atualizar a temperatura ao longo da simulação.

Esse procedimento corresponde ao método de Euler explícito.

Neste momento, entretanto, o objetivo principal não é estudar métodos numéricos em profundidade.

O método será utilizado como uma ferramenta simples para compreender a dinâmica da malha de controle.

1.16.4 Uma Observação Importante sobre as Unidades

No modelo térmico:

$$\frac{dT}{dt}$$

foi calculado inicialmente em:

$$^{\circ}\text{C/s}.$$

Portanto, o intervalo de integração utilizado na equação:

$$T_{k+1} = T_k + \frac{dT}{dt} \Delta t$$

deverá estar expresso em segundos.

Se desejarmos utilizar:

$$\Delta t = 1 \text{ s},$$

então:

$$dt = 1 \text{ s.}$$

Essa escolha será utilizada inicialmente por permitir uma simulação simples e suficientemente detalhada da dinâmica térmica do processo.

1.17 Simulação do Trocador com Controlador Proporcional

Antes de simular o controlador PI, será importante construir novamente o caso proporcional.

Isso permitirá utilizar o controlador P como referência para comparação.

Serão mantidos os parâmetros utilizados anteriormente:

$$T_{SP} = 80^\circ\text{C},$$

$$K_p = 2 \frac{\%}{^\circ\text{C}},$$

$$u_0 = 40\%,$$

$$K_v = 0,01 \frac{\text{kg/min}}{\%},$$

$$\lambda_v = 2,0 \times 10^6 \text{ J/kg},$$

$$c_p = 4000 \text{ J/(kg } ^\circ\text{C)},$$

$$T_{in} = 30^\circ\text{C},$$

$$T_{amb} = 25^\circ\text{C},$$

$$U_L = 20 \text{ W/}^\circ\text{C},$$

e:

$$C = 1,0 \times 10^5 \text{ J/}^\circ\text{C}.$$

A condição inicial será:

$$T_0 = 70^\circ\text{C}.$$

1.17.1 Definição do Tempo de Simulação

Inicialmente, será simulada uma operação de:

$$20 \text{ min.}$$

Como o modelo térmico utiliza segundos, temos:

$$20 \text{ min} = 1200 \text{ s.}$$

No programa:

```
# Tempo total de simulacao (s)
tempo_total = 1200

# Passo de integracao (s)
dt = 1.0
```

O vetor de tempo pode ser construído posteriormente.

Antes disso, vamos compreender como uma única etapa da simulação é realizada.

1.17.2 Primeira Etapa da Simulação

Considere inicialmente:

$$T_0 = 70^\circ\text{C}.$$

O erro será:

$$e_0 = T_{SP} - T_0.$$

Portanto:

$$e_0 = 80 - 70 = 10^\circ\text{C}.$$

A ação proporcional será:

$$u_{P,0} = K_p e_0.$$

Assim:

$$u_{P,0} = (2)(10) = 20\%.$$

A abertura da válvula será:

$$u_{v,0} = u_0 + u_{P,0},$$

resultando em:

$$u_{v,0} = 60\%.$$

A vazão de vapor será:

$$\dot{m}_{v,0} = K_v u_{v,0},$$

logo:

$$\dot{m}_{v,0} = 0,600 \text{ kg/min.}$$

Convertendo:

$$\dot{m}_{v,0} = 0,010 \text{ kg/s.}$$

A potência fornecida pelo vapor será:

$$\dot{Q}_{v,0} = \dot{m}_{v,0} \lambda_v,$$

portanto:

$$\dot{Q}_{v,0} = 20\,000 \text{ W.}$$

Esse resultado é o mesmo obtido anteriormente no cálculo manual.

1.17.3 Cálculo da Nova Temperatura

Para a condição inicial:

$$T_0 = 70^\circ\text{C},$$

e:

$$\dot{m}_{in} = 3,0 \text{ kg/min} = 0,050 \text{ kg/s},$$

a potência associada à corrente de processo será:

$$\dot{Q}_{processo,0} = (0,050)(4000)(70 - 30),$$

resultando em:

$$\dot{Q}_{processo,0} = 8000 \text{ W}.$$

As perdas térmicas serão:

$$\dot{Q}_{perdas,0} = 20(70 - 25),$$

logo:

$$\dot{Q}_{perdas,0} = 900 \text{ W}.$$

A potência líquida será:

$$\dot{Q}_{liq,0} = 20\,000 - 8000 - 900,$$

portanto:

$$\dot{Q}_{liq,0} = 11\,100 \text{ W}.$$

A taxa de variação da temperatura será:

$$\left(\frac{dT}{dt}\right)_0 = \frac{11\,100}{100\,000},$$

resultando em:

$$\left(\frac{dT}{dt}\right)_0 = 0,111 \text{ °C/s}.$$

Como:

$$dt = 1 \text{ s},$$

a temperatura no instante seguinte será:

$$T_1 = T_0 + \left(\frac{dT}{dt}\right)_0 dt.$$

Portanto:

$$T_1 = 70 + (0,111)(1),$$

resultando em:

$$\boxed{T_1 = 70,111 \text{ °C}}.$$

Esse é um momento importante da simulação.

Pela primeira vez, a temperatura no instante seguinte não foi medida.

Ela foi **calculada pelo modelo**.

1.17.4 O Ciclo de Cálculo se Repete

No instante seguinte, a nova temperatura:

$$T_1 = 70,111^\circ\text{C}$$

será utilizada para calcular um novo erro:

$$e_1 = T_{SP} - T_1.$$

Esse novo erro produzirá uma nova ação proporcional.

A nova ação modificará a abertura da válvula.

A abertura modificará a vazão de vapor.

A vazão de vapor modificará a potência térmica fornecida.

O balanço de energia produzirá uma nova derivada.

E essa derivada produzirá:

$$T_2.$$

Portanto:

$$T_0 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \dots$$

é construído iterativamente.

Essa estrutura reproduz computacionalmente a evolução temporal de uma malha de controle.

1.17.5 Implementação Computacional Passo a Passo

Inicialmente, importe as bibliotecas:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Agora defina os parâmetros do processo:

```
# Setpoint de temperatura (graus Celsius)
T_sp = 80.0

# Ganho proporcional (%/grau Celsius)
Kp = 2.0

# Abertura nominal da valvula (%)
u0 = 40.0
```

```

# Coeficiente simplificado da valvula (kg/min/%)
Kv = 0.01

# Calor latente do vapor (J/kg)
lambda_v = 2.0e6

# Calor especifico da corrente de processo (J/(kg.grau Celsius))
cp = 4000.0

# Temperatura de entrada da corrente (graus Celsius)
T_in = 30.0

# Temperatura ambiente (graus Celsius)
T_amb = 25.0

# Coeficiente global simplificado de perdas (W/grau Celsius)
U_L = 20.0

# Capacidade termica efetiva do sistema (J/grau Celsius)
C = 1.0e5

```

Em seguida, defina o tempo de simulação:

```

# Passo de integracao (s)
dt = 1.0

# Tempo total de simulacao (s)
tempo_total = 1200

# Vetor de tempo
tempo = np.arange(
    0.0,
    tempo_total + dt,
    dt
)

```

O vetor `tempo` conterà os instantes:

0, 1, 2, 3, ..., 1200s.

1.17.6 Criação das Variáveis da Simulação

Será necessário armazenar a evolução temporal das principais variáveis.

No programa:

```
# Numero de pontos da simulacao
```

```
n = len(tempo)
```

```
# Criacao dos vetores
```

```
T_P = np.zeros(n)
```

```
erro_P = np.zeros(n)
```

```
u_P = np.zeros(n)
```

```
u_v_P = np.zeros(n)
```

```
m_v_P = np.zeros(n)
```

```
Q_v_P = np.zeros(n)
```

```
Q_processo_P = np.zeros(n)
```

```
Q_perdas_P = np.zeros(n)
```

```
Q_liq_P = np.zeros(n)
```

Cada vetor armazenará a história temporal de uma grandeza.

Por exemplo:

`T_P`

armazenará a temperatura simulada com o controlador proporcional.

A condição inicial será:

```
# Condição inicial da temperatura
```

```
T_P[0] = 70.0
```

1.17.7 Introdução da Perturbação

A vazão da corrente de processo será inicialmente:

$$\dot{m}_{in} = 3,0 \text{ kg/min.}$$

Após:

10 min,

será introduzida a mesma perturbação estudada anteriormente:

$$\dot{m}_{in} = 4,0 \text{ kg/min.}$$

Como:

$$10 \text{ min} = 600 \text{ s,}$$

podemos construir:

```

# Vazao da corrente de processo (kg/min)
m_in = np.zeros(n)

# Condicao antes da perturbacao
m_in[tempo < 600.0] = 3.0

# Condicao depois da perturbacao
m_in[tempo >= 600.0] = 4.0

```

Agora a perturbação faz parte da própria simulação.

1.17.8 Construção do Algoritmo do Controlador P

A simulação será realizada utilizando uma estrutura de repetição.

No programa:

```

# Simulacao do sistema com controlador proporcional
for k in range(n - 1):

    # Erro de controle
    erro_P[k] = T_sp - T_P[k]

    # Acao proporcional
    u_P[k] = Kp * erro_P[k]

    # Abertura da valvula
    u_v_P[k] = u0 + u_P[k]

    # Limites fisicos da valvula
    u_v_P[k] = np.clip(
        u_v_P[k],
        0.0,
        100.0
    )

    # Vazao de vapor (kg/min)
    m_v_P[k] = Kv * u_v_P[k]

    # Conversao da vazao de vapor para kg/s
    m_v_kg_s = m_v_P[k] / 60.0

    # Potencia fornecida pelo vapor (W)
    Q_v_P[k] = m_v_kg_s * lambda_v

```

```

# Conversao da vazao de processo para kg/s
m_in_kg_s = m_in[k] / 60.0

# Potencia associada a corrente de processo (W)
Q_processo_P[k] = (
m_in_kg_s
* cp
* (T_P[k] - T_in)
)

# Perdas termicas (W)
Q_perdas_P[k] = (
U_L
* (T_P[k] - T_amb)
)

# Potencia termica liquida (W)
Q_liq_P[k] = (
Q_v_P[k]
- Q_processo_P[k]
- Q_perdas_P[k]
)

# Taxa de variacao da temperatura (graus Celsius/s)
dT_dt = Q_liq_P[k] / C

# Atualizacao da temperatura pelo metodo de Euler
T_P[k + 1] = (
T_P[k]
+ dT_dt * dt
)

```

Essa estrutura representa computacionalmente toda a malha.

1.17.9 Lendo o Algoritmo como uma Malha Física

O programa deve ser interpretado na mesma ordem em que o processo funciona.

Inicialmente:

$$T_P[k]$$

representa a temperatura atual.

O controlador calcula:

$$e_k = T_{SP} - T_k.$$

Em seguida:

$$u_{P,k} = K_p e_k.$$

A ação do controlador modifica:

$$u_{v,k}.$$

A válvula determina:

$$\dot{m}_{v,k}.$$

A vazão de vapor determina:

$$\dot{Q}_{v,k}.$$

O balanço de energia determina:

$$\dot{Q}_{liq,k}.$$

A potência líquida determina:

$$\left(\frac{dT}{dt} \right)_k.$$

Finalmente:

$$T_{k+1} = T_k + \left(\frac{dT}{dt} \right)_k dt.$$

O valor calculado retorna ao início da sequência.

Portanto:

$$\boxed{\text{processo} \rightarrow \text{medição} \rightarrow \text{erro} \rightarrow \text{controlador} \rightarrow \text{válvula} \rightarrow \text{processo}}.$$

Agora a realimentação não está apenas representada em um diagrama.

Ela está implementada numericamente.

1.17.10 Organização dos Resultados em um DataFrame

Após a simulação, os vetores podem ser organizados em um DataFrame.

No programa:

```
# Atualizacao das variaveis no ultimo ponto
erro_P[-1] = T_sp - T_P[-1]
```

```

u_P[-1] = Kp * erro_P[-1]

u_v_P[-1] = np.clip(
    u0 + u_P[-1],
    0.0,
    100.0
)

m_v_P[-1] = Kv * u_v_P[-1]

# Criacao do DataFrame com os resultados
sim_P = pd.DataFrame({
    "tempo_s": tempo,
    "tempo_min": tempo / 60.0,
    "T_P": T_P,
    "erro_P": erro_P,
    "u_P": u_P,
    "u_v_P": u_v_P,
    "m_v_P": m_v_P,
    "m_in": m_in
})

```

Abra a variável:

```
sim_P
```

no Explorador de Variáveis.

Agora o DataFrame possui uma interpretação diferente daquela utilizada anteriormente.

Antes, as linhas representavam medições registradas experimentalmente.

Agora, as linhas representam estados calculados pelo modelo em cada instante da simulação.

Portanto:

DataFrame anterior → dados medidos

enquanto:

DataFrame atual → dados simulados

Essa distinção é fundamental em Engenharia.

1.17.11 Visualização da Resposta com Controlador P

A temperatura simulada pode ser representada por:

```
# Resposta de temperatura com controlador P
```

```
plt.figure()

plt.plot(
    sim_P["tempo_min"],
    sim_P["T_P"],
    label="T com controlador P"
)

plt.axhline(
    y=T_sp,
    linestyle="--",
    label="Setpoint"
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Resposta do processo com controlador P")
plt.grid()
plt.legend()

plt.show()
```

A linha vertical em:

$$t = 10 \text{ min}$$

identifica o instante em que a vazão da corrente de processo aumenta.

Agora será possível observar diretamente como o modelo prevê a resposta da temperatura antes e depois da perturbação.

1.17.12 Uma Pergunta antes de Simular o Controlador PI

Antes de construir a simulação com controlador PI, analise o gráfico obtido para o controlador proporcional.

Observe principalmente o comportamento da temperatura após:

$$t = 10 \text{ min.}$$

Pergunte:

A temperatura retorna exatamente para 80 °C?

Em seguida, observe a coluna:

`erro_P`

no Explorador de Variáveis.

Se permanecer um erro diferente de zero após o sistema atingir uma nova região de equilíbrio, o comportamento observado numericamente corresponde ao offset discutido anteriormente.

A simulação permite, portanto, verificar dinamicamente uma característica que inicialmente havia sido compreendida por meio de argumentos físicos e matemáticos.

1.18 Preparação para a Simulação com Controlador PI

A estrutura utilizada para o controlador proporcional será mantida.

A principal diferença será a inclusão de uma memória do erro.

Em cada instante, além de calcular:

$$e_k,$$

será necessário atualizar:

$$I_k.$$

Portanto, o controlador PI utilizará:

$$u_{PI,k} = K_p e_k + K_i I_k.$$

A nova sequência será:

$$\boxed{T_k \rightarrow e_k \rightarrow I_k \rightarrow u_{PI,k} \rightarrow u_{v,k} \rightarrow \dot{m}_{v,k} \rightarrow \dot{Q}_{v,k} \rightarrow \dot{Q}_{liq,k} \rightarrow T_{k+1}}.$$

A comparação entre os controladores P e PI será realizada utilizando:

- a mesma condição inicial;
- o mesmo modelo térmico;
- o mesmo setpoint;
- a mesma perturbação;
- os mesmos parâmetros físicos do processo.

Dessa maneira, a diferença observada nas respostas será consequência da estratégia de controle utilizada.

1.19 Simulação do Trocador com Controlador PI

A simulação com controlador proporcional mostrou como o modelo matemático pode ser utilizado para calcular a evolução da temperatura ao longo do tempo.

Agora será introduzida a ação integral.

O objetivo não será construir um programa completamente novo.

Ao contrário, será mantida a mesma estrutura utilizada anteriormente, acrescentando apenas aquilo que diferencia o controlador PI do controlador P.

No controlador proporcional:

$$u_{P,k} = K_p e_k.$$

No controlador PI:

$$u_{PI,k} = K_p e_k + K_i I_k,$$

em que:

$$I_k$$

representa o erro acumulado.

Em tempo discreto:

$$I_k = I_{k-1} + e_k \Delta t.$$

Portanto, a principal novidade será a necessidade de armazenar e atualizar essa memória durante a simulação.

1.19.1 Um Cuidado Importante com as Unidades de Tempo

Na discussão inicial da ação integral, o tempo foi expresso em minutos.

Por essa razão, foi utilizado:

$$[K_i] = \frac{\%}{^{\circ}\text{C min}}.$$

Na simulação dinâmica, entretanto, o balanço de energia utiliza:

$$\frac{dT}{dt}$$

em $^{\circ}\text{C/s}$, e o passo de integração foi definido como:

$$dt = 1 \text{ s}.$$

Isso não significa que seja necessário redefinir o ganho integral.

Podemos manter K_i em:

$$\frac{\%}{^{\circ}\text{C min}}$$

desde que o intervalo utilizado na acumulação do erro seja convertido para minutos.

Assim:

$$\Delta t_I = \frac{dt}{60}.$$

Para:

$$dt = 1 \text{ s},$$

temos:

$$\Delta t_I = \frac{1}{60} \text{ min.}$$

Essa conversão é fundamental para manter a consistência dimensional da implementação.

No programa:

```
# Passo de integracao do modelo termico (s)
dt = 1.0

# Mesmo intervalo expresso em minutos para a acao integral
dt_min = dt / 60.0
```

Dessa maneira, duas escalas de tempo aparecem no mesmo algoritmo:

$$dt$$

em segundos, utilizado para integrar o balanço de energia, e:

$$dt_{\min}$$

em minutos, utilizado para acumular o erro de acordo com a unidade escolhida para K_i .

1.19.2 Definição do Ganho Integral

Será utilizado inicialmente:

$$K_i = 0,5 \frac{\%}{^{\circ}\text{C min}}.$$

No programa:

```
# Ganho integral (%/(grau Celsius.min))
Ki = 0.5
```

Esse valor será utilizado inicialmente para observar o comportamento do controlador.

Posteriormente, será possível investigar como diferentes valores de K_i modificam a resposta dinâmica do sistema.

1.19.3 Criação das Variáveis do Controlador PI

Assim como na simulação anterior, serão criados vetores para armazenar a evolução temporal das principais variáveis.

No programa:

```
# Criacao dos vetores da simulacao PI
T_PI = np.zeros(n)

erro_PI = np.zeros(n)
erro_integral_PI = np.zeros(n)

u_P_PI = np.zeros(n)
u_I_PI = np.zeros(n)
u_PI = np.zeros(n)

u_v_PI = np.zeros(n)
m_v_PI = np.zeros(n)

Q_v_PI = np.zeros(n)
Q_processo_PI = np.zeros(n)
Q_perdas_PI = np.zeros(n)
Q_liq_PI = np.zeros(n)
```

A condição inicial será exatamente a mesma utilizada para o controlador proporcional:

$$T_{PI,0} = 70^\circ\text{C}.$$

Portanto:

```
# Mesma condicao inicial utilizada na simulacao P
T_PI[0] = 70.0
```

Essa igualdade é importante para que a comparação entre os controladores seja realizada nas mesmas condições.

1.19.4 Inicialização da Memória Integral

No início da simulação, será adotado:

$$I_0 = 0.$$

No programa:

```
# Condicao inicial da memoria integral
erro_integral_PI[0] = 0.0
```

Fisicamente, essa condição significa que estamos iniciando a experiência computacional sem erro previamente acumulado.

Essa é uma condição inicial adotada para a simulação.

Em uma malha industrial que já estivesse operando anteriormente, o estado integral poderia possuir um valor diferente de zero.

1.19.5 Primeiro Instante: a Ação Proporcional

Na condição inicial:

$$T_{PI,0} = 70^{\circ}\text{C},$$

o erro será:

$$e_0 = 80 - 70 = 10^{\circ}\text{C}.$$

A parcela proporcional será:

$$u_{P,0} = K_p e_0.$$

Como:

$$K_p = 2 \frac{\%}{^{\circ}\text{C}},$$

temos:

$$u_{P,0} = 20\%.$$

Até esse ponto, não existe diferença em relação ao controlador proporcional estudado anteriormente.

1.19.6 Primeiro Instante: a Memória Integral

Como foi adotado:

$$I_0 = 0,$$

a contribuição integral inicial será:

$$u_{I,0} = K_i I_0.$$

Portanto:

$$u_{I,0} = 0.$$

A ação total inicialmente será:

$$u_{PI,0} = u_{P,0} + u_{I,0},$$

resultando em:

$$u_{PI,0} = 20\%.$$

Consequentemente, no instante inicial, os controladores P e PI produzem a mesma ação.

A diferença começará a surgir à medida que o erro permanecer presente e for acumulado ao longo do tempo.

1.19.7 Como a Memória Será Atualizada

Após um intervalo de:

$$dt = 1 \text{ s},$$

teremos:

$$dt_{\min} = \frac{1}{60} \text{ min.}$$

Se durante esse primeiro intervalo considerarmos aproximadamente:

$$e_0 = 10^\circ\text{C},$$

a contribuição acrescentada à memória será:

$$e_0 dt_{\min} = 10 \left(\frac{1}{60} \right),$$

portanto:

$$e_0 dt_{\min} = 0,1667^\circ\text{C min.}$$

Assim:

$$I_1 = I_0 + e_0 dt_{\min},$$

resultando em:

$$\boxed{I_1 = 0,1667^\circ\text{C min.}}$$

No instante seguinte, essa memória já estará disponível para o controlador.

A contribuição integral correspondente será:

$$u_{I,1} = K_i I_1.$$

Para:

$$K_i = 0,5 \frac{\%}{^{\circ}\text{C min}},$$

temos aproximadamente:

$$u_{I,1} = 0,0833\%.$$

O valor ainda é pequeno porque apenas um segundo de erro foi acumulado.

Entretanto, se o erro continuar presente, novas contribuições serão adicionadas à memória.

É dessa acumulação sucessiva que surge a ação integral.

1.19.8 Construção do Algoritmo PI

Agora podemos incorporar esse raciocínio ao algoritmo.

No programa:

```
# Simulacao do sistema com controlador PI
for k in range(n - 1):

    # Erro de controle
    erro_PI[k] = T_sp - T_PI[k]

    # Parcela proporcional (%)
    u_P_PI[k] = Kp * erro_PI[k]

    # Parcela integral (%)
    u_I_PI[k] = Ki * erro_integral_PI[k]

    # Acao total do controlador PI (%)
    u_PI[k] = u_P_PI[k] + u_I_PI[k]

    # Abertura da valvula (%)
    u_v_PI[k] = u0 + u_PI[k]

    # Limites fisicos da valvula
    u_v_PI[k] = np.clip(
        u_v_PI[k],
        0.0,
```

CAPÍTULO 1. CONTROLE DE TEMPERATURA EM TROCADORES DE CALOR COM COMP

100.0

)

Vazao de vapor (kg/min)

$m_v_PI[k] = K_v * u_v_PI[k]$

Conversao da vazao de vapor para kg/s

$m_v_kg_s = m_v_PI[k] / 60.0$

Potencia fornecida pelo vapor (W)

$Q_v_PI[k] = m_v_kg_s * \lambda_v$

Conversao da vazao de processo para kg/s

$m_in_kg_s = m_in[k] / 60.0$

Potencia associada a corrente de processo (W)

$Q_processo_PI[k] = ($
 $m_in_kg_s$
 $* c_p$
 $* (T_PI[k] - T_in)$
 $)$

Perdas termicas (W)

$Q_perdas_PI[k] = ($
 U_L
 $* (T_PI[k] - T_amb)$
 $)$

Potencia termica liquida (W)

$Q_liq_PI[k] = ($
 $Q_v_PI[k]$
 $- Q_processo_PI[k]$
 $- Q_perdas_PI[k]$
 $)$

Taxa de variacao da temperatura (graus Celsius/s)

$dT_dt = Q_liq_PI[k] / C$

Atualizacao da temperatura

$T_PI[k + 1] = ($
 $T_PI[k]$
 $+ dT_dt * dt$
 $)$

```
# Atualizacao da memoria integral
erro_integral_PI[k + 1] = (
    erro_integral_PI[k]
    + erro_PI[k] * dt_min
)
```

1.19.9 Onde Está a Memória no Algoritmo?

Observe novamente a última operação do ciclo:

```
erro_integral_PI[k + 1] = (
    erro_integral_PI[k]
    + erro_PI[k] * dt_min
)
```

Essa instrução é particularmente importante.

Ela representa diretamente:

$$I_{k+1} = I_k + e_k \Delta t.$$

O programa precisa conservar:

$$I_k$$

para calcular:

$$I_{k+1}.$$

Portanto, a memória integral discutida fisicamente aparece agora de maneira explícita no algoritmo.

Podemos representar essa relação por:

$$\boxed{\text{erro presente} + \text{memória anterior} \rightarrow \text{nova memória}}.$$

A cada repetição do algoritmo, o controlador carrega consigo informações sobre aquilo que aconteceu anteriormente.

1.19.10 Finalização dos Valores no Último Instante

A estrutura de repetição foi executada até:

$$n - 2,$$

porque cada etapa calcula a temperatura do instante seguinte.

Para completar as variáveis do último ponto:

```

# Variaveis do ultimo instante
erro_PI[-1] = T_sp - T_PI[-1]

u_P_PI[-1] = Kp * erro_PI[-1]

u_I_PI[-1] = (
Ki * erro_integral_PI[-1]
)

u_PI[-1] = (
u_P_PI[-1]
+ u_I_PI[-1]
)

u_v_PI[-1] = np.clip(
u0 + u_PI[-1],
0.0,
100.0
)

m_v_PI[-1] = (
Kv * u_v_PI[-1]
)

```

Agora todos os vetores possuem valores correspondentes aos instantes armazenados no vetor de tempo.

1.19.11 Organização dos Resultados PI em um DataFrame

Os resultados podem ser organizados em um novo DataFrame:

```

# DataFrame da simulacao PI
sim_PI = pd.DataFrame({
    "tempo_s": tempo,
    "tempo_min": tempo / 60.0,
    "T_PI": T_PI,
    "erro_PI": erro_PI,
    "erro_integral_PI": erro_integral_PI,
    "u_P_PI": u_P_PI,
    "u_I_PI": u_I_PI,
    "u_PI": u_PI,
    "u_v_PI": u_v_PI,
    "m_v_PI": m_v_PI,
    "m_in": m_in
})

```

Abra:

`sim_PI`

no Explorador de Variáveis.

Observe especialmente as colunas:

`erro_PI` `erro_integral_PI` `u_P_PI` `u_I_PI` `u_PI`.

Agora é possível acompanhar numericamente a formação da ação PI durante a própria dinâmica do processo.

1.20 Comparação Dinâmica entre os Controladores P e PI

Os dois controladores foram submetidos ao mesmo processo.

Também foram mantidos:

$$T_0 = 70^\circ\text{C},$$

o mesmo setpoint:

$$T_{SP} = 80^\circ\text{C},$$

e a mesma perturbação:

$$\dot{m}_{in} : 3,0 \rightarrow 4,0 \text{ kg/min}$$

em:

$$t = 10 \text{ min.}$$

Podemos agora comparar diretamente suas respostas.

1.20.1 Comparação das Temperaturas

No programa:

```
# Comparacao das respostas P e PI
plt.figure()

plt.plot(
    sim_P["tempo_min"],
    sim_P["T_P"],
    label="Controlador P"
```

```

)

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["T_PI"],
    label="Controlador PI"
)

plt.axhline(
    y=T_sp,
    linestyle="--",
    label="Setpoint"
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Comparacao entre os controladores P e PI")
plt.grid()
plt.legend()

plt.show()

```

Esse gráfico deve ser analisado como resultado de uma experiência computacional.

Não observe apenas qual curva está mais próxima do setpoint.

Procure identificar o mecanismo responsável pela diferença entre as respostas.

1.20.2 O que Deve Ser Procurado no Gráfico?

No controlador proporcional, a ação é determinada por:

$$u_P = K_p e.$$

Quando uma nova condição de carga exige uma ação adicional permanente, o controlador P necessita manter um erro diferente de zero para produzir essa ação.

Por isso, após uma perturbação, pode surgir:

$$\boxed{e_{ss} \neq 0}.$$

No controlador PI, por outro lado:

$$u_{PI} = K_p e + K_i I.$$

A parcela integral pode conservar uma ação adicional mesmo quando:

$$e \rightarrow 0.$$

Portanto, a simulação deverá permitir observar a tendência de eliminação do erro em regime permanente.

1.20.3 Comparação dos Erros

A comparação pode ser realizada também diretamente sobre os erros:

```
# Comparacao dos erros de controle
plt.figure()

plt.plot(
    sim_P["tempo_min"],
    sim_P["erro_P"],
    label="Erro - P"
)

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["erro_PI"],
    label="Erro - PI"
)

plt.axhline(
    y=0.0,
    linestyle="--",
    label="Erro zero"
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Erro (graus Celsius)")
plt.title("Erro de controle: P versus PI")
plt.grid()
```

```
plt.legend()
```

```
plt.show()
```

Agora a pergunta de engenharia pode ser formulada diretamente:

Qual controlador consegue levar o erro de regime permanente para mais próximo de zero?

1.20.4 Observando a Memória Integral

Para compreender como o controlador PI realiza essa correção, represente:

```
# Evolucao da memoria integral
plt.figure()

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["erro_integral_PI"]
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Erro integral (graus Celsius.min)")
plt.title("Memoria integral do controlador PI")
plt.grid()
plt.legend()

plt.show()
```

Observe que a memória integral não precisa retornar a zero quando o erro instantâneo desaparece.

Quando:

$$e = 0,$$

temos:

$$\frac{dI}{dt} = 0.$$

Consequentemente, o valor integral pode permanecer constante.

Essa característica é fundamental.

O valor armazenado na memória pode continuar produzindo:

$$u_I \neq 0$$

mesmo quando:

$$e = 0.$$

1.20.5 Observando as Parcelas P e I

Outra análise importante consiste em observar separadamente as contribuições proporcional e integral:

```
# Parcelas proporcional e integral
plt.figure()

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["u_P_PI"],
    label="Parcela P"
)

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["u_I_PI"],
    label="Parcela I"
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Acao de controle (%)")
plt.title("Contribuicoes P e I do controlador PI")
plt.grid()
plt.legend()

plt.show()
```

Esse gráfico permite responder a uma questão que dificilmente seria percebida apenas observando a temperatura:

Quem está sustentando a ação de controle quando o erro se aproxima de zero?

À medida que:

$$e \rightarrow 0,$$

a parcela proporcional tende a:

$$u_P \rightarrow 0.$$

Entretanto, a parcela integral pode permanecer:

$$u_I \neq 0.$$

Assim, a ação necessária para manter a nova condição de operação passa a ser sustentada pela memória integral.

1.20.6 Comparação das Aberturas das Válvulas

Também podemos observar como cada estratégia atua fisicamente sobre o elemento final de controle:

```
# Comparacao das aberturas das valvulas
```

```
plt.figure()
```

```
plt.plot(
    sim_P["tempo_min"],
    sim_P["u_v_P"],
    label="Valvula - P"
)
```

```
plt.plot(
    sim_PI["tempo_min"],
    sim_PI["u_v_PI"],
    label="Valvula - PI"
)
```

```
plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)
```

```
plt.xlabel("Tempo (min)")
plt.ylabel("Abertura da valvula (%)")
```

```
plt.title("Atuacao sobre a valvula: P versus PI")
plt.grid()
plt.legend()

plt.show()
```

Esse gráfico estabelece uma conexão direta entre a matemática do controlador e o equipamento industrial.

O controlador não modifica diretamente a temperatura.

Ele modifica a abertura da válvula.

A válvula modifica a vazão de vapor.

A vazão modifica a transferência de energia.

Somente então a temperatura é alterada.

Portanto:

controlador $\rightarrow$ válvula $\rightarrow$ vazão $\rightarrow$ energia $\rightarrow$ temperatura

1.20.7 A Descoberta do Aluno

Neste ponto, a diferença entre os controladores P e PI não precisa ser apresentada apenas como uma definição teórica.

Ela pode ser descoberta a partir dos resultados da própria simulação.

O controlador P reage ao erro.

O controlador PI também reage ao erro, mas possui adicionalmente uma memória da sua permanência.

Essa memória permite manter uma ação corretiva mesmo quando o erro instantâneo se aproxima de zero.

Assim:

Controlador P $e \neq 0$ pode ser necessário para sustentar a correção

enquanto:

Controlador PI $e \rightarrow 0$ enquanto a parcela integral pode sustentar a correção

A eliminação do offset deixa, portanto, de ser uma regra a ser memorizada. Ela passa a ser uma consequência observável da maneira como cada controlador utiliza o erro.

1.21 Quantificando o Desempenho dos Controladores

A inspeção visual dos gráficos permite identificar diferenças importantes entre os controladores.

Entretanto, uma análise de Engenharia não deve depender exclusivamente da observação visual.

Podemos quantificar algumas dessas diferenças.

Começamos pelo erro existente no último instante da simulação.

No programa:

```
# Erro no ultimo instante da simulacao
erro_final_P = sim_P["erro_P"].iloc[-1]
erro_final_PI = sim_PI["erro_PI"].iloc[-1]

print("Erro final - controlador P:", erro_final_P)
print("Erro final - controlador PI:", erro_final_PI)
```

O comando:

```
.iloc[-1]
```

seleciona o último valor armazenado na coluna.

Agora compare:

$$e_{final,P}$$

e:

$$e_{final,PI}.$$

A comparação numérica complementa aquilo que foi observado graficamente.

1.21.1 O Erro Final é Sempre o Erro de Regime Permanente?

É necessário, entretanto, fazer uma distinção importante.

O último valor de uma simulação somente pode ser interpretado como uma boa aproximação do erro de regime permanente se o sistema já estiver suficientemente próximo de uma condição estacionária.

Portanto:

último valor da simulação $\neq$ necessariamente regime permanente.

Antes de concluir que determinado controlador apresenta ou não offset, é necessário observar se a resposta ainda está variando significativamente no final do intervalo simulado. Essa observação evita conclusões incorretas baseadas apenas no término arbitrário do tempo de simulação.

1.22 Compreendendo Fisicamente a Eliminação do Offset

A simulação permitiu observar uma diferença importante entre os controladores P e PI. Entretanto, ainda podemos fazer uma pergunta mais profunda:

Por que o controlador proporcional precisa manter um erro para operar após uma mudança de carga?

Para responder a essa pergunta, não começaremos pelo controlador.

Começaremos pelo próprio processo.

O trocador de calor necessita receber determinada quantidade de energia para manter sua temperatura constante.

Quando a vazão da corrente de processo aumenta, a demanda térmica também aumenta. Consequentemente, uma nova quantidade de vapor será necessária para manter:

$$T = 80^{\circ}\text{C}.$$

Vamos determinar essa quantidade a partir do balanço de energia.

1.22.1 Condição de Equilíbrio Térmico

O modelo utilizado para o trocador foi:

$$C \frac{dT}{dt} = \dot{Q}_v - \dot{Q}_{\text{processo}} - \dot{Q}_{\text{perdas}}.$$

Em uma condição de equilíbrio térmico, a temperatura permanece constante.

Portanto:

$$\frac{dT}{dt} = 0.$$

Substituindo essa condição no balanço:

$$0 = \dot{Q}_v - \dot{Q}_{\text{processo}} - \dot{Q}_{\text{perdas}}.$$

Logo:

$$\boxed{\dot{Q}_v = \dot{Q}_{\text{processo}} + \dot{Q}_{\text{perdas}}}.$$

Essa equação possui uma interpretação física direta.

Para manter a temperatura constante, a potência fornecida pelo vapor deve compensar exatamente a energia requerida pela corrente de processo e as perdas térmicas para o ambiente.

1.22.2 Condição de Equilíbrio Antes da Perturbação

Considere inicialmente:

$$T = 80\text{ }^{\circ}\text{C}$$

e:

$$\dot{m}_{in} = 3,0\text{ kg/min.}$$

Convertendo a vazão para unidades do SI:

$$\dot{m}_{in} = \frac{3,0}{60} = 0,050\text{ kg/s.}$$

A potência térmica associada à corrente de processo será:

$$\dot{Q}_{processo} = \dot{m}_{in}c_p(T - T_{in}).$$

Utilizando:

$$c_p = 4000\text{ J/(kg }^{\circ}\text{C)}$$

e:

$$T_{in} = 30\text{ }^{\circ}\text{C},$$

temos:

$$\dot{Q}_{processo} = (0,050)(4000)(80 - 30).$$

Portanto:

$$\boxed{\dot{Q}_{processo} = 10\,000\text{ W}}.$$

As perdas térmicas são:

$$\dot{Q}_{perdas} = U_L(T - T_{amb}).$$

Como:

$$U_L = 20\text{ W/}^{\circ}\text{C}$$

e:

$$T_{amb} = 25\text{ }^{\circ}\text{C},$$

temos:

$$\dot{Q}_{perdas} = 20(80 - 25),$$

resultando em:

$$\dot{Q}_{perdas} = 1100 \text{ W}.$$

Portanto, para manter exatamente:

$$T = 80^\circ\text{C},$$

a potência fornecida pelo vapor deverá ser:

$$\dot{Q}_v = 10\,000 + 1100,$$

logo:

$$\dot{Q}_v = 11\,100 \text{ W}.$$

1.22.3 Qual Vazão de Vapor é Necessária?

A potência fornecida pelo vapor foi modelada por:

$$\dot{Q}_v = \dot{m}_v \lambda_v.$$

Portanto:

$$\dot{m}_v = \frac{\dot{Q}_v}{\lambda_v}.$$

Utilizando:

$$\lambda_v = 2,0 \times 10^6 \text{ J/kg},$$

temos:

$$\dot{m}_v = \frac{11\,100}{2,0 \times 10^6}.$$

Assim:

$$\dot{m}_v = 0,00555 \text{ kg/s}.$$

Convertendo para kg/min:

$$\dot{m}_v = (0,00555)(60),$$

resultando em:

$$\dot{m}_v = 0,333 \text{ kg/min}.$$

Portanto, antes da perturbação, o processo necessita aproximadamente:

$$0,333 \text{ kg/min}$$

de vapor para permanecer exatamente em:

$$80^\circ\text{C}.$$

1.22.4 Qual Deve Ser a Abertura da Válvula?

O modelo simplificado da válvula é:

$$\dot{m}_v = K_v u_v.$$

Portanto:

$$u_v = \frac{\dot{m}_v}{K_v}.$$

Utilizando:

$$K_v = 0,01 \frac{\text{kg/min}}{\%},$$

temos:

$$u_v = \frac{0,333}{0,01}.$$

Assim:

$$u_{v,eq} \approx 33,3\%.$$

Esse resultado possui grande importância física.

Para manter o trocador exatamente em:

$$80^\circ\text{C}$$

antes da perturbação, a válvula deveria permanecer aproximadamente:

$$33,3\% \text{ aberta}.$$

1.22.5 Uma Descoberta sobre a Abertura Nominal

Na simulação anterior foi adotada uma abertura nominal:

$$u_0 = 40\%.$$

Entretanto, o balanço de energia mostrou que, para as condições físicas adotadas, a abertura necessária para manter exatamente o setpoint antes da perturbação é:

$$u_{v,eq} \approx 33,3\%.$$

Portanto:

$$u_0 \neq u_{v,eq}.$$

Isso significa que o valor de 40% utilizado anteriormente deve ser interpretado apenas como uma condição nominal ilustrativa e não como a abertura de equilíbrio correspondente ao modelo térmico adotado.

Essa diferença é importante porque uma simulação dinâmica deve utilizar uma condição nominal consistente com o balanço de energia quando se deseja estudar com precisão o comportamento do controlador em torno de um ponto de operação.

A partir deste ponto, para a comparação entre os controladores P e PI, será adotado:

$$u_0 = 33,3\%$$

como aproximação da abertura de equilíbrio antes da perturbação.

1.22.6 O que Acontece quando a Vazão de Processo Aumenta?

Agora considere a perturbação:

$$\dot{m}_{in} : 3,0 \rightarrow 4,0 \text{ kg/min.}$$

Desejamos novamente manter:

$$T = 80^\circ\text{C}.$$

A nova vazão da corrente de processo será:

$$\dot{m}_{in} = \frac{4,0}{60} = 0,06667 \text{ kg/s.}$$

A nova potência térmica requerida pela corrente será:

$$\dot{Q}_{processo} = (0,06667)(4000)(80 - 30).$$

Portanto:

$$\dot{Q}_{processo} \approx 13\,333 \text{ W}.$$

As perdas térmicas permanecem:

$$\dot{Q}_{perdas} = 1100 \text{ W},$$

pois estamos procurando a condição necessária para manter:

$$T = 80^\circ\text{C}.$$

Assim, a nova potência requerida do vapor será:

$$\dot{Q}_v = 13\,333 + 1100,$$

resultando em:

$$\boxed{\dot{Q}_v \approx 14\,433 \text{ W}}.$$

1.22.7 Nova Vazão de Vapor Necessária

Utilizando:

$$\dot{Q}_v = \dot{m}_v \lambda_v,$$

temos:

$$\dot{m}_v = \frac{14\,433}{2,0 \times 10^6}.$$

Portanto:

$$\dot{m}_v \approx 0,007217 \text{ kg/s}.$$

Convertendo para kg/min:

$$\dot{m}_v \approx 0,433 \text{ kg/min}.$$

Logo:

$$\boxed{\dot{m}_{v,eq} \approx 0,433 \text{ kg/min}}.$$

1.22.8 Nova Abertura de Equilíbrio da Válvula

Como:

$$\dot{m}_v = K_v u_v,$$

a abertura necessária será:

$$u_v = \frac{0,433}{0,01}.$$

Portanto:

$$\boxed{u_{v,eq} \approx 43,3\%}.$$

Agora temos duas condições de equilíbrio importantes:

Condição	$\dot{m}_{in}$ (kg/min)	$u_{v,eq}$ (%)
Antes da perturbação	3,0	33,3
Após a perturbação	4,0	43,3

A perturbação exige, portanto, aproximadamente:

$$43,3 - 33,3 = 10,0$$

pontos percentuais adicionais de abertura da válvula.

Assim:

$$\boxed{\Delta u_v \approx 10\%}.$$

Esse resultado será fundamental para compreender o offset do controlador proporcional.

1.23 Por que o Controlador P Produz Offset?

Antes da perturbação, a abertura necessária era aproximadamente:

$$u_0 = 33,3\%.$$

Após a perturbação, o processo passa a exigir:

$$u_{v,eq} = 43,3\%.$$

Portanto, o controlador precisa fornecer aproximadamente:

$$u_P = 10\%$$

de ação adicional.

Para um controlador proporcional:

$$u_P = K_p e.$$

Como:

$$K_p = 2 \frac{\%}{^\circ\text{C}},$$

temos:

$$10 = 2e.$$

Logo:

$$\boxed{e = 5^\circ\text{C}}.$$

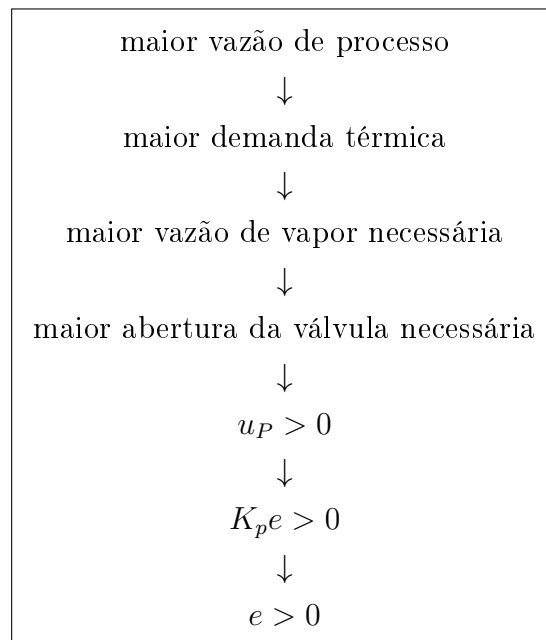
Isso significa que, nesse modelo simplificado, o controlador proporcional precisaria manter aproximadamente um erro de temperatura para produzir a ação adicional exigida pela nova carga.

Essa é a origem física do offset.

Não é o controlador que “deseja” manter a temperatura diferente do setpoint.

O problema é que sua ação adicional depende diretamente da existência de erro.

Podemos representar esse raciocínio por:



Portanto, o offset não deve ser interpretado apenas como uma característica matemática do controlador P.

Ele surge da necessidade física de produzir uma nova ação permanente sobre o processo.

1.23.1 Estimativa Mais Precisa do Offset

O cálculo anterior utilizou a abertura necessária em 80°C para construir uma interpretação física simples.

Entretanto, quando o controlador P apresenta offset, a temperatura de equilíbrio real será inferior a 80°C .

Consequentemente, a demanda térmica e as perdas também serão ligeiramente diferentes. Podemos determinar o valor de equilíbrio de forma mais rigorosa.

Para o controlador P:

$$u_v = u_0 + K_p(T_{SP} - T).$$

A vazão de vapor é:

$$\dot{m}_v = K_v u_v.$$

Na condição de equilíbrio:

$$\dot{Q}_v = \dot{Q}_{processo} + \dot{Q}_{perdas}.$$

Portanto:

$$\frac{K_v [u_0 + K_p(T_{SP} - T)]}{60} \lambda_v = \frac{\dot{m}_{in}}{60} c_p (T - T_{in}) + U_L (T - T_{amb}).$$

Essa equação permite calcular diretamente a temperatura de equilíbrio do sistema com controlador proporcional.

Para a condição após a perturbação:

$$\dot{m}_{in} = 4 \text{ kg/min},$$

e utilizando os parâmetros adotados neste exemplo, obtém-se aproximadamente:

$$T_{eq,P} \approx 76,5^\circ\text{C}.$$

Portanto, o erro de regime permanente é aproximadamente:

$$e_{ss,P} = 80 - 76,5,$$

resultando em:

$$e_{ss,P} \approx 3,5^\circ\text{C}.$$

Esse valor é menor que a estimativa inicial de 5°C porque, quando a temperatura diminui, a própria demanda térmica da corrente de processo e as perdas para o ambiente também diminuem.

Esse resultado mostra uma característica importante da análise de processos:

uma estimativa física simples $\rightarrow$ compreensão $\rightarrow$ modelo completo $\rightarrow$ resultado mais preciso.

1.24 Como o Controlador PI Resolve o Problema

Considere agora o controlador PI:

$$u_{PI} = K_p e + K_i I.$$

Após a perturbação, sabemos que o processo necessita aproximadamente:

$$10\%$$

de abertura adicional para permanecer exatamente no setpoint.

Quando a ação integral consegue fornecer essa correção, podemos ter:

$$e = 0.$$

Consequentemente:

$$u_P = K_p e = 0.$$

Entretanto:

$$u_I = K_i I$$

pode permanecer diferente de zero.

Na condição de equilíbrio desejada:

$$u_I \approx 10\%.$$

Assim:

$$u_v = 33,3 + 10 = 43,3\%.$$

A válvula permanece suficientemente aberta para atender à nova demanda térmica mesmo com:

$$e = 0.$$

Portanto:

$e = 0$ $\Downarrow$ $u_P = 0$ mas $u_I \approx 10\%$ $\Downarrow$ $u_v \approx 43,3\%$
--

Essa é a explicação física para a eliminação do offset pelo controlador PI.

1.24.1 Quanto de Erro Precisa Ser Acumulado?

Foi adotado:

$$K_i = 0,5 \frac{\%}{^{\circ}\text{C min}}.$$

Se a parcela integral necessária em regime permanente é aproximadamente:

$$u_I = 10\%,$$

então:

$$u_I = K_i I.$$

Logo:

$$I = \frac{u_I}{K_i}.$$

Substituindo:

$$I = \frac{10}{0,5},$$

obtemos:

$$\boxed{I \approx 20^{\circ}\text{C min}}.$$

Observe o significado desse resultado.

Quando o erro instantâneo finalmente tende a zero, a memória integral não precisa desaparecer.

Ela pode permanecer aproximadamente em:

$$20^{\circ}\text{C min}.$$

Essa memória produz:

$$u_I = (0,5)(20) = 10\%.$$

E essa ação mantém a válvula na nova abertura necessária.

Portanto:

$$\boxed{\text{erro atual} = 0 \quad \text{mas} \quad \text{memória do erro} \neq 0}.$$

Essa é uma das características fundamentais do controlador PI.

1.24.2 Interpretação Final

Agora podemos compreender a diferença entre os dois controladores sem recorrer apenas aos gráficos da simulação.

O processo sofreu uma perturbação.

A nova condição exige maior fornecimento de energia.

Para fornecer essa energia, a válvula precisa permanecer mais aberta.

O controlador P somente consegue manter essa abertura adicional se existir um erro.

O controlador PI pode armazenar na parcela integral a ação adicional necessária.

Assim:

Controlador P	→	ação adicional sustentada pelo erro
Controlador PI	→	ação adicional sustentada pela memória integral

Essa interpretação conecta diretamente:

fenômeno físico → balanço de energia → condição de equilíbrio → controlador → offset.

A eliminação do offset deixa, assim, de ser apenas uma propriedade declarada do controlador PI. Ela pode ser deduzida a partir das necessidades energéticas do próprio processo.

1.25 Experiência Computacional de Rejeição de Perturbação

A análise da condição de equilíbrio mostrou que, para:

$$T = 80^{\circ}\text{C}$$

e:

$$\dot{m}_{in} = 3,0 \text{ kg/min},$$

a abertura de válvula necessária é aproximadamente:

$$u_{v,eq} = 33,3\%.$$

Esse resultado permite construir uma nova experiência computacional com uma condição inicial fisicamente consistente.

Diferentemente da simulação anterior, na qual o processo iniciou em:

$$T_0 = 70^{\circ}\text{C},$$

agora será considerado um processo que já se encontra estabilizado no setpoint:

$$T_0 = T_{SP} = 80^\circ\text{C}.$$

A abertura nominal será:

$$u_0 = 33,3\%.$$

Assim, antes da perturbação:

$$e = 0,$$

e nenhuma ação corretiva adicional é necessária.

O objetivo desta nova simulação será responder à seguinte pergunta:

Como os controladores P e PI reagem quando um processo inicialmente estabilizado sofre uma perturbação?

Essa análise é denominada **rejeição de perturbação**.

1.25.1 Separando Dois Problemas de Controle

As duas experiências computacionais desenvolvidas neste capítulo possuem objetivos diferentes.

Na primeira, foi adotado:

$$T_0 = 70^\circ\text{C}$$

para observar como o controlador conduz o processo em direção ao setpoint.

Essa situação permite estudar uma resposta de partida.

Na segunda experiência, será adotado:

$$T_0 = T_{SP} = 80^\circ\text{C}.$$

O processo permanecerá inicialmente em equilíbrio e, posteriormente, será submetido a uma perturbação.

Portanto:

$T_0 \neq T_{SP}$	$\longrightarrow$	resposta de partida
$T_0 = T_{SP}$	$\longrightarrow$	rejeição de perturbação

Essa separação é importante porque permite investigar diferentes aspectos do desempenho de uma malha de controle.

1.25.2 Condição Inicial da Nova Experiência

No instante inicial:

$$T_0 = 80^\circ\text{C}.$$

Portanto:

$$e_0 = T_{SP} - T_0,$$

logo:

$$e_0 = 80 - 80 = 0.$$

Para o controlador proporcional:

$$u_{P,0} = K_p e_0 = 0.$$

Como a abertura nominal foi definida a partir do balanço de energia:

$$u_0 = 33,3\%,$$

temos:

$$u_{v,0} = u_0 + u_{P,0}.$$

Portanto:

$$\boxed{u_{v,0} = 33,3\%}.$$

Essa abertura fornece aproximadamente a quantidade de vapor necessária para manter o processo em equilíbrio térmico.

Assim, antes da perturbação:

$$\boxed{T_{PV} = T_{SP}, \quad e = 0, \quad u_P = 0, \quad u_v \approx 33,3\%}.$$

Essa será a condição de referência para a experiência.

1.25.3 Aplicação da Perturbação

Durante os primeiros:

$$10 \text{ min},$$

a vazão da corrente de processo será mantida em:

$$\dot{m}_{in} = 3,0 \text{ kg/min.}$$

No instante:

$$t = 10 \text{ min,}$$

será aplicada uma perturbação:

$$\boxed{\dot{m}_{in} : 3,0 \rightarrow 4,0 \text{ kg/min.}}$$

O setpoint permanecerá:

$$T_{SP} = 80^\circ\text{C.}$$

Portanto, nenhuma mudança será solicitada ao valor desejado da temperatura.

A única modificação será realizada na carga térmica do processo.

Essa condição permitirá observar a capacidade de cada controlador de rejeitar a perturbação.

1.25.4 O que Esperamos Fisicamente após a Perturbação?

Imediatamente após o aumento de:

$$\dot{m}_{in},$$

uma quantidade maior de fluido passa pelo trocador de calor.

Como a corrente entra a:

$$T_{in} = 30^\circ\text{C,}$$

a demanda térmica aumenta.

Inicialmente, entretanto, a abertura da válvula ainda está próxima de:

$$33,3\%.$$

Consequentemente, o vapor fornecido deixa de ser suficiente para manter o novo balanço energético.

Assim:

$$\dot{Q}_{processo} + \dot{Q}_{perdas} > \dot{Q}_v.$$

Portanto:

$$\dot{Q}_{liq} < 0,$$

e:

$$\frac{dT}{dt} < 0.$$

A temperatura começa a diminuir.

Quando:

$$T_{PV} < T_{SP},$$

surge um erro positivo:

$$e = T_{SP} - T_{PV} > 0.$$

Somente então o controlador começa a aumentar a abertura da válvula.

A sequência física é:

$$\boxed{\dot{m}_{in} \uparrow \rightarrow \text{demanda térmica} \uparrow \rightarrow \dot{Q}_{liq} < 0 \rightarrow T_{PV} \downarrow \rightarrow e \uparrow \rightarrow \text{ação do controlador} \uparrow}.$$

Observe que a perturbação ocorre primeiro no processo.

O controlador somente reage depois que seus efeitos aparecem na variável medida.

Essa característica é típica de uma estratégia de controle por realimentação.

1.26 Rejeição de Perturbação com Controlador P

Inicialmente será investigado o controlador proporcional.

Os parâmetros serão:

$$T_{SP} = 80^\circ\text{C},$$

$$K_p = 2 \frac{\%}{^\circ\text{C}},$$

e:

$$u_0 = 33,3\%.$$

O tempo total de simulação será:

$$30 \text{ min.}$$

A escolha de um intervalo maior que o utilizado anteriormente permitirá observar com mais clareza a aproximação do sistema a uma nova condição de equilíbrio após a perturbação.

1.26.1 Definição das Condições da Simulação

No programa:

```
# Setpoint (graus Celsius)
T_sp = 80.0

# Ganho proporcional (%/grau Celsius)
Kp = 2.0

# Abertura de equilibrio antes da perturbacao (%)
u0 = 33.3

# Passo de integracao (s)
dt = 1.0

# Tempo total de simulacao: 30 min
tempo_total = 30 * 60

# Vetor de tempo
tempo = np.arange(
    0.0,
    tempo_total + dt,
    dt
)

# Numero de pontos
n = len(tempo)
```

A condição inicial será:

```
# Vetor de temperatura
T_P = np.zeros(n)

# Processo inicialmente no setpoint
T_P[0] = 80.0
```

1.26.2 Construção da Perturbação

A vazão da corrente de processo será:

```
# Vazao da corrente de processo (kg/min)
m_in = np.zeros(n)

# Antes da perturbacao
```

```
m_in[tempo < 600.0] = 3.0
```

```
# Depois da perturbacao
```

```
m_in[tempo >= 600.0] = 4.0
```

Como:

$$600 \text{ s} = 10 \text{ min},$$

a perturbação ocorrerá exatamente no instante planejado.

1.26.3 Variáveis que Serão Armazenadas

Serão armazenadas as principais variáveis necessárias para interpretar o comportamento da malha:

```
# Vetores para armazenamento dos resultados
```

```
erro_P = np.zeros(n)
```

```
u_P = np.zeros(n)
```

```
u_v_P = np.zeros(n)
```

```
m_v_P = np.zeros(n)
```

```
Q_v_P = np.zeros(n)
```

```
Q_processo_P = np.zeros(n)
```

```
Q_perdas_P = np.zeros(n)
```

```
Q_liq_P = np.zeros(n)
```

A estrutura da simulação continuará sendo:

$$T \rightarrow e \rightarrow u_P \rightarrow u_v \rightarrow \dot{m}_v \rightarrow \dot{Q}_v \rightarrow \dot{Q}_{liq} \rightarrow T.$$

1.26.4 Simulação do Controlador P

Utilizando os mesmos parâmetros físicos desenvolvidos anteriormente:

```
# Parametros fisicos
```

```
Kv = 0.01
```

```
lambda_v = 2.0e6
```

```
cp = 4000.0
```

```
T_in = 30.0
```

```
T_amb = 25.0
```

```
U_L = 20.0
```

```
C = 1.0e5
```

a simulação pode ser realizada por:

CAPÍTULO 1. CONTROLE DE TEMPERATURA EM TROCADORES DE CALOR COM COMP

```
# Simulacao com controlador P
for k in range(n - 1):

    # Erro
    erro_P[k] = T_sp - T_P[k]

    # Acao proporcional
    u_P[k] = Kp * erro_P[k]

    # Abertura da valvula
    u_v_P[k] = np.clip(
        u0 + u_P[k],
        0.0,
        100.0
    )

    # Vazao de vapor (kg/min)
    m_v_P[k] = Kv * u_v_P[k]

    # Vazoes em kg/s
    m_v_kg_s = m_v_P[k] / 60.0
    m_in_kg_s = m_in[k] / 60.0

    # Potencia fornecida pelo vapor
    Q_v_P[k] = (
        m_v_kg_s * lambda_v
    )

    # Potencia requerida pela corrente
    Q_processo_P[k] = (
        m_in_kg_s
        * cp
        * (T_P[k] - T_in)
    )

    # Perdas termicas
    Q_perdas_P[k] = (
        U_L
        * (T_P[k] - T_amb)
    )

    # Potencia liquida
    Q_liq_P[k] = (
        Q_v_P[k]
```

```

- Q_processo_P[k]
- Q_perdas_P[k]
)

# Derivada da temperatura
dT_dt = Q_liq_P[k] / C

# Metodo de Euler
T_P[k + 1] = (
T_P[k]
+ dT_dt * dt
)

```

Ao final:

```

# Valores correspondentes ao ultimo instante
erro_P[-1] = T_sp - T_P[-1]

u_P[-1] = Kp * erro_P[-1]

u_v_P[-1] = np.clip(
u0 + u_P[-1],
0.0,
100.0
)

m_v_P[-1] = Kv * u_v_P[-1]

```

1.26.5 Organização dos Resultados

Os resultados serão novamente organizados em um DataFrame:

```

sim_P = pd.DataFrame({
    "tempo_s": tempo,
    "tempo_min": tempo / 60.0,
    "T_P": T_P,
    "erro_P": erro_P,
    "u_P": u_P,
    "u_v_P": u_v_P,
    "m_v_P": m_v_P,
    "m_in": m_in
})

```

Abra:

sim_P

no Explorador de Variáveis.

Antes de construir qualquer gráfico, observe os dados próximos de:

$$t = 10 \text{ min.}$$

Procure identificar numericamente a sequência:

$$\dot{m}_{in} \uparrow \rightarrow T_P \downarrow \rightarrow e_P \uparrow \rightarrow u_P \uparrow \rightarrow u_v \uparrow.$$

Essa observação permite verificar diretamente no DataFrame a propagação da perturbação através da malha de controle.

1.26.6 Observando o Instante da Perturbação no DataFrame

Podemos selecionar somente os dados próximos do instante da perturbação.

Por exemplo:

```
# Dados entre 9,9 e 10,2 minutos
janela_P = sim_P.loc[
(sim_P["tempo_min"] >= 9.9)
& (sim_P["tempo_min"] <= 10.2)
]

print(janela_P)
```

Abra também:

janela_P

no Explorador de Variáveis.

Agora procure responder:

Qual variável muda primeiro?

A vazão de processo:

$$\dot{m}_{in}$$

é a variável na qual a perturbação foi aplicada.

A temperatura não muda instantaneamente de valor.

Ela começa a apresentar uma tendência de queda porque o balanço de energia foi alterado.

Em seguida, o erro aumenta e o controlador modifica a abertura da válvula.

Portanto, os dados permitem reconstruir a sequência causal da resposta do processo.

1.26.7 Resposta do Controlador P à Perturbação

Depois de observar numericamente os dados, podemos construir o gráfico da temperatura.

```
plt.figure()

plt.plot(
    sim_P["tempo_min"],
    sim_P["T_P"],
    label="Temperatura - P"
)

plt.axhline(
    y=T_sp,
    linestyle="--",
    label="Setpoint"
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Rejeicao de perturbacao com controlador P")
plt.grid()
plt.legend()

plt.show()
```

Antes de:

$$t = 10 \text{ min},$$

a temperatura permanece próxima do setpoint.

Após a perturbação, a temperatura diminui.

O controlador reage aumentando a abertura da válvula e o sistema caminha para uma nova condição de equilíbrio.

Entretanto, essa nova condição não coincide exatamente com:

$$T_{SP} = 80^\circ\text{C}.$$

Permanece um erro de regime permanente.

Esse é o offset do controlador proporcional.

1.26.8 Comparando a Previsão Matemática com a Simulação

Anteriormente, a condição de equilíbrio do modelo com controlador proporcional foi determinada matematicamente.

Para:

$$\dot{m}_{in} = 4 \text{ kg/min},$$

obteve-se aproximadamente:

$$T_{eq,P} \approx 76,5^\circ\text{C}.$$

Portanto, o erro previsto em regime permanente é:

$$e_{ss,P} = 80 - 76,5,$$

ou aproximadamente:

$$e_{ss,P} \approx 3,5^\circ\text{C}.$$

Agora podemos verificar se a simulação dinâmica caminha para esse mesmo resultado.

No programa:

```
# Valores finais da simulacao P
T_final_P = sim_P["T_P"].iloc[-1]
erro_final_P = sim_P["erro_P"].iloc[-1]

print("Temperatura final - P:", T_final_P)
print("Erro final - P:", erro_final_P)
```

A comparação entre o cálculo analítico e a simulação constitui uma etapa importante de verificação do modelo.

Se o tempo de simulação for suficientemente longo para que o processo se aproxime do equilíbrio, os resultados deverão se aproximar da condição calculada matematicamente.

Portanto:

$\text{equação de equilíbrio} \longleftrightarrow \text{simulação dinâmica}$
--

Quando duas abordagens independentes conduzem ao mesmo comportamento, aumenta nossa confiança na implementação computacional.

1.27 Rejeição de Perturbação com Controlador PI

A mesma experiência será agora repetida utilizando um controlador PI.

Nenhuma característica física do processo será alterada.

Serão mantidos:

$$T_0 = 80^\circ\text{C},$$

$$T_{SP} = 80^\circ\text{C},$$

$$u_0 = 33,3\%,$$

e a mesma perturbação:

$$\dot{m}_{in} : 3,0 \rightarrow 4,0 \text{ kg/min}$$

em:

$$t = 10 \text{ min.}$$

A única diferença será a estratégia utilizada pelo controlador.

Será adotado:

$$K_p = 2 \frac{\%}{^\circ\text{C}}$$

e:

$$K_i = 0,5 \frac{\%}{^\circ\text{C min}}.$$

1.27.1 Criação das Variáveis PI

No programa:

```
# Ganho integral (%/(grau Celsius.min))
```

```
Ki = 0.5
```

```
# Vetores da simulacao PI
```

```
T_PI = np.zeros(n)
```

```
erro_PI = np.zeros(n)
```

```
erro_integral_PI = np.zeros(n)
```

```
u_P_PI = np.zeros(n)
```

```
u_I_PI = np.zeros(n)
```

```
u_PI = np.zeros(n)
```

```
u_v_PI = np.zeros(n)
```

```
m_v_PI = np.zeros(n)
```

```

Q_v_PI = np.zeros(n)
Q_processo_PI = np.zeros(n)
Q_perdas_PI = np.zeros(n)
Q_liq_PI = np.zeros(n)

# Condicao inicial
T_PI[0] = 80.0

# Memoria integral inicialmente nula
erro_integral_PI[0] = 0.0

# Passo de integracao expresso em minutos
dt_min = dt / 60.0

```

Antes da perturbação:

$$e = 0.$$

Portanto, a memória integral não cresce.

Assim:

$$I = 0$$

e:

$$u_I = 0.$$

A válvula permanece aproximadamente em sua abertura nominal:

$$u_v \approx 33,3\%.$$

1.27.2 Simulação do Controlador PI

A simulação será:

```

# Simulacao com controlador PI
for k in range(n - 1):

# Erro
erro_PI[k] = T_sp - T_PI[k]

# Acao proporcional
u_P_PI[k] = Kp * erro_PI[k]

```

```
# Acao integral
u_I_PI[k] = (
Ki * erro_integral_PI[k]
)

# Acao total PI
u_PI[k] = (
u_P_PI[k]
+ u_I_PI[k]
)

# Abertura da valvula
u_v_PI[k] = np.clip(
u0 + u_PI[k],
0.0,
100.0
)

# Vazao de vapor
m_v_PI[k] = (
Kv * u_v_PI[k]
)

# Conversoes para kg/s
m_v_kg_s = m_v_PI[k] / 60.0
m_in_kg_s = m_in[k] / 60.0

# Potencia do vapor
Q_v_PI[k] = (
m_v_kg_s * lambda_v
)

# Potencia requerida pelo processo
Q_processo_PI[k] = (
m_in_kg_s
* cp
* (T_PI[k] - T_in)
)

# Perdas termicas
Q_perdas_PI[k] = (
U_L
* (T_PI[k] - T_amb)
)
```

```

# Potencia liquida
Q_liq_PI[k] = (
    Q_v_PI[k]
    - Q_processo_PI[k]
    - Q_perdas_PI[k]
)

# Derivada da temperatura
dT_dt = (
    Q_liq_PI[k] / C
)

# Atualizacao da temperatura
T_PI[k + 1] = (
    T_PI[k]
    + dT_dt * dt
)

# Atualizacao da memoria integral
erro_integral_PI[k + 1] = (
    erro_integral_PI[k]
    + erro_PI[k] * dt_min
)

```

Observe novamente que a diferença fundamental em relação ao controlador P aparece em:

```

erro_integral_PI[k + 1] = (
    erro_integral_PI[k]
    + erro_PI[k] * dt_min
)

```

Essa instrução permite ao controlador carregar informações dos instantes anteriores.

1.27.3 Finalização e Organização dos Resultados

Complete as variáveis correspondentes ao último instante:

```

erro_PI[-1] = T_sp - T_PI[-1]

u_P_PI[-1] = Kp * erro_PI[-1]

u_I_PI[-1] = (
    Ki * erro_integral_PI[-1]

```

```

)

u_PI[-1] = (
    u_P_PI[-1]
    + u_I_PI[-1]
)

u_v_PI[-1] = np.clip(
    u0 + u_PI[-1],
    0.0,
    100.0
)

m_v_PI[-1] = (
    Kv * u_v_PI[-1]
)

```

Agora organize os resultados:

```

sim_PI = pd.DataFrame({
    "tempo_s": tempo,
    "tempo_min": tempo / 60.0,
    "T_PI": T_PI,
    "erro_PI": erro_PI,
    "erro_integral_PI": erro_integral_PI,
    "u_P_PI": u_P_PI,
    "u_I_PI": u_I_PI,
    "u_PI": u_PI,
    "u_v_PI": u_v_PI,
    "m_v_PI": m_v_PI,
    "m_in": m_in
})

```

Abra:

`sim_PI`

no Explorador de Variáveis.

Observe inicialmente os dados próximos da perturbação.

Depois, observe as últimas linhas da tabela.

Pergunte:

O que aconteceu com o erro?

Em seguida:

O que aconteceu com a parcela proporcional?

E finalmente:

O que aconteceu com a parcela integral?

1.28 A Experiência P versus PI

Os dois controladores foram submetidos exatamente à mesma perturbação.

Agora suas temperaturas podem ser representadas simultaneamente.

```
plt.figure()

plt.plot(
    sim_P["tempo_min"],
    sim_P["T_P"],
    label="Controlador P"
)

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["T_PI"],
    label="Controlador PI"
)

plt.axhline(
    y=T_sp,
    linestyle="--",
    label="Setpoint"
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Rejeicao de perturbacao: P versus PI")
plt.grid()
plt.legend()

plt.show()
```

Observe inicialmente o intervalo:

$$t < 10 \text{ min.}$$

Os dois processos encontram-se próximos da mesma condição de equilíbrio.

No instante:

$$t = 10 \text{ min,}$$

a mesma perturbação é aplicada aos dois sistemas.

A partir desse ponto, as respostas começam a se diferenciar.

O controlador P aumenta sua ação à medida que a temperatura se afasta do setpoint.

Entretanto, para sustentar a ação adicional exigida pela nova carga, precisa manter um erro diferente de zero.

O controlador PI também reage inicialmente por meio da ação proporcional.

Entretanto, enquanto o erro permanece presente, a ação integral começa a se acumular.

Com o passar do tempo, essa parcela assume progressivamente a responsabilidade de sustentar a nova abertura da válvula.

Consequentemente, a temperatura pode retornar em direção ao setpoint mesmo que a nova condição de carga permaneça.

1.28.1 Observando a Transferência da Ação P para a Ação I

Uma das características mais importantes do controlador PI pode ser observada representando separadamente suas duas contribuições:

```
plt.figure()

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["u_P_PI"],
    label="Parcela proporcional"
)

plt.plot(
    sim_PI["tempo_min"],
    sim_PI["u_I_PI"],
    label="Parcela integral"
)

plt.axvline(
    x=10.0,
    linestyle="--",
    label="Perturbacao"
```

)

```
plt.xlabel("Tempo (min)")
plt.ylabel("Acao de controle (%)")
plt.title("Distribuicao da acao no controlador PI")
plt.grid()
plt.legend()

plt.show()
```

Imediatamente após a perturbação, o erro começa a aumentar.

Consequentemente, a parcela proporcional:

$$u_P = K_p e$$

também aumenta.

Enquanto o erro permanece positivo, a memória integral cresce e:

$$u_I$$

aumenta progressivamente.

À medida que a temperatura retorna ao setpoint:

$$e \rightarrow 0,$$

e, portanto:

$$u_P \rightarrow 0.$$

Entretanto, a parcela integral permanece próxima do valor necessário para sustentar a nova condição de operação.

Assim, ocorre uma mudança progressiva na origem da ação corretiva:

logo após a perturbação	→	forte participação de u_P
próximo ao novo equilíbrio	→	ação sustentada por u_I

Essa interpretação permite compreender fisicamente o funcionamento do controlador PI.

1.28.2 A Simulação Confirma a Previsão Física?

Antes da simulação, o balanço de energia permitiu prever que, após a perturbação, a válvula deveria operar aproximadamente em:

$$u_v = 43,3\%$$

para manter:

$$T = 80^{\circ}\text{C}.$$

Como:

$$u_0 = 33,3\%,$$

esperamos que, em regime permanente, a parcela integral forneça aproximadamente:

$$u_I = 10\%.$$

Podemos verificar os valores finais calculados:

```
print(
    "Temperatura final:",
    sim_PI["T_PI"].iloc[-1]
)

print(
    "Erro final:",
    sim_PI["erro_PI"].iloc[-1]
)

print(
    "Parcela P final:",
    sim_PI["u_P_PI"].iloc[-1]
)

print(
    "Parcela I final:",
    sim_PI["u_I_PI"].iloc[-1]
)

print(
    "Abertura final da valvula:",
    sim_PI["u_v_PI"].iloc[-1]
)
```

Se o sistema estiver suficientemente próximo do regime permanente, espera-se observar aproximadamente:

$$T_{PI} \rightarrow 80^{\circ}\text{C},$$

$$e_{PI} \rightarrow 0,$$

$$u_P \rightarrow 0,$$

$$u_I \rightarrow 10\%,$$

e:

$$u_v \rightarrow 43,3\%.$$

Essa comparação é particularmente importante porque conecta três formas de analisar o mesmo problema:

previsão física $\longleftrightarrow$ cálculo matemático $\longleftrightarrow$ simulação computacional
--

A computação científica não substitui o conhecimento do processo.

Ela permite testar, visualizar e aprofundar previsões construídas a partir dos princípios de Engenharia.

1.29 Influência do Ganho Proporcional no Comportamento do Processo

Até este momento foi utilizado:

$$K_p = 2 \frac{\%}{^{\circ}\text{C}}.$$

Esse valor permitiu compreender o funcionamento do controlador proporcional e construir a comparação com o controlador PI.

Entretanto, uma pergunta importante ainda permanece:

Por que utilizar exatamente esse valor de ganho?

Nada impede, matematicamente, que sejam utilizados outros valores de:

$$K_p.$$

Por exemplo:

$$K_p = 0,5, \quad K_p = 2,0, \quad K_p = 5,0$$

com unidades de:

$$\frac{\%}{^{\circ}\text{C}}.$$

Mas esses diferentes valores produzirão a mesma resposta?

Essa pergunta será investigada computacionalmente.

1.29.1 Interpretação Física do Ganho Proporcional

O controlador proporcional é descrito por:

$$u_P = K_p e.$$

Portanto, K_p estabelece quanto a saída do controlador será modificada para cada unidade de erro.

Considere, por exemplo:

$$e = 2^\circ\text{C}.$$

Se:

$$K_p = 0,5 \frac{\%}{^\circ\text{C}},$$

então:

$$u_P = (0,5)(2) = 1\%.$$

Para:

$$K_p = 2 \frac{\%}{^\circ\text{C}},$$

temos:

$$u_P = (2)(2) = 4\%.$$

E, para:

$$K_p = 5 \frac{\%}{^\circ\text{C}},$$

resulta:

$$u_P = (5)(2) = 10\%.$$

Assim, para o mesmo erro:

$K_p \uparrow \Rightarrow \text{maior ação corretiva}.$

Fisicamente, no trocador de calor, uma ação corretiva maior significa uma modificação mais intensa da abertura da válvula de vapor.

Portanto:

$$K_p$$

pode ser interpretado como uma medida da intensidade com que o controlador reage ao erro.

1.29.2 Uma Primeira Previsão antes da Simulação

Antes de executar qualquer programa, podemos construir algumas expectativas.

Para um valor pequeno de K_p , espera-se uma reação menos intensa do controlador.

A válvula será modificada mais lentamente em relação ao erro e o processo poderá apresentar maior desvio em relação ao setpoint.

Para valores maiores de K_p , a reação será mais intensa.

Isso poderá aproximar mais rapidamente a temperatura do valor desejado.

Entretanto, aumentar continuamente K_p não significa necessariamente obter uma resposta cada vez melhor.

Uma ação excessivamente intensa pode produzir respostas agressivas, oscilações ou comportamento indesejado, dependendo da dinâmica do processo.

Assim, surge uma primeira ideia importante:

$$\boxed{\text{ganho maior} \neq \text{necessariamente melhor controle}}.$$

A simulação permitirá investigar essa questão.

1.30 Experiência Computacional: Diferentes Valores de K_p

Será utilizada novamente a experiência de rejeição de perturbação.

O processo inicia em:

$$T_0 = 80^\circ\text{C},$$

com:

$$\dot{m}_{in} = 3 \text{ kg/min},$$

e, em:

$$t = 10 \text{ min},$$

ocorre:

$$\dot{m}_{in} : 3 \rightarrow 4 \text{ kg/min}.$$

Todos os parâmetros físicos permanecerão inalterados.

A única variável modificada será:

$$K_p.$$

Serão comparados inicialmente:

$$K_p = 0,5, \quad K_p = 2,0, \quad K_p = 5,0.$$

1.30.1 Por que Alterar Apenas uma Variável?

Quando se deseja investigar a influência de determinado parâmetro, é conveniente modificar apenas esse parâmetro e manter os demais constantes.

Essa estratégia permite relacionar as diferenças observadas diretamente à variável que foi alterada.

Portanto:

mesmo processo + mesma perturbação + diferente K_p

constitui uma experiência computacional controlada.

Esse procedimento é semelhante ao utilizado em experimentos físicos: modifica-se uma variável e observa-se sua influência sobre o sistema.

1.30.2 Transformando a Simulação em uma Função

Nas simulações anteriores, o algoritmo foi escrito passo a passo.

Essa abordagem foi importante para compreender a sequência física e matemática dos cálculos.

Agora, entretanto, desejamos repetir exatamente a mesma simulação para diferentes valores de:

$$K_p.$$

Copiar várias vezes o mesmo programa não seria uma solução eficiente.

Podemos então organizar a simulação em uma função.

No programa:

```
def simular_controlador_P(Kp):

    # Vetores
    T = np.zeros(n)
    erro = np.zeros(n)
    u = np.zeros(n)
    u_v = np.zeros(n)
    m_v = np.zeros(n)
```

CAPÍTULO 1. CONTROLE DE TEMPERATURA EM TROCADORES DE CALOR COM COMP

```
# Condição inicial
T[0] = 80.0

# Simulação
for k in range(n - 1):

    # Erro
    erro[k] = T_sp - T[k]

    # Ação proporcional
    u[k] = Kp * erro[k]

    # Abertura da válvula
    u_v[k] = np.clip(
        u0 + u[k],
        0.0,
        100.0
    )

    # Vazão de vapor
    m_v[k] = Kv * u_v[k]

    # Conversões de vazão
    m_v_kg_s = m_v[k] / 60.0
    m_in_kg_s = m_in[k] / 60.0

    # Potência fornecida pelo vapor
    Q_v = (
        m_v_kg_s * lambda_v
    )

    # Potência requerida pelo processo
    Q_processo = (
        m_in_kg_s
        * cp
        * (T[k] - T_in)
    )

    # Perdas térmicas
    Q_perdas = (
        U_L
        * (T[k] - T_amb)
    )
```

```

# Potencia liquida
Q_liq = (
    Q_v
    - Q_processo
    - Q_perdas
)

# Modelo dinamico
dT_dt = Q_liq / C

# Metodo de Euler
T[k + 1] = (
    T[k]
    + dT_dt * dt
)

# Ultimo ponto
erro[-1] = T_sp - T[-1]

u[-1] = Kp * erro[-1]

u_v[-1] = np.clip(
    u0 + u[-1],
    0.0,
    100.0
)

m_v[-1] = Kv * u_v[-1]

# DataFrame
resultado = pd.DataFrame({
    "tempo_s": tempo,
    "tempo_min": tempo / 60.0,
    "T": T,
    "erro": erro,
    "u": u,
    "u_v": u_v,
    "m_v": m_v,
    "m_in": m_in
})

return resultado

```

A função recebe como entrada:

$$K_p$$

e retorna um DataFrame contendo a resposta dinâmica correspondente.

Assim, a estrutura física do modelo permanece a mesma.

Apenas o ganho do controlador é alterado.

1.30.3 Execução das Três Experiências

Agora podemos executar a mesma experiência para três ganhos diferentes:

```
sim_Kp_05 = simular_controlador_P(0.5)
```

```
sim_Kp_20 = simular_controlador_P(2.0)
```

```
sim_Kp_50 = simular_controlador_P(5.0)
```

Cada variável contém um DataFrame completo da respectiva simulação.

No Explorador de Variáveis do Spyder, podem ser observados:

```
sim_Kp_05
```

```
sim_Kp_20
```

e:

```
sim_Kp_50.
```

As três tabelas possuem a mesma estrutura.

A diferença está apenas no comportamento das variáveis calculadas.

1.30.4 Comparação das Temperaturas

As três respostas podem ser representadas no mesmo gráfico:

```
plt.figure()
```

```
plt.plot(
    sim_Kp_05["tempo_min"],
    sim_Kp_05["T"],
    label="Kp = 0.5"
)
```

```
plt.plot(
    sim_Kp_20["tempo_min"],
    sim_Kp_20["T"],
```

```

label="Kp = 2.0"
)

plt.plot(
sim_Kp_50["tempo_min"],
sim_Kp_50["T"],
label="Kp = 5.0"
)

plt.axhline(
y=T_sp,
linestyle="--",
label="Setpoint"
)

plt.axvline(
x=10.0,
linestyle="--",
label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Influencia do ganho proporcional")
plt.grid()
plt.legend()

plt.show()

```

Agora não devemos apenas observar qual curva parece “melhor”.

Devemos procurar relações entre K_p e o comportamento físico da malha.

1.30.5 Interpretação dos Resultados

Quando:

$$K_p$$

é pequeno, um determinado erro produz uma pequena modificação na abertura da válvula.

Portanto, o controlador apresenta uma ação menos intensa.

Quando:

$$K_p$$

aumenta, a mesma diferença entre:

$$T_{SP}$$

e:

$$T_{PV}$$

produz uma ação corretiva maior.

Assim:

$$\boxed{K_p \uparrow \rightarrow \text{maior sensibilidade ao erro}}.$$

Em um controlador exclusivamente proporcional, o aumento de K_p também tende a reduzir o erro necessário para produzir determinada ação corretiva.

Isso pode ser compreendido diretamente de:

$$u_P = K_p e.$$

Reorganizando:

$$e = \frac{u_P}{K_p}.$$

Para uma determinada ação requerida pelo processo:

$$\boxed{K_p \uparrow \rightarrow e \downarrow}.$$

Portanto, aumentar o ganho proporcional pode reduzir o offset.

Entretanto, isso não significa necessariamente eliminá-lo.

Enquanto uma ação proporcional permanente for necessária:

$$u_P \neq 0,$$

será necessário:

$$e \neq 0$$

para qualquer valor finito de:

$$K_p.$$

1.30.6 Quantificando o Offset para Diferentes Valores de K_p

Os valores finais podem ser comparados diretamente.

No programa:

```

erro_final_05 = (
sim_Kp_05["erro"].iloc[-1]
)

erro_final_20 = (
sim_Kp_20["erro"].iloc[-1]
)

erro_final_50 = (
sim_Kp_50["erro"].iloc[-1]
)

print("Erro final para Kp = 0.5:",
erro_final_05)

print("Erro final para Kp = 2.0:",
erro_final_20)

print("Erro final para Kp = 5.0:",
erro_final_50)

```

Essa comparação transforma uma impressão visual em uma informação quantitativa. Pode-se então construir a hipótese:

$$K_p \uparrow \rightarrow |e_{ss}| \downarrow$$

para o sistema analisado.

Entretanto, mesmo com o aumento de K_p , o controlador proporcional continua necessitando de erro para sustentar a correção exigida pela nova carga.

1.30.7 Construção de uma Tabela Comparativa

Os resultados também podem ser organizados em uma única tabela:

```

comparacao_Kp = pd.DataFrame({
    "Kp": [0.5, 2.0, 5.0],

    "T_final": [
sim_Kp_05["T"].iloc[-1],
sim_Kp_20["T"].iloc[-1],
sim_Kp_50["T"].iloc[-1]
],

    "erro_final": [
sim_Kp_05["erro"].iloc[-1],

```

```

sim_Kp_20["erro"].iloc[-1],
sim_Kp_50["erro"].iloc[-1]
],

"u_v_final": [
sim_Kp_05["u_v"].iloc[-1],
sim_Kp_20["u_v"].iloc[-1],
sim_Kp_50["u_v"].iloc[-1]
]
})

```

```
print(comparacao_Kp)
```

Abra:

comparacao_Kp

no Explorador de Variáveis.

Agora cada linha representa uma experiência computacional.

Essa organização permite comparar diretamente como a escolha do ganho modifica o desempenho da malha.

1.31 O Surgimento do Problema de Sintonia

As experiências anteriores mostraram que diferentes valores de:

$$K_p$$

produzem diferentes comportamentos dinâmicos.

Portanto, a escolha do ganho do controlador não deve ser arbitrária.

Surge então um novo problema de Engenharia:

Como escolher os parâmetros do controlador de forma que o processo apresente um comportamento satisfatório?

Essa escolha é denominada:

sintonia do controlador

ou:

ajuste do controlador

.

Sintonizar um controlador significa selecionar seus parâmetros de modo a obter um desempenho considerado adequado para o processo.

Entretanto, a expressão “desempenho adequado” exige uma discussão adicional. Uma resposta pode ser considerada satisfatória porque:

- retorna rapidamente ao setpoint;
- apresenta pequeno desvio máximo;
- possui pequeno erro de regime permanente;
- evita oscilações excessivas;
- não exige movimentos excessivamente intensos do elemento final de controle.

Portanto, a sintonia envolve compromissos.

Não existe, em geral, um único parâmetro que seja melhor em todos os aspectos.

1.32 Influência do Ganho Integral

No controlador PI:

$$u_{PI} = K_p e + K_i I.$$

Já foi observado que a parcela integral permite manter uma ação corretiva mesmo quando:

$$e \rightarrow 0.$$

Entretanto, permanece outra pergunta:

Com que rapidez essa memória deve atuar?

A resposta está diretamente relacionada ao valor de:

$$K_i.$$

Considere a mesma memória integral:

$$I = 4^\circ\text{C min.}$$

Para:

$$K_i = 0,1 \frac{\%}{^\circ\text{C min}},$$

temos:

$$u_I = 0,4\%.$$

Para:

$$K_i = 0,5 \frac{\%}{^{\circ}\text{C min}},$$

temos:

$$u_I = 2,0\%.$$

E para:

$$K_i = 1,0 \frac{\%}{^{\circ}\text{C min}},$$

resulta:

$$u_I = 4,0\%.$$

Portanto:

$$K_i \uparrow \rightarrow \text{maior efeito da memória acumulada}.$$

Um ganho integral pequeno produz uma correção acumulativa mais suave.

Um ganho integral maior faz com que a memória exerça influência mais intensa sobre o controlador.

1.33 Experiência Computacional: Diferentes Valores de K_i

Nesta nova experiência será mantido:

$$K_p = 2,0 \frac{\%}{^{\circ}\text{C}},$$

e serão comparados:

$$K_i = 0,1, \quad K_i = 0,5, \quad K_i = 1,0$$

em:

$$\frac{\%}{^{\circ}\text{C min}}.$$

Assim, somente a intensidade da ação integral será modificada.

1.33.1 Função para Simulação do Controlador PI

Podemos novamente organizar o modelo em uma função:

```
def simular_controlador_PI(Kp, Ki):
```

```

# Vetores
T = np.zeros(n)

erro = np.zeros(n)
erro_integral = np.zeros(n)

u_P = np.zeros(n)
u_I = np.zeros(n)
u_PI = np.zeros(n)

u_v = np.zeros(n)

# Condicao inicial
T[0] = 80.0
erro_integral[0] = 0.0

# Simulacao
for k in range(n - 1):

    # Erro
    erro[k] = T_sp - T[k]

    # Parcelas P e I
    u_P[k] = Kp * erro[k]

    u_I[k] = (
        Ki * erro_integral[k]
    )

    # Acao total
    u_PI[k] = (
        u_P[k] + u_I[k]
    )

    # Valvula
    u_v[k] = np.clip(
        u0 + u_PI[k],
        0.0,
        100.0
    )

    # Vazao de vapor
    m_v = Kv * u_v[k]

```

```

# Conversoes
m_v_kg_s = m_v / 60.0
m_in_kg_s = m_in[k] / 60.0

# Balanco de energia
Q_v = (
    m_v_kg_s * lambda_v
)

Q_processo = (
    m_in_kg_s
    * cp
    * (T[k] - T_in)
)

Q_perdas = (
    U_L
    * (T[k] - T_amb)
)

Q_liq = (
    Q_v
    - Q_processo
    - Q_perdas
)

# Modelo dinamico
dT_dt = Q_liq / C

T[k + 1] = (
    T[k]
    + dT_dt * dt
)

# Atualizacao da memoria
erro_integral[k + 1] = (
    erro_integral[k]
    + erro[k] * dt_min
)

# Ultimo instante
erro[-1] = T_sp - T[-1]

```

```

u_P[-1] = Kp * erro[-1]

u_I[-1] = (
    Ki * erro_integral[-1]
)

u_PI[-1] = (
    u_P[-1] + u_I[-1]
)

u_v[-1] = np.clip(
    u0 + u_PI[-1],
    0.0,
    100.0
)

# DataFrame
resultado = pd.DataFrame({
    "tempo_s": tempo,
    "tempo_min": tempo / 60.0,
    "T": T,
    "erro": erro,
    "erro_integral": erro_integral,
    "u_P": u_P,
    "u_I": u_I,
    "u_PI": u_PI,
    "u_v": u_v,
    "m_in": m_in
})

return resultado

```

1.33.2 Execução das Simulações

As três experiências são executadas por:

```

sim_Ki_01 = simular_controlador_PI(
    2.0, 0.1
)

sim_Ki_05 = simular_controlador_PI(
    2.0, 0.5
)

```

```
sim_Ki_10 = simular_controlador_PI(
2.0, 1.0
)
```

Agora podemos comparar diretamente as respostas.

1.33.3 Comparação das Respostas para Diferentes Valores de K_i

```
plt.figure()

plt.plot(
sim_Ki_01["tempo_min"],
sim_Ki_01["T"],
label="Ki = 0.1"
)

plt.plot(
sim_Ki_05["tempo_min"],
sim_Ki_05["T"],
label="Ki = 0.5"
)

plt.plot(
sim_Ki_10["tempo_min"],
sim_Ki_10["T"],
label="Ki = 1.0"
)

plt.axhline(
y=T_sp,
linestyle="--",
label="Setpoint"
)

plt.axvline(
x=10.0,
linestyle="--",
label="Perturbacao"
)

plt.xlabel("Tempo (min)")
plt.ylabel("Temperatura (graus Celsius)")
plt.title("Influencia do ganho integral")
plt.grid()
```

```
plt.legend()
```

```
plt.show()
```

Observe como a intensidade da ação integral modifica a velocidade com que o controlador tenta eliminar o erro produzido pela perturbação.

Para valores pequenos de:

$$K_i,$$

a memória exerce influência mais lentamente.

Para valores maiores, a contribuição integral cresce mais rapidamente.

Entretanto, uma ação integral excessivamente intensa pode produzir uma resposta demasiadamente agressiva e, dependendo da dinâmica do processo, favorecer oscilações ou ultrapassagens indesejadas do setpoint.

Portanto:

$$K_i \uparrow \rightarrow \text{eliminação potencialmente mais rápida do offset}$$

mas:

$$K_i \text{ excessivo} \rightarrow \text{possível degradação da resposta dinâmica}.$$

1.34 Sintonia como um Problema de Compromisso

As experiências realizadas permitem perceber que os parâmetros do controlador modificam diferentes características da resposta.

O ganho proporcional determina a intensidade da reação ao erro instantâneo.

O ganho integral determina a intensidade com que o erro acumulado influencia a ação de controle.

Assim:

$$K_p \rightarrow \text{reação ao erro presente}$$

e:

$$K_i \rightarrow \text{reação à permanência do erro}.$$

Escolher valores muito pequenos pode produzir respostas lentas ou desvios significativos.

Escolher valores excessivamente elevados pode produzir uma atuação desnecessariamente agressiva.

Portanto, sintonizar um controlador significa procurar um compromisso entre diferentes objetivos de desempenho.

Uma boa sintonia deve considerar não apenas a aproximação ao setpoint, mas também a dinâmica do processo e as limitações físicas dos equipamentos.

No caso do trocador de calor, por exemplo, a válvula possui limites físicos:

$$0\% \leq u_v \leq 100\%.$$

Além disso, mudanças excessivamente rápidas ou frequentes na posição da válvula podem ser indesejáveis operacionalmente.

Assim, a análise do controlador não deve considerar apenas:

$$T(t).$$

Também é necessário observar:

$$u_v(t).$$

Controle de processos é, portanto, um problema simultaneamente matemático, computacional e físico.